

DSA MASTERY ROADMAP

From Zero to Interview-Ready

30 Patterns · 180 LeetCode Problems · Complete Python Solutions

Covers: Sliding Window · Two Pointers · Binary Search · Hashing · Prefix Sum · Monotonic Stack
Heap · Intervals · Greedy · Linked Lists · Trees · Graphs · Union-Find
Topological Sort · Shortest Path · MST · Trie · Bit Manipulation · DP (1D, Knapsack, Grid, String)

How to Use This Document

This document is organized into 30 DSA patterns. Each pattern covers 6 LeetCode problems with complete teaching content:

- Problem Understanding — Plain-English explanation with examples
- Pattern Recognition — Why this problem fits the pattern; how to spot it in interviews
- Brute Force Solution — Naive approach with complexity analysis
- Optimized Solution — Optimal approach with step-by-step intuition
- Python Code — Clean, commented, production-quality solution
- Dry Run — Variable-by-variable walkthrough
- Complexity Analysis — Time and space with justification
- Interview Tips — Edge cases, follow-ups, and what interviewers look for
- Pattern Summary (at end of each pattern) — Core intuition, tricks, and variations

Recommended study order for beginners:

- Week 1–2: Sliding Window, Two Pointers, Hashing, Prefix Sum, Binary Search
- Week 3–4: Greedy, Intervals, Linked List, Tree DFS/BFS, Binary Search on Answer
- Week 5–6: Backtracking, Graph BFS/DFS, Heap, Union Find, Trie
- Week 7–8: Topological Sort, Shortest Path, MST, Bit Manipulation
- Week 9–10: DP (1D → Knapsack → Grid → String)

Pattern 1: Sliding Window

Problems: LC 3, LC 76, LC 209, LC 424, LC 567, LC 904

Problem 1: LC 3 — Longest Substring Without Repeating Characters

LeetCode #3 · leetcode.com/problems/

🔍 Problem Understanding

Find the length of the longest substring with all unique characters.

🔍 Pattern Recognition

This is a SLIDING WINDOW problem. Key signals: contiguous subarray/substring, find max/min length with condition, window shrinks/grows. The window approach avoids recomputing overlapping elements.

🔍 Brute Force Approach

Nested loops: for each starting index, iterate to find valid window. $O(n^2)$ or $O(n^3)$ time. Inefficient because we recompute overlapping sub-problems.

⚡ Optimized Solution

Use a set/dict to track characters in the current window. Expand right; when duplicate found, shrink from left.

🔍 Python Code

```
# LeetCode 3: Longest Substring Without Repeating Characters
# Pattern: Sliding Window

left=0, right=0, seen={}
for right in range(len(s)):
    while s[right] in seen:
        seen.remove(s[left]); left+=1
    seen.add(s[right])
    max_len = max(max_len, right-left+1)
```

🔍 Dry Run

$s = \text{'abcabcbb'} \rightarrow 3$ ('abc'). $s = \text{'bbbbbb'} \rightarrow 1$.

🔍 Complexity Analysis

$O(n)$ time, $O(k)$ space where k = charset size.

🔍 Interview Tips

Ask: 'Is the window fixed or variable size?' Start with brute force $O(n^2)$, then explain how window avoids recomputation. Edge cases: empty string, all same characters, $k >$ array length.

Problem 2: LC 76 — Minimum Window Substring

LeetCode #76 · leetcode.com/problems/

🔍 Problem Understanding

Find the smallest window in *s* that contains all chars of *t*.

🔍 Pattern Recognition

This is a SLIDING WINDOW problem. Key signals: contiguous subarray/substring, find max/min length with condition, window shrinks/grows. The window approach avoids recomputing overlapping elements.

🔍 Brute Force Approach

Nested loops: for each starting index, iterate to find valid window. $O(n^2)$ or $O(n^3)$ time. Inefficient because we recompute overlapping sub-problems.

⚡ Optimized Solution

Expand right to include all *t* chars, then shrink left as much as possible while still valid.

🔍 Python Code

```
# LeetCode 76: Minimum Window Substring
# Pattern: Sliding Window

from collections import Counter
need=Counter(t); missing=len(t)
l=0; res=''
for r,c in enumerate(s):
    if need[c]>0: missing-=1
    need[c]-=1
    if missing==0:
        while need[s[l]]<0: need[s[l]]+=1; l+=1
        if not res or r-l+1<len(res): res=s[l:r+1]
        need[s[l]]+=1; missing+=1; l+=1
return res
```

🔍 Dry Run

s='ADOBECODEBANC', *t*='ABC' → 'BANC'

🔍 Complexity Analysis

$O(n)$ time, $O(k)$ space.

🔍 Interview Tips

Ask: 'Is the window fixed or variable size?' Start with brute force $O(n^2)$, then explain how window avoids recomputation. Edge cases: empty string, all same characters, $k >$ array length.

Problem 3: LC 209 — Minimum Size Subarray Sum

LeetCode #209 · leetcode.com/problems/

🔍 Problem Understanding

Find the minimum length subarray with $\text{sum} \geq \text{target}$.

🔍 Pattern Recognition

This is a SLIDING WINDOW problem. Key signals: contiguous subarray/substring, find max/min length with condition, window shrinks/grows. The window approach avoids recomputing overlapping elements.

🔍 Brute Force Approach

Nested loops: for each starting index, iterate to find valid window. $O(n^2)$ or $O(n^3)$ time. Inefficient because we recompute overlapping sub-problems.

⚡ Optimized Solution

Expand right adding elements. When $\text{sum} \geq \text{target}$, shrink left and record length.

🔍 Python Code

```
# LeetCode 209: Minimum Size Subarray Sum
# Pattern: Sliding Window

left=0; cur=0; res=float('inf')
for right in range(len(nums)):
    cur+=nums[right]
    while cur>=target:
        res=min(res, right-left+1)
        cur-=nums[left]; left+=1
return res if res!=float('inf') else 0
```

🔍 Dry Run

target=7, nums=[2,3,1,2,4,3] → 2 ([4,3])

🔍 Complexity Analysis

$O(n)$ time, $O(1)$ space.

🔍 Interview Tips

Ask: 'Is the window fixed or variable size?' Start with brute force $O(n^2)$, then explain how window avoids recomputation. Edge cases: empty string, all same characters, $k > \text{array length}$.

Problem 4: LC 424 — Longest Repeating Character Replacement

LeetCode #424 · leetcode.com/problems/

🔍 Problem Understanding

Replace at most k chars to get longest substring with same character.

🔍 Pattern Recognition

This is a SLIDING WINDOW problem. Key signals: contiguous subarray/substring, find max/min length with condition, window shrinks/grows. The window approach avoids recomputing overlapping elements.

🔍 Brute Force Approach

Nested loops: for each starting index, iterate to find valid window. $O(n^2)$ or $O(n^3)$ time. Inefficient because we recompute overlapping sub-problems.

⚡ Optimized Solution

Track max frequency char in window. If $\text{window_size} - \text{max_freq} > k$, shrink left.

🔗 Python Code

```
# LeetCode 424: Longest Repeating Character Replacement
# Pattern: Sliding Window

count={}; max_freq=0; left=0; res=0
for right in range(len(s)):
    count[s[right]]=count.get(s[right],0)+1
    max_freq=max(max_freq,count[s[right]])
    if (right-left+1)-max_freq>k:
        count[s[left]]-=1; left+=1
    res=max(res,right-left+1)
return res
```

🔗 Dry Run

$s = \text{'AABABBA'}$, $k=1 \rightarrow 4$

🔗 Complexity Analysis

$O(n)$ time, $O(1)$ space (26 letters).

🔗 Interview Tips

Ask: 'Is the window fixed or variable size?' Start with brute force $O(n^2)$, then explain how window avoids recomputation. Edge cases: empty string, all same characters, $k > \text{array length}$.

Problem 5: LC 567 — Permutation in String

LeetCode #567 · leetcode.com/problems/

🔗 Problem Understanding

Check if any permutation of s_1 exists as a substring in s_2 .

🔗 Pattern Recognition

This is a SLIDING WINDOW problem. Key signals: contiguous subarray/substring, find max/min length with condition, window shrinks/grows. The window approach avoids recomputing overlapping elements.

🔗 Brute Force Approach

Nested loops: for each starting index, iterate to find valid window. $O(n^2)$ or $O(n^3)$ time. Inefficient because we recompute overlapping sub-problems.

⚡ Optimized Solution

Fixed window of size $\text{len}(s_1)$. Slide and compare frequency maps.

🔗 Python Code

```
# LeetCode 567: Permutation in String
# Pattern: Sliding Window

from collections import Counter
n=len(s1); need=Counter(s1)
cur=Counter(s2[:n])
if cur==need: return True
for i in range(n,len(s2)):
    cur[s2[i]]+=1
    cur[s2[i-n]]-=1
    if cur[s2[i-n]]==0: del cur[s2[i-n]]
    if cur==need: return True
return False
```

🔗 Dry Run

s1='ab', s2='eidbaooo' → True (s2[3:5]='ba')

🔗 Complexity Analysis

O(n) time, O(1) space.

🔗 Interview Tips

Ask: 'Is the window fixed or variable size?' Start with brute force O(n²), then explain how window avoids recomputation. Edge cases: empty string, all same characters, k > array length.

Problem 6: LC 904 — Fruit Into Baskets

LeetCode #904 · leetcode.com/problems/

🔗 Problem Understanding

Pick from a row of trees; can only carry 2 types of fruit. Max fruits = longest subarray with ≤2 distinct values.

🔗 Pattern Recognition

This is a SLIDING WINDOW problem. Key signals: contiguous subarray/substring, find max/min length with condition, window shrinks/grows. The window approach avoids recomputing overlapping elements.

🔗 Brute Force Approach

Nested loops: for each starting index, iterate to find valid window. O(n²) or O(n³) time. Inefficient because we recompute overlapping sub-problems.

⚡ Optimized Solution

Sliding window keeping at most 2 distinct fruit types in basket dict.

🔗 Python Code

```
# LeetCode 904: Fruit Into Baskets
# Pattern: Sliding Window

basket={}; left=0; res=0
for right in range(len(fruits)):
```

```
f=fruits[right]
basket[f]=basket.get(f,0)+1
while len(basket)>2:
    basket[fruits[left]]-=1
    if basket[fruits[left]]==0: del basket[fruits[left]]
    left+=1
res=max(res,right-left+1)
return res
```

🔗 Dry Run

fruits=[1,2,1,2,3,3,3] → 4

🔗 Complexity Analysis

O(n) time, O(1) space.

🔗 Interview Tips

Ask: 'Is the window fixed or variable size?' Start with brute force $O(n^2)$, then explain how window avoids recomputation. Edge cases: empty string, all same characters, $k > \text{array length}$.

Pattern 1 Summary: Sliding Window

Core intuition: maintain a window that satisfies constraints. Expand right to grow, shrink left when violated. Recognition tricks: 'longest/shortest subarray/substring with condition', 'at most K distinct'.

Variations: fixed window, variable window, multiple conditions.

Pattern 2: Two Pointers

Problems: LC 11, LC 15, LC 16, LC 18, LC 42, LC 167

Problem 1: LC 11 — Container With Most Water

LeetCode #11 · leetcode.com/problems/

🔖 Problem Understanding

Given heights of walls, find two that form the container holding most water.

🔖 Pattern Recognition

This is a TWO POINTERS problem. Key signals: sorted array, pair/triplet sum, optimization from both ends. Two pointers eliminate the need for nested loops.

🔖 Brute Force Approach

Nested loops: try every pair ($O(n^2)$) or triplet ($O(n^3)$). Sort the array but still check all combinations.

🔖 Optimized Solution

Start with widest container ($l=0, r=n-1$). Move the shorter wall inward (moving taller wall can only decrease area).

🔖 Python Code

```
# LeetCode 11: Container With Most Water
# Pattern: Two Pointers

l, r = 0, len(height) - 1; res = 0
while l < r:
    res = max(res, min(height[l], height[r]) * (r - l))
    if height[l] < height[r]: l += 1
    else: r -= 1
return res
```

🔖 Dry Run

height=[1,8,6,2,5,4,8,3,7] → 49

🔖 Complexity Analysis

$O(n)$ time, $O(1)$ space.

🔖 Interview Tips

Always sort first (unless the problem says otherwise). Clearly explain why moving the pointer you chose is correct. Edge cases: duplicates, negative numbers, empty array.

Problem 2: LC 15 — 3Sum

LeetCode #15 · leetcode.com/problems/

🔗 Problem Understanding

Find all unique triplets that sum to zero.

🔗 Pattern Recognition

This is a TWO POINTERS problem. Key signals: sorted array, pair/triplet sum, optimization from both ends. Two pointers eliminate the need for nested loops.

🔗 Brute Force Approach

Nested loops: try every pair ($O(n^2)$) or triplet ($O(n^3)$). Sort the array but still check all combinations.

⚡ Optimized Solution

Sort array. For each element i , use two pointers l, r to find pairs summing to $-nums[i]$. Skip duplicates.

🔗 Python Code

```
# LeetCode 15: 3Sum
# Pattern: Two Pointers

nums.sort(); res=[]
for i in range(len(nums)-2):
    if i>0 and nums[i]==nums[i-1]: continue
    l,r=i+1,len(nums)-1
    while l<r:
        s=nums[i]+nums[l]+nums[r]
        if s==0:
            res.append([nums[i],nums[l],nums[r]])
            while l<r and nums[l]==nums[l+1]: l+=1
            while l<r and nums[r]==nums[r-1]: r-=1
            l+=1; r-=1
        elif s<0: l+=1
        else: r-=1
    return res
```

🔗 Dry Run

$nums = [-1, 0, 1, 2, -1, -4] \rightarrow [[-1, -1, 2], [-1, 0, 1]]$

🔗 Complexity Analysis

$O(n^2)$ time, $O(1)$ space.

🔗 Interview Tips

Always sort first (unless the problem says otherwise). Clearly explain why moving the pointer you chose is correct. Edge cases: duplicates, negative numbers, empty array.

Problem 3: LC 16 — 3Sum Closest

LeetCode #16 · leetcode.com/problems/

🔗 Problem Understanding

Find the sum of three integers closest to target.

🔗 Pattern Recognition

This is a TWO POINTERS problem. Key signals: sorted array, pair/triplet sum, optimization from both ends. Two pointers eliminate the need for nested loops.

🔍 Brute Force Approach

Nested loops: try every pair ($O(n^2)$) or triplet ($O(n^3)$). Sort the array but still check all combinations.

⚡ Optimized Solution

Sort. For each i , two-pointer on the rest. Track closest sum seen.

📄 Python Code

```
# LeetCode 16: 3Sum Closest
# Pattern: Two Pointers

nums.sort(); closest=float('inf')
for i in range(len(nums)-2):
    l, r=i+1, len(nums)-1
    while l<r:
        s=nums[i]+nums[l]+nums[r]
        if abs(s-target)<abs(closest-target): closest=s
        if s<target: l+=1
        elif s>target: r-=1
        else: return s
    return closest
```

🔍 Dry Run

nums=[-1,2,1,-4], target=1 → 2

🔍 Complexity Analysis

$O(n^2)$ time, $O(1)$ space.

🔍 Interview Tips

Always sort first (unless the problem says otherwise). Clearly explain why moving the pointer you chose is correct. Edge cases: duplicates, negative numbers, empty array.

Problem 4: LC 18 — 4Sum

LeetCode #18 · leetcode.com/problems/

🔍 Problem Understanding

Find all unique quadruplets summing to target.

🔍 Pattern Recognition

This is a TWO POINTERS problem. Key signals: sorted array, pair/triplet sum, optimization from both ends. Two pointers eliminate the need for nested loops.

🔍 Brute Force Approach

Nested loops: try every pair ($O(n^2)$) or triplet ($O(n^3)$). Sort the array but still check all combinations.

⚡ Optimized Solution

Sort. Two nested loops for first two elements, then two pointers for last two. Skip duplicates at each level.

Python Code

```
# LeetCode 18: 4Sum
# Pattern: Two Pointers

nums.sort(); res=[]; n=len(nums)
for i in range(n-3):
    if i>0 and nums[i]==nums[i-1]: continue
    for j in range(i+1,n-2):
        if j>i+1 and nums[j]==nums[j-1]: continue
        l,r=j+1,n-1
        while l<r:
            s=nums[i]+nums[j]+nums[l]+nums[r]
            if s==target:
                res.append([nums[i],nums[j],nums[l],nums[r]])
                while l<r and nums[l]==nums[l+1]: l+=1
                while l<r and nums[r]==nums[r-1]: r-=1
                l+=1; r-=1
            elif s<target: l+=1
            else: r-=1
return res
```

Dry Run

nums=[1,0,-1,0,-2,2], target=0 → [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]

Complexity Analysis

$O(n^3)$ time, $O(1)$ space.

Interview Tips

Always sort first (unless the problem says otherwise). Clearly explain why moving the pointer you chose is correct. Edge cases: duplicates, negative numbers, empty array.

Problem 5: LC 42 — Trapping Rain Water

LeetCode #42 · leetcode.com/problems/

Problem Understanding

Given elevation map, compute how much water it can trap.

Pattern Recognition

This is a TWO POINTERS problem. Key signals: sorted array, pair/triplet sum, optimization from both ends. Two pointers eliminate the need for nested loops.

Brute Force Approach

Nested loops: try every pair ($O(n^2)$) or triplet ($O(n^3)$). Sort the array but still check all combinations.

Optimized Solution

At each position $\text{water} = \min(\text{max_left}, \text{max_right}) - \text{height}[i]$. Use two pointers tracking max from each side.

Python Code

```
# LeetCode 42: Trapping Rain Water
# Pattern: Two Pointers

l, r = 0, len(height) - 1; lmax = rmax = res = 0
while l < r:
    if height[l] <= height[r]:
        if height[l] >= lmax: lmax = height[l]
        else: res += lmax - height[l]
        l += 1
    else:
        if height[r] >= rmax: rmax = height[r]
        else: res += rmax - height[r]
        r -= 1
return res
```

Dry Run

height=[0,1,0,2,1,0,1,3,2,1,2,1] → 6

Complexity Analysis

O(n) time, O(1) space.

Interview Tips

Always sort first (unless the problem says otherwise). Clearly explain why moving the pointer you chose is correct. Edge cases: duplicates, negative numbers, empty array.

Problem 6: LC 167 — Two Sum II

LeetCode #167 · leetcode.com/problems/

Problem Understanding

Sorted array; find two numbers summing to target. Return 1-indexed positions.

Pattern Recognition

This is a TWO POINTERS problem. Key signals: sorted array, pair/triplet sum, optimization from both ends. Two pointers eliminate the need for nested loops.

Brute Force Approach

Nested loops: try every pair ($O(n^2)$) or triplet ($O(n^3)$). Sort the array but still check all combinations.

Optimized Solution

Classic two-pointer on sorted array: if $\text{sum} < \text{target}$ move left pointer right; if $\text{sum} > \text{target}$ move right pointer left.

Python Code

```
# LeetCode 167: Two Sum II
# Pattern: Two Pointers

l, r = 0, len(numbers) - 1
while l < r:
    s = numbers[l] + numbers[r]
    if s == target: return [l+1, r+1]
    elif s < target: l += 1
    else: r -= 1
```

🔍 Dry Run

numbers=[2,7,11,15], target=9 → [1,2]

🔍 Complexity Analysis

O(n) time, O(1) space.

🔍 Interview Tips

Always sort first (unless the problem says otherwise). Clearly explain why moving the pointer you chose is correct. Edge cases: duplicates, negative numbers, empty array.

Pattern 2 Summary: Two Pointers

Core intuition: use two indices to avoid nested loops on sorted data. Recognition tricks: 'pair/triplet sum', 'container', 'reverse'. Variations: opposite ends, same direction (fast/slow), partition.

Pattern 3: Fast/Slow Pointers (Linked List)

Problems: LC 141, LC 142, LC 19, LC 876, LC 160, LC 234

Problem 1: LC 141 — Linked List Cycle

LeetCode #141 · leetcode.com/problems/

🔍 Problem Understanding

Detect if a linked list has a cycle.

🔍 Pattern Recognition

This is a FAST/SLOW POINTERS problem. Key signals: linked list cycle detection, finding middle, palindrome check. Floyd's algorithm is the go-to technique.

🔍 Brute Force Approach

Use a hash set to store visited nodes. If a node is seen again → cycle. $O(n)$ space. Floyd's approach saves space.

⚡ Optimized Solution

Floyd's cycle detection: slow moves 1 step, fast moves 2. If they meet, there's a cycle.

🔍 Python Code

```
# LeetCode 141: Linked List Cycle
# Pattern: Fast/Slow Pointers (Linked List)

slow=fast=head
while fast and fast.next:
    slow=slow.next; fast=fast.next.next
    if slow==fast: return True
return False
```

🔍 Dry Run

head=[3,2,0,-4], pos=1 → True

🔍 Complexity Analysis

$O(n)$ time, $O(1)$ space.

🔍 Interview Tips

Explain Floyd's cycle detection intuitively: fast pointer 'laps' slow in a cycle. Edge cases: empty list, single node, no cycle.

Problem 2: LC 142 — Linked List Cycle II

LeetCode #142 · leetcode.com/problems/

🔍 Problem Understanding

Find the node where the cycle begins.

🔍 Pattern Recognition

This is a FAST/SLOW POINTERS problem. Key signals: linked list cycle detection, finding middle, palindrome check. Floyd's algorithm is the go-to technique.

🔍 Brute Force Approach

Use a hash set to store visited nodes. If a node is seen again → cycle. $O(n)$ space. Floyd's approach saves space.

⚡ Optimized Solution

Detect cycle with Floyd's. Then reset one pointer to head. Move both 1 step at a time; they meet at cycle entry.

🔍 Python Code

```
# LeetCode 142: Linked List Cycle II
# Pattern: Fast/Slow Pointers (Linked List)

slow=fast=head
while fast and fast.next:
    slow=slow.next; fast=fast.next.next
    if slow==fast:
        slow2=head
        while slow!=slow2: slow=slow.next; slow2=slow2.next
        return slow
return None
```

🔍 Dry Run

head=[3,2,0,-4], pos=1 → node with val 2

🔍 Complexity Analysis

$O(n)$ time, $O(1)$ space.

🔍 Interview Tips

Explain Floyd's cycle detection intuitively: fast pointer 'laps' slow in a cycle. Edge cases: empty list, single node, no cycle.

Problem 3: LC 19 — Remove Nth Node From End of List

LeetCode #19 · leetcode.com/problems/

🔍 Problem Understanding

Remove the nth node from the end of a linked list.

🔍 Pattern Recognition

This is a FAST/SLOW POINTERS problem. Key signals: linked list cycle detection, finding middle, palindrome check. Floyd's algorithm is the go-to technique.

🔍 Brute Force Approach

Use a hash set to store visited nodes. If a node is seen again → cycle. $O(n)$ space. Floyd's approach saves space.

⚡ Optimized Solution

Move fast pointer $n+1$ steps ahead. Then move both until fast reaches end. `slow.next` is the target.

🔗 Python Code

```
# LeetCode 19: Remove Nth Node From End of List
# Pattern: Fast/Slow Pointers (Linked List)

dummy=ListNode(0,head); slow=fast=dummy
for _ in range(n+1): fast=fast.next
while fast: slow=slow.next; fast=fast.next
slow.next=slow.next.next
return dummy.next
```

🔗 Dry Run

`head=[1,2,3,4,5]`, `n=2` → `[1,2,3,5]`

🔗 Complexity Analysis

$O(n)$ time, $O(1)$ space.

🔗 Interview Tips

Explain Floyd's cycle detection intuitively: fast pointer 'laps' slow in a cycle. Edge cases: empty list, single node, no cycle.

Problem 4: LC 876 — Middle of the Linked List

LeetCode #876 · [leetcode.com/problems/](https://leetcode.com/problems/middle-of-the-linked-list/)

🔗 Problem Understanding

Find the middle node. If even length, return second middle.

🔗 Pattern Recognition

This is a FAST/SLOW POINTERS problem. Key signals: linked list cycle detection, finding middle, palindrome check. Floyd's algorithm is the go-to technique.

🔗 Brute Force Approach

Use a hash set to store visited nodes. If a node is seen again → cycle. $O(n)$ space. Floyd's approach saves space.

⚡ Optimized Solution

Slow moves 1 step, fast moves 2. When fast reaches end, slow is at middle.

🔗 Python Code

```
# LeetCode 876: Middle of the Linked List
# Pattern: Fast/Slow Pointers (Linked List)

slow=fast=head
```

```
while fast and fast.next:
    slow=slow.next; fast=fast.next.next
return slow
```

🔍 Dry Run

head=[1,2,3,4,5] → node 3; head=[1,2,3,4,5,6] → node 4

🔍 Complexity Analysis

O(n) time, O(1) space.

🔍 Interview Tips

Explain Floyd's cycle detection intuitively: fast pointer 'laps' slow in a cycle. Edge cases: empty list, single node, no cycle.

Problem 5: LC 160 — Intersection of Two Linked Lists

LeetCode #160 · leetcode.com/problems/

🔍 Problem Understanding

Find the node where two linked lists intersect.

🔍 Pattern Recognition

This is a FAST/SLOW POINTERS problem. Key signals: linked list cycle detection, finding middle, palindrome check. Floyd's algorithm is the go-to technique.

🔍 Brute Force Approach

Use a hash set to store visited nodes. If a node is seen again → cycle. O(n) space. Floyd's approach saves space.

⚡ Optimized Solution

Two pointers a,b. When one reaches end, redirect to other list's head. They meet at intersection after traversing combined length.

🔍 Python Code

```
# LeetCode 160: Intersection of Two Linked Lists
# Pattern: Fast/Slow Pointers (Linked List)

a,b=headA,headB
while a!=b:
    a=a.next if a else headB
    b=b.next if b else headA
return a
```

🔍 Dry Run

listA=[4,1,8,4,5], listB=[5,6,1,8,4,5], intersect at node 8

🔍 Complexity Analysis

O(m+n) time, O(1) space.

🔖 Interview Tips

Explain Floyd's cycle detection intuitively: fast pointer 'laps' slow in a cycle. Edge cases: empty list, single node, no cycle.

Problem 6: LC 234 — Palindrome Linked List

LeetCode #234 · leetcode.com/problems/

🔖 Problem Understanding

Check if a linked list is a palindrome.

🔖 Pattern Recognition

This is a FAST/SLOW POINTERS problem. Key signals: linked list cycle detection, finding middle, palindrome check. Floyd's algorithm is the go-to technique.

🔖 Brute Force Approach

Use a hash set to store visited nodes. If a node is seen again → cycle. $O(n)$ space. Floyd's approach saves space.

⚡ Optimized Solution

Find middle (fast/slow). Reverse second half. Compare first and second halves.

🔖 Python Code

```
# LeetCode 234: Palindrome Linked List
# Pattern: Fast/Slow Pointers (Linked List)

# Find middle
slow=fast=head
while fast and fast.next: slow=slow.next; fast=fast.next.next
# Reverse second half
prev=None
while slow: slow.next,prev,slow=prev,slow,slow.next
# Compare
l,r=head,prev
while r:
    if l.val!=r.val: return False
    l=l.next; r=r.next
return True
```

🔖 Dry Run

head=[1,2,2,1] → True; head=[1,2] → False

🔖 Complexity Analysis

$O(n)$ time, $O(1)$ space.

🔖 Interview Tips

Explain Floyd's cycle detection intuitively: fast pointer 'laps' slow in a cycle. Edge cases: empty list, single node, no cycle.

Pattern 3 Summary: Fast/Slow Pointers (Linked List)

Apply the core pattern technique consistently.

Pattern 4: Binary Search on Sorted Data

Problems: LC 704, LC 35, LC 34, LC 33, LC 153, LC 162

Problem 1: LC 704 — Binary Search

LeetCode #704 · leetcode.com/problems/

🔖 Problem Understanding

Classic binary search: find target in sorted array. Return index or -1.

🔖 Pattern Recognition

This is a BINARY SEARCH problem. Key signals: sorted data, find position/boundary, $O(\log n)$ required. Identify whether you're searching for exact value or boundary.

🔖 Brute Force Approach

Linear scan: check every element one by one. $O(n)$ time. Doesn't use the sorted property at all.

⚡ Optimized Solution

Maintain lo,hi. Check mid. Shrink half based on comparison.

🔖 Python Code

```
# LeetCode 704: Binary Search
# Pattern: Binary Search on Sorted Data

lo,hi=0,len(nums)-1
while lo<=hi:
    mid=(lo+hi)//2
    if nums[mid]==target: return mid
    elif nums[mid]<target: lo=mid+1
    else: hi=mid-1
return -1
```

🔖 Dry Run

nums=[-1,0,3,5,9,12], target=9 → 4

🔖 Complexity Analysis

$O(\log n)$ time, $O(1)$ space.

🔖 Interview Tips

Define your invariant clearly ($lo \leq target \leq hi$ vs $lo < target \leq hi$). Explain why you move $lo = mid+1$ vs $hi = mid$. Edge cases: empty array, single element, target not present.

Problem 2: LC 35 — Search Insert Position

LeetCode #35 · leetcode.com/problems/

🔍 Problem Understanding

Find where target should be inserted in sorted array (or its index).

🔍 Pattern Recognition

This is a BINARY SEARCH problem. Key signals: sorted data, find position/boundary, $O(\log n)$ required. Identify whether you're searching for exact value or boundary.

🔍 Brute Force Approach

Linear scan: check every element one by one. $O(n)$ time. Doesn't use the sorted property at all.

⚡ Optimized Solution

Standard binary search. Return lo at the end — it's the insertion point.

🔍 Python Code

```
# LeetCode 35: Search Insert Position
# Pattern: Binary Search on Sorted Data

lo, hi = 0, len(nums) - 1
while lo <= hi:
    mid = (lo + hi) // 2
    if nums[mid] == target: return mid
    elif nums[mid] < target: lo = mid + 1
    else: hi = mid - 1
return lo
```

🔍 Dry Run

nums=[1,3,5,6], target=5 → 2; target=2 → 1

🔍 Complexity Analysis

$O(\log n)$ time, $O(1)$ space.

🔍 Interview Tips

Define your invariant clearly ($lo \leq target \leq hi$ vs $lo < target \leq hi$). Explain why you move $lo = mid + 1$ vs $hi = mid$. Edge cases: empty array, single element, target not present.

Problem 3: LC 34 — Find First and Last Position of Element in Sorted Array

LeetCode #34 · leetcode.com/problems/

🔍 Problem Understanding

Find start and end indices of target. Return [-1,-1] if not found.

🔍 Pattern Recognition

This is a BINARY SEARCH problem. Key signals: sorted data, find position/boundary, $O(\log n)$ required. Identify whether you're searching for exact value or boundary.

🔍 Brute Force Approach

Linear scan: check every element one by one. $O(n)$ time. Doesn't use the sorted property at all.

⚡ Optimized Solution

Two separate binary searches: one biased left (find first), one biased right (find last).

🔗 Python Code

```
# LeetCode 34: Find First and Last Position of Element in Sorted Array
# Pattern: Binary Search on Sorted Data

def find(bias):
    lo, hi, idx = 0, len(nums) - 1, -1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            idx = mid
            if bias == 'left': hi = mid - 1
            else: lo = mid + 1
        elif nums[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return idx
return [find('left'), find('right')]
```

🔗 Dry Run

nums=[5,7,7,8,8,10], target=8 → [3,4]

🔗 Complexity Analysis

$O(\log n)$ time, $O(1)$ space.

🔗 Interview Tips

Define your invariant clearly ($lo \leq target \leq hi$ vs $lo < target \leq hi$). Explain why you move $lo = mid + 1$ vs $hi = mid$. Edge cases: empty array, single element, target not present.

Problem 4: LC 33 — Search in Rotated Sorted Array

LeetCode #33 · leetcode.com/problems/

🔗 Problem Understanding

Array was sorted then rotated at some pivot. Find target.

🔗 Pattern Recognition

This is a BINARY SEARCH problem. Key signals: sorted data, find position/boundary, $O(\log n)$ required. Identify whether you're searching for exact value or boundary.

🔗 Brute Force Approach

Linear scan: check every element one by one. $O(n)$ time. Doesn't use the sorted property at all.

⚡ Optimized Solution

At mid, one half is always sorted. Check if target falls in sorted half; search there, else search other half.

🔗 Python Code

```
# LeetCode 33: Search in Rotated Sorted Array
# Pattern: Binary Search on Sorted Data

lo,hi=0,len(nums)-1
while lo<=hi:
    mid=(lo+hi)//2
    if nums[mid]==target: return mid
    if nums[lo]<=nums[mid]: # left sorted
        if nums[lo]<=target<nums[mid]: hi=mid-1
        else: lo=mid+1
    else: # right sorted
        if nums[mid]<target<=nums[hi]: lo=mid+1
        else: hi=mid-1
return -1
```

🔍 Dry Run

nums=[4,5,6,7,0,1,2], target=0 → 4

🔍 Complexity Analysis

$O(\log n)$ time, $O(1)$ space.

🔍 Interview Tips

Define your invariant clearly ($lo \leq target \leq hi$ vs $lo < target \leq hi$). Explain why you move $lo = mid+1$ vs $hi = mid$. Edge cases: empty array, single element, target not present.

Problem 5: LC 153 — Find Minimum in Rotated Sorted Array

LeetCode #153 · leetcode.com/problems/

🔍 Problem Understanding

Find the minimum element in a rotated sorted array.

🔍 Pattern Recognition

This is a BINARY SEARCH problem. Key signals: sorted data, find position/boundary, $O(\log n)$ required. Identify whether you're searching for exact value or boundary.

🔍 Brute Force Approach

Linear scan: check every element one by one. $O(n)$ time. Doesn't use the sorted property at all.

⚡ Optimized Solution

If $mid > hi$, minimum is in right half. Otherwise left half (including mid).

🔍 Python Code

```
# LeetCode 153: Find Minimum in Rotated Sorted Array
# Pattern: Binary Search on Sorted Data

lo,hi=0,len(nums)-1
while lo<hi:
    mid=(lo+hi)//2
    if nums[mid]>nums[hi]: lo=mid+1
```



```
    else: hi=mid
return nums[lo]
```

🔍 Dry Run

nums=[3,4,5,1,2] → 1

🔍 Complexity Analysis

$O(\log n)$ time, $O(1)$ space.

🔍 Interview Tips

Define your invariant clearly ($lo \leq target \leq hi$ vs $lo < target \leq hi$). Explain why you move $lo = mid+1$ vs $hi = mid$. Edge cases: empty array, single element, target not present.

Problem 6: LC 162 — Find Peak Element

LeetCode #162 · leetcode.com/problems/

🔍 Problem Understanding

Find any peak element (greater than neighbors). Array has no equal adjacent elements.

🔍 Pattern Recognition

This is a BINARY SEARCH problem. Key signals: sorted data, find position/boundary, $O(\log n)$ required. Identify whether you're searching for exact value or boundary.

🔍 Brute Force Approach

Linear scan: check every element one by one. $O(n)$ time. Doesn't use the sorted property at all.

⚡ Optimized Solution

If $nums[mid] < nums[mid+1]$, peak is in right half. Otherwise left half.

🔍 Python Code

```
# LeetCode 162: Find Peak Element
# Pattern: Binary Search on Sorted Data

lo, hi = 0, len(nums) - 1
while lo < hi:
    mid = (lo + hi) // 2
    if nums[mid] < nums[mid + 1]: lo = mid + 1
    else: hi = mid
return lo
```

🔍 Dry Run

nums=[1,2,3,1] → 2

🔍 Complexity Analysis

$O(\log n)$ time, $O(1)$ space.

🔍 Interview Tips

Define your invariant clearly ($lo \leq target \leq hi$ vs $lo < target \leq hi$). Explain why you move $lo = mid + 1$ vs $hi = mid$. Edge cases: empty array, single element, target not present.

Pattern 4 Summary: Binary Search on Sorted Data

Core intuition: eliminate half the search space each step. Recognition tricks: sorted input + $O(\log n)$ requirement. The key skill: determine if answer is in left or right half.

Pattern 5: Binary Search on Answer

Problems: LC 875, LC 1011, LC 410, LC 774, LC 1283, LC 1482

Problem 1: LC 875 — Koko Eating Bananas

LeetCode #875 · leetcode.com/problems/

🔗 Problem Understanding

Find the minimum eating speed k so Koko can eat all piles within h hours.

🔗 Pattern Recognition

This is BINARY SEARCH ON ANSWER. Key signals: 'minimize the maximum' or 'maximize the minimum', feasibility function exists. Search the answer space, not the data.

🔗 Brute Force Approach

Try every possible answer from minimum to maximum. $O(\text{max_val} * n)$ time. Binary search reduces to $O(\log(\text{max_val}) * n)$.

⚡ Optimized Solution

Binary search on speed k [$1.. \text{max}(\text{piles})$]. Check if total hours with speed $k \leq h$.

🔗 Python Code

```
# LeetCode 875: Koko Eating Bananas
# Pattern: Binary Search on Answer

import math
def feasible(k):
    return sum(math.ceil(p/k) for p in piles) <= h
lo, hi = 1, max(piles)
while lo < hi:
    mid = (lo + hi) // 2
    if feasible(mid): hi = mid
    else: lo = mid + 1
return lo
```

🔗 Dry Run

piles=[3,6,7,11], h=8 → 4

🔗 Complexity Analysis

$O(n \log m)$ where $m = \text{max}(\text{piles})$.

🔗 Interview Tips

Explain: 'I'm binary searching the answer space, not the array.' Write feasibility function first, then the binary search wrapper. Edge cases: minimum possible answer.

Problem 2: LC 1011 — Capacity to Ship Packages Within D Days

LeetCode #1011 · leetcode.com/problems/

🔍 Problem Understanding

Find minimum ship capacity to ship all packages in d days.

🔍 Pattern Recognition

This is BINARY SEARCH ON ANSWER. Key signals: 'minimize the maximum' or 'maximize the minimum', feasibility function exists. Search the answer space, not the data.

🔍 Brute Force Approach

Try every possible answer from minimum to maximum. $O(\text{max_val} * n)$ time. Binary search reduces to $O(\log(\text{max_val}) * n)$.

⚡ Optimized Solution

BS on capacity $[\text{max}(\text{weights}).. \text{sum}(\text{weights})]$. Check if packages fit in d days.

🔍 Python Code

```
# LeetCode 1011: Capacity to Ship Packages Within D Days
# Pattern: Binary Search on Answer

def feasible(cap):
    days=1; cur=0
    for w in weights:
        if cur+w>cap: days+=1; cur=0
        cur+=w
    return days<=d
lo,hi=max(weights),sum(weights)
while lo<hi:
    mid=(lo+hi)//2
    if feasible(mid): hi=mid
    else: lo=mid+1
return lo
```

🔍 Dry Run

weights=[1,2,3,4,5,6,7,8,9,10], d=5 → 15

🔍 Complexity Analysis

$O(n \log(\text{sum}-\text{max}))$ time.

🔍 Interview Tips

Explain: 'I'm binary searching the answer space, not the array.' Write feasibility function first, then the binary search wrapper. Edge cases: minimum possible answer.

Problem 3: LC 410 — Split Array Largest Sum

LeetCode #410 · leetcode.com/problems/

🔍 Problem Understanding

Split array into m subarrays to minimize the largest subarray sum.

🔍 Pattern Recognition

This is BINARY SEARCH ON ANSWER. Key signals: 'minimize the maximum' or 'maximize the minimum', feasibility function exists. Search the answer space, not the data.

🔍 Brute Force Approach

Try every possible answer from minimum to maximum. $O(\text{max_val} * n)$ time. Binary search reduces to $O(\log(\text{max_val}) * n)$.

⚡ Optimized Solution

Binary search on answer [max..sum]. Check how many parts needed for a given max sum.

🔍 Python Code

```
# LeetCode 410: Split Array Largest Sum
# Pattern: Binary Search on Answer

def feasible(limit):
    parts=1; cur=0
    for n in nums:
        if cur+n>limit: parts+=1; cur=0
        cur+=n
    return parts<=m
lo,hi=max(nums),sum(nums)
while lo<hi:
    mid=(lo+hi)//2
    if feasible(mid): hi=mid
    else: lo=mid+1
return lo
```

🔍 Dry Run

nums=[7,2,5,10,8], m=2 → 18

🔍 Complexity Analysis

$O(n \log(\text{sum}-\text{max}))$ time.

🔍 Interview Tips

Explain: 'I'm binary searching the answer space, not the array.' Write feasibility function first, then the binary search wrapper. Edge cases: minimum possible answer.

Problem 4: LC 774 — Minimize Max Distance to Gas Station

LeetCode #774 · leetcode.com/problems/

🔍 Problem Understanding

Add k gas stations to minimize the maximum gap between adjacent stations.

🔍 Pattern Recognition

This is BINARY SEARCH ON ANSWER. Key signals: 'minimize the maximum' or 'maximize the minimum', feasibility function exists. Search the answer space, not the data.

🔍 Brute Force Approach

Try every possible answer from minimum to maximum. $O(\text{max_val} * n)$ time. Binary search reduces to $O(\log(\text{max_val}) * n)$.

⚡ Optimized Solution

Binary search on the answer (maximum gap). For a given gap D , count how many stations needed.

🔍 Python Code

```
# LeetCode 774: Minimize Max Distance to Gas Station
# Pattern: Binary Search on Answer

import math
def count(D):
    return sum(math.ceil((stations[i+1]-stations[i])/D)-1
               for i in range(len(stations)-1))
lo,hi=0.0,stations[-1]-stations[0]
for _ in range(100): # float BS
    mid=(lo+hi)/2
    if count(mid)<=k: hi=mid
    else: lo=mid
return lo
```

🔍 Dry Run

stations=[1,2,3,4,5,6,7,8,9,10], k=9 → 0.5

🔍 Complexity Analysis

$O(n \log(1/\epsilon))$ time.

🔍 Interview Tips

Explain: 'I'm binary searching the answer space, not the array.' Write feasibility function first, then the binary search wrapper. Edge cases: minimum possible answer.

Problem 5: LC 1283 — Find the Smallest Divisor Given a Threshold

LeetCode #1283 · leetcode.com/problems/

🔍 Problem Understanding

Find smallest divisor such that sum of $\text{ceil}(\text{num}/\text{divisor}) \leq \text{threshold}$.

🔍 Pattern Recognition

This is BINARY SEARCH ON ANSWER. Key signals: 'minimize the maximum' or 'maximize the minimum', feasibility function exists. Search the answer space, not the data.

🔍 Brute Force Approach

Try every possible answer from minimum to maximum. $O(\text{max_val} * n)$ time. Binary search reduces to $O(\log(\text{max_val}) * n)$.

⚡ Optimized Solution

Binary search on divisor $[1..max(nums)]$. Check feasibility.

🔗 Python Code

```
# LeetCode 1283: Find the Smallest Divisor Given a Threshold
# Pattern: Binary Search on Answer

import math
def feasible(d):
    return sum(math.ceil(n/d) for n in nums) <= threshold
lo, hi = 1, max(nums)
while lo < hi:
    mid = (lo + hi) // 2
    if feasible(mid): hi = mid
    else: lo = mid + 1
return lo
```

🔗 Dry Run

nums=[1,2,5,9], threshold=6 → 5

🔗 Complexity Analysis

$O(n \log m)$ time.

🔗 Interview Tips

Explain: 'I'm binary searching the answer space, not the array.' Write feasibility function first, then the binary search wrapper. Edge cases: minimum possible answer.

Problem 6: LC 1482 — Minimum Number of Days to Make m Bouquets

LeetCode #1482 · leetcode.com/problems/

🔗 Problem Understanding

Each bouquet needs k adjacent bloomed flowers. Find min days to make m bouquets.

🔗 Pattern Recognition

This is BINARY SEARCH ON ANSWER. Key signals: 'minimize the maximum' or 'maximize the minimum', feasibility function exists. Search the answer space, not the data.

🔗 Brute Force Approach

Try every possible answer from minimum to maximum. $O(max_val * n)$ time. Binary search reduces to $O(\log(max_val) * n)$.

⚡ Optimized Solution

Binary search on days. Check if m bouquets possible by day d.

🔗 Python Code

```
# LeetCode 1482: Minimum Number of Days to Make m Bouquets
# Pattern: Binary Search on Answer
```

```

def feasible(day):
    bouquets=cur=0
    for b in bloomDay:
        if b<=day: cur+=1
        else: bouquets+=cur//k; cur=0
    bouquets+=cur//k
    return bouquets>=m
if len(bloomDay)<m*k: return -1
lo,hi=min(bloomDay),max(bloomDay)
while lo<hi:
    mid=(lo+hi)//2
    if feasible(mid): hi=mid
    else: lo=mid+1
return lo

```

🔗 Dry Run

bloomDay=[1,10,3,10,2], m=3, k=1 → 3

🔗 Complexity Analysis

$O(n \log(\max - \min))$ time.

🔗 Interview Tips

Explain: 'I'm binary searching the answer space, not the array.' Write feasibility function first, then the binary search wrapper. Edge cases: minimum possible answer.

Pattern 5 Summary: Binary Search on Answer

Core intuition: if we can check 'is X feasible?', binary search the answer. Recognition tricks: 'minimize the maximum', 'maximize the minimum', monotone feasibility. The feasibility function is the key.

Pattern 6: Hashing / Frequency Maps

Problems: LC 1, LC 217, LC 242, LC 49, LC 128, LC 347

Problem 1: LC 1 — Two Sum

LeetCode #1 · leetcode.com/problems/

🔍 Problem Understanding

Find two numbers in array that sum to target. Return their indices.

🔍 Pattern Recognition

This is a HASHING problem. Key signals: $O(1)$ lookup needed, frequency count, complement search. Hash maps trade space for time.

🔍 Brute Force Approach

Nested loops: for each element, scan rest of array to find complement or duplicate. $O(n^2)$ time.

🚀 Optimized Solution

Store each number's index in a hash map. For each num, check if complement (target-num) exists.

🔍 Python Code

```
# LeetCode 1: Two Sum
# Pattern: Hashing / Frequency Maps

seen={}
for i,n in enumerate(nums):
    comp=target-n
    if comp in seen: return [seen[comp],i]
    seen[n]=i
```

🔍 Dry Run

nums=[2,7,11,15], target=9 → [0,1]

🔍 Complexity Analysis

$O(n)$ time, $O(n)$ space.

🔍 Interview Tips

Trade-off: $O(n)$ space for $O(1)$ lookup. Interviewers love follow-up: 'Can you do it with $O(1)$ space?' Edge cases: empty array, all duplicates, target sum with negative numbers.

Problem 2: LC 217 — Contains Duplicate

LeetCode #217 · leetcode.com/problems/

🔍 Problem Understanding

Return True if any value appears at least twice.

🔍 Pattern Recognition

This is a HASHING problem. Key signals: $O(1)$ lookup needed, frequency count, complement search. Hash maps trade space for time.

🔍 Brute Force Approach

Nested loops: for each element, scan rest of array to find complement or duplicate. $O(n^2)$ time.

⚡ Optimized Solution

Add each element to a set. If it's already there, return True.

🔍 Python Code

```
# LeetCode 217: Contains Duplicate
# Pattern: Hashing / Frequency Maps

seen=set()
for n in nums:
    if n in seen: return True
    seen.add(n)
return False
```

🔍 Dry Run

nums=[1,2,3,1] → True

🔍 Complexity Analysis

$O(n)$ time, $O(n)$ space.

🔍 Interview Tips

Trade-off: $O(n)$ space for $O(1)$ lookup. Interviewers love follow-up: 'Can you do it with $O(1)$ space?' Edge cases: empty array, all duplicates, target sum with negative numbers.

Problem 3: LC 242 — Valid Anagram

LeetCode #242 · leetcode.com/problems/

🔍 Problem Understanding

Check if t is an anagram of s (same characters, possibly reordered).

🔍 Pattern Recognition

This is a HASHING problem. Key signals: $O(1)$ lookup needed, frequency count, complement search. Hash maps trade space for time.

🔍 Brute Force Approach

Nested loops: for each element, scan rest of array to find complement or duplicate. $O(n^2)$ time.

⚡ Optimized Solution

Count character frequencies in both strings; compare.

Python Code

```
# LeetCode 242: Valid Anagram
# Pattern: Hashing / Frequency Maps

from collections import Counter
return Counter(s)==Counter(t)
```

Dry Run

s='anagram', t='nagaram' → True

Complexity Analysis

O(n) time, O(1) space (fixed alphabet).

Interview Tips

Trade-off: O(n) space for O(1) lookup. Interviewers love follow-up: 'Can you do it with O(1) space?' Edge cases: empty array, all duplicates, target sum with negative numbers.

Problem 4: LC 49 — Group Anagrams

LeetCode #49 · leetcode.com/problems/

Problem Understanding

Group strings that are anagrams of each other.

Pattern Recognition

This is a HASHING problem. Key signals: O(1) lookup needed, frequency count, complement search. Hash maps trade space for time.

Brute Force Approach

Nested loops: for each element, scan rest of array to find complement or duplicate. O(n²) time.

⚡ Optimized Solution

Sort each word → use sorted word as key in hash map → group original words.

Python Code

```
# LeetCode 49: Group Anagrams
# Pattern: Hashing / Frequency Maps

from collections import defaultdict
groups=defaultdict(list)
for word in strs:
    groups[tuple(sorted(word))].append(word)
return list(groups.values())
```

Dry Run

strs=['eat','tea','tan','ate','nat','bat'] → [['eat','tea','ate'], ['tan','nat'], ['bat']]

Complexity Analysis

O(n*k log k) where k=max word length.

🔖 Interview Tips

Trade-off: $O(n)$ space for $O(1)$ lookup. Interviewers love follow-up: 'Can you do it with $O(1)$ space?' Edge cases: empty array, all duplicates, target sum with negative numbers.

Problem 5: LC 128 — Longest Consecutive Sequence

LeetCode #128 · leetcode.com/problems/

🔖 Problem Understanding

Find length of longest consecutive sequence in unsorted array. $O(n)$ required.

🔖 Pattern Recognition

This is a HASHING problem. Key signals: $O(1)$ lookup needed, frequency count, complement search. Hash maps trade space for time.

🔖 Brute Force Approach

Nested loops: for each element, scan rest of array to find complement or duplicate. $O(n^2)$ time.

⚡ Optimized Solution

Put all numbers in a set. For each number that is a sequence start ($\text{num}-1$ not in set), count streak.

🔖 Python Code

```
# LeetCode 128: Longest Consecutive Sequence
# Pattern: Hashing / Frequency Maps

num_set=set(nums); best=0
for n in num_set:
    if n-1 not in num_set: # sequence start
        cur=n; streak=1
        while cur+1 in num_set: cur+=1; streak+=1
        best=max(best,streak)
return best
```

🔖 Dry Run

nums=[100,4,200,1,3,2] → 4 (1,2,3,4)

🔖 Complexity Analysis

$O(n)$ time, $O(n)$ space.

🔖 Interview Tips

Trade-off: $O(n)$ space for $O(1)$ lookup. Interviewers love follow-up: 'Can you do it with $O(1)$ space?' Edge cases: empty array, all duplicates, target sum with negative numbers.

Problem 6: LC 347 — Top K Frequent Elements

LeetCode #347 · leetcode.com/problems/

🔍 Problem Understanding

Return k most frequent elements.

🔍 Pattern Recognition

This is a HASHING problem. Key signals: $O(1)$ lookup needed, frequency count, complement search. Hash maps trade space for time.

🔍 Brute Force Approach

Nested loops: for each element, scan rest of array to find complement or duplicate. $O(n^2)$ time.

⚡ Optimized Solution

Count frequencies. Use bucket sort (index=frequency) or heap for top-k.

🔍 Python Code

```
# LeetCode 347: Top K Frequent Elements
# Pattern: Hashing / Frequency Maps

from collections import Counter
count=Counter(nums)
buckets=[[] for _ in range(len(nums)+1)]
for n,freq in count.items(): buckets[freq].append(n)
res=[]
for i in range(len(buckets)-1,-1,-1):
    res.extend(buckets[i])
    if len(res)>=k: return res[:k]
```

🔍 Dry Run

nums=[1,1,1,2,2,3], k=2 → [1,2]

🔍 Complexity Analysis

$O(n)$ time with bucket sort, $O(n)$ space.

🔍 Interview Tips

Trade-off: $O(n)$ space for $O(1)$ lookup. Interviewers love follow-up: 'Can you do it with $O(1)$ space?' Edge cases: empty array, all duplicates, target sum with negative numbers.

Pattern 6 Summary: Hashing / Frequency Maps

Core intuition: $O(1)$ lookup by sacrificing $O(n)$ space. Recognition tricks: complement search, frequency counting, grouping. The key pattern: 'have I seen this before?'

Pattern 7: Prefix Sum / Running Sum

Problems: LC 303, LC 560, LC 724, LC 930, LC 974, LC 523

Problem 1: LC 303 — Range Sum Query - Immutable

LeetCode #303 · leetcode.com/problems/

🔖 Problem Understanding

Answer multiple range sum queries on a fixed array efficiently.

🔖 Pattern Recognition

This is a PREFIX SUM problem. Key signals: range sum queries, count subarrays with given sum. $\text{prefix}[i] - \text{prefix}[j]$ gives sum of subarray $[j+1..i]$.

🔖 Brute Force Approach

For each range query, sum all elements from l to r . $O(n)$ per query, $O(n^2)$ total. Prefix sum gives $O(1)$ per query.

⚡ Optimized Solution

Precompute $\text{prefix}[i] = \text{sum}(\text{nums}[0..i-1])$. Query $[l,r] = \text{prefix}[r+1] - \text{prefix}[l]$.

🔖 Python Code

```
# LeetCode 303: Range Sum Query - Immutable
# Pattern: Prefix Sum / Running Sum

class NumArray:
    def __init__(self, nums):
        self.pre = [0] * (len(nums) + 1)
        for i, n in enumerate(nums): self.pre[i+1] = self.pre[i] + n
    def sumRange(self, l, r):
        return self.pre[r+1] - self.pre[l]
```

🔖 Dry Run

$\text{nums} = [-2, 0, 3, -5, 2, -1]$; $\text{sumRange}(0,2)=1$; $\text{sumRange}(2,5)=-1$

🔖 Complexity Analysis

$O(n)$ build, $O(1)$ query.

🔖 Interview Tips

Visualize prefix array and explain how $\text{range sum} = \text{prefix}[r+1] - \text{prefix}[l]$. For subarray sum = k , explain why we check $\text{prefix} - k$ in the hash map. Edge cases: sum can be 0, array has negative numbers.

Problem 2: LC 560 — Subarray Sum Equals K

LeetCode #560 · leetcode.com/problems/

🔗 Problem Understanding

Count subarrays with sum exactly equal to k.

🔗 Pattern Recognition

This is a PREFIX SUM problem. Key signals: range sum queries, count subarrays with given sum. $\text{prefix}[i] - \text{prefix}[j]$ gives sum of subarray $[j+1..i]$.

🔗 Brute Force Approach

For each range query, sum all elements from l to r. $O(n)$ per query, $O(n^2)$ total. Prefix sum gives $O(1)$ per query.

⚡ Optimized Solution

$\text{prefix_sum}[j] - \text{prefix_sum}[i] = k$ means subarray $[i+1..j]$ sums to k. Hash map counts prefix sums seen.

🔗 Python Code

```
# LeetCode 560: Subarray Sum Equals K
# Pattern: Prefix Sum / Running Sum

from collections import defaultdict
count=defaultdict(int); count[0]=1
cur=res=0
for n in nums:
    cur+=n
    res+=count[cur-k]
    count[cur]+=1
return res
```

🔗 Dry Run

$\text{nums}=[1,1,1]$, $k=2 \rightarrow 2$

🔗 Complexity Analysis

$O(n)$ time, $O(n)$ space.

🔗 Interview Tips

Visualize prefix array and explain how $\text{range sum} = \text{prefix}[r+1] - \text{prefix}[l]$. For subarray sum = k, explain why we check $\text{prefix} - k$ in the hash map. Edge cases: sum can be 0, array has negative numbers.

Problem 3: LC 724 — Find Pivot Index

LeetCode #724 · leetcode.com/problems/

🔗 Problem Understanding

Find index where left sum equals right sum.

🔗 Pattern Recognition

This is a PREFIX SUM problem. Key signals: range sum queries, count subarrays with given sum. $\text{prefix}[i] - \text{prefix}[j]$ gives sum of subarray $[j+1..i]$.

🔗 Brute Force Approach

For each range query, sum all elements from l to r. $O(n)$ per query, $O(n^2)$ total. Prefix sum gives $O(1)$ per query.

⚡ Optimized Solution

Total sum known. Iterate left sum; right sum = total - left - nums[i]. Check equality.

🔗 Python Code

```
# LeetCode 724: Find Pivot Index
# Pattern: Prefix Sum / Running Sum

total = sum(nums); left = 0
for i, n in enumerate(nums):
    if left == total - left - n: return i
    left += n
return -1
```

🔗 Dry Run

nums=[1,7,3,6,5,6] → 3

🔗 Complexity Analysis

$O(n)$ time, $O(1)$ space.

🔗 Interview Tips

Visualize prefix array and explain how range sum = prefix[r+1] - prefix[l]. For subarray sum = k, explain why we check prefix - k in the hash map. Edge cases: sum can be 0, array has negative numbers.

Problem 4: LC 930 — Binary Subarrays With Sum

LeetCode #930 · leetcode.com/problems/

🔗 Problem Understanding

Count subarrays with sum equal to goal in binary array.

🔗 Pattern Recognition

This is a PREFIX SUM problem. Key signals: range sum queries, count subarrays with given sum. prefix[i]-prefix[j] gives sum of subarray [j+1..i].

🔗 Brute Force Approach

For each range query, sum all elements from l to r. $O(n)$ per query, $O(n^2)$ total. Prefix sum gives $O(1)$ per query.

⚡ Optimized Solution

Same as LC 560: count prefix sums in hash map.

🔗 Python Code

```
# LeetCode 930: Binary Subarrays With Sum
# Pattern: Prefix Sum / Running Sum

from collections import defaultdict
```



```
count=defaultdict(int); count[0]=1
cur=res=0
for n in nums:
    cur+=n
    res+=count[cur-goal]
    count[cur]+=1
return res
```

🔗 Dry Run

nums=[1,0,1,0,1], goal=2 → 4

🔗 Complexity Analysis

O(n) time, O(n) space.

🔗 Interview Tips

Visualize prefix array and explain how range sum = prefix[r+1] - prefix[l]. For subarray sum = k, explain why we check prefix - k in the hash map. Edge cases: sum can be 0, array has negative numbers.

Problem 5: LC 974 — Subarray Sums Divisible by K

LeetCode #974 · leetcode.com/problems/

🔗 Problem Understanding

Count subarrays whose sum is divisible by k.

🔗 Pattern Recognition

This is a PREFIX SUM problem. Key signals: range sum queries, count subarrays with given sum. prefix[i]-prefix[j] gives sum of subarray [j+1..i].

🔗 Brute Force Approach

For each range query, sum all elements from l to r. O(n) per query, O(n²) total. Prefix sum gives O(1) per query.

⚡ Optimized Solution

Two prefix sums with same remainder mod k mean subarray between them is divisible by k.

🔗 Python Code

```
# LeetCode 974: Subarray Sums Divisible by K
# Pattern: Prefix Sum / Running Sum

from collections import defaultdict
count=defaultdict(int); count[0]=1
cur=res=0
for n in nums:
    cur=(cur+n)%k
    res+=count[cur]
    count[cur]+=1
return res
```

🔗 Dry Run

nums=[4,5,0,-2,-3,1], k=5 → 7

🔖 Complexity Analysis

O(n) time, O(k) space.

🔖 Interview Tips

Visualize prefix array and explain how range sum = prefix[r+1] - prefix[l]. For subarray sum = k, explain why we check prefix - k in the hash map. Edge cases: sum can be 0, array has negative numbers.

Problem 6: LC 523 — Continuous Subarray Sum

LeetCode #523 · leetcode.com/problems/

🔖 Problem Understanding

Check if there's a subarray of length ≥ 2 whose sum is a multiple of k.

🔖 Pattern Recognition

This is a PREFIX SUM problem. Key signals: range sum queries, count subarrays with given sum. prefix[i]-prefix[j] gives sum of subarray [j+1..i].

🔖 Brute Force Approach

For each range query, sum all elements from l to r. O(n) per query, O(n²) total. Prefix sum gives O(1) per query.

⚡ Optimized Solution

If prefix[i] % k == prefix[j] % k and j-i $\geq$ 2, subarray sums to multiple of k.

🔖 Python Code

```
# LeetCode 523: Continuous Subarray Sum
# Pattern: Prefix Sum / Running Sum

seen={0:-1}
cur=0
for i,n in enumerate(nums):
    cur=(cur+n)%k
    if cur in seen:
        if i-seen[cur]>=2: return True
    else: seen[cur]=i
return False
```

🔖 Dry Run

nums=[23,2,4,6,7], k=6 → True

🔖 Complexity Analysis

O(n) time, O(min(n,k)) space.

🔖 Interview Tips

Visualize prefix array and explain how range sum = prefix[r+1] - prefix[l]. For subarray sum = k, explain why we check prefix - k in the hash map. Edge cases: sum can be 0, array has negative numbers.

Pattern 7 Summary: Prefix Sum / Running Sum

Core intuition: precompute cumulative sums for $O(1)$ range queries. Recognition tricks: range sum queries, count subarrays with target sum. Key formula: $\text{sum}[l..r] = \text{prefix}[r+1] - \text{prefix}[l]$.

Pattern 8: Difference Array / Range Updates

Problems: LC 370, LC 1094, LC 1109, LC 1893, LC 1943, LC 2381

Problem 1: LC 370 — Range Addition

LeetCode #370 · [leetcode.com/problems/](https://leetcode.com/problems/range-addition/)

🔖 Problem Understanding

Apply k range increment operations $[l, r, val]$ to an array of zeros. Return final array.

🔖 Pattern Recognition

This is a DIFFERENCE ARRAY problem. Key signals: range update operations, add value to all elements in range. $diff[l] += val$, $diff[r+1] -= val$, then prefix sum.

🔖 Brute Force Approach

For each range update, iterate through and update each element. $O(n)$ per update, $O(n*k)$ total. Difference array reduces each update to $O(1)$.

⚡ Optimized Solution

Difference array: add val at l , subtract at $r+1$. Then prefix sum to get result.

🔖 Python Code

```
# LeetCode 370: Range Addition
# Pattern: Difference Array / Range Updates

diff = [0] * (n+1)
for l, r, val in updates:
    diff[l] += val; diff[r+1] -= val
cur = 0; res = []
for i in range(n): cur += diff[i]; res.append(cur)
return res
```

🔖 Dry Run

$n=5$, $updates = [[1,3,2],[2,4,3],[0,2,-2]] \rightarrow [-2,0,3,5,3]$

🔖 Complexity Analysis

$O(n+k)$ time, $O(n)$ space.

🔖 Interview Tips

Explain: 'Instead of updating each element, I mark boundaries and reconstruct with prefix sum.'
Common follow-up: 2D difference arrays.

Problem 2: LC 1094 — Car Pooling

LeetCode #1094 · [leetcode.com/problems/](https://leetcode.com/problems/car-pooling/)

🔗 Problem Understanding

Check if a car with capacity can carry all passengers given trips [num,from,to].

🔗 Pattern Recognition

This is a DIFFERENCE ARRAY problem. Key signals: range update operations, add value to all elements in range. $\text{diff}[l] += \text{val}$, $\text{diff}[r+1] -= \text{val}$, then prefix sum.

🔗 Brute Force Approach

For each range update, iterate through and update each element. $O(n)$ per update, $O(n*k)$ total. Difference array reduces each update to $O(1)$.

⚡ Optimized Solution

Difference array on stops. Add passengers at from, remove at to.

🔗 Python Code

```
# LeetCode 1094: Car Pooling
# Pattern: Difference Array / Range Updates

stops=[0]*1001
for num,frm,to in trips:
    stops[frm]+=num; stops[to]-=num
cur=0
for s in stops:
    cur+=s
    if cur>capacity: return False
return True
```

🔗 Dry Run

trips=[[2,1,5],[3,3,7]], capacity=4 → False

🔗 Complexity Analysis

$O(n+1001)$ time.

🔗 Interview Tips

Explain: 'Instead of updating each element, I mark boundaries and reconstruct with prefix sum.'
Common follow-up: 2D difference arrays.

Problem 3: LC 1109 — Corporate Flight Bookings

LeetCode #1109 · leetcode.com/problems/

🔗 Problem Understanding

Each booking [first,last,seats] adds seats to flights first..last. Return array of total seats per flight.

🔗 Pattern Recognition

This is a DIFFERENCE ARRAY problem. Key signals: range update operations, add value to all elements in range. $\text{diff}[l] += \text{val}$, $\text{diff}[r+1] -= \text{val}$, then prefix sum.

🔗 Brute Force Approach

For each range update, iterate through and update each element. $O(n)$ per update, $O(n*k)$ total. Difference array reduces each update to $O(1)$.

⚡ Optimized Solution

Difference array on 1-indexed flights.

🔗 Python Code

```
# LeetCode 1109: Corporate Flight Bookings
# Pattern: Difference Array / Range Updates

diff=[0]*(n+2)
for first,last,seats in bookings:
    diff[first]+=seats; diff[last+1]-=seats
cur=0; res=[]
for i in range(1,n+1): cur+=diff[i]; res.append(cur)
return res
```

🔗 Dry Run

bookings=[[1,2,10],[2,3,20],[2,5,25]], n=5 → [10,55,45,25,25]

🔗 Complexity Analysis

$O(n+k)$ time.

🔗 Interview Tips

Explain: 'Instead of updating each element, I mark boundaries and reconstruct with prefix sum.'

Common follow-up: 2D difference arrays.

Problem 4: LC 1893 — Check if All the Integers in a Range Are Covered

LeetCode #1893 · leetcode.com/problems/

🔗 Problem Understanding

Given list of ranges, check if every integer in [left,right] is covered.

🔗 Pattern Recognition

This is a DIFFERENCE ARRAY problem. Key signals: range update operations, add value to all elements in range. $\text{diff}[l] += \text{val}$, $\text{diff}[r+1] -= \text{val}$, then prefix sum.

🔗 Brute Force Approach

For each range update, iterate through and update each element. $O(n)$ per update, $O(n*k)$ total. Difference array reduces each update to $O(1)$.

⚡ Optimized Solution

Difference array on 1..50. Mark covered positions. Check [left,right] all covered.

🔗 Python Code

```
# LeetCode 1893: Check if All the Integers in a Range Are Covered
# Pattern: Difference Array / Range Updates
```

```

diff=[0]*52
for l,r in ranges:
    diff[l]+=1; diff[r+1]-=1
cur=0
for i in range(1,51):
    cur+=diff[i]
    if left<=i<=right and cur==0: return False
return True

```

🔗 Dry Run

ranges=[[1,2],[3,4],[5,6]], left=2, right=5 → True

🔗 Complexity Analysis

$O(n+50)$ time.

🔗 Interview Tips

Explain: 'Instead of updating each element, I mark boundaries and reconstruct with prefix sum.'

Common follow-up: 2D difference arrays.

Problem 5: LC 1943 — Describe the Painting

LeetCode #1943 · leetcode.com/problems/

🔗 Problem Understanding

Merge overlapping segments with summed values.

🔗 Pattern Recognition

This is a DIFFERENCE ARRAY problem. Key signals: range update operations, add value to all elements in range. $\text{diff}[l] += \text{val}$, $\text{diff}[r+1] -= \text{val}$, then prefix sum.

🔗 Brute Force Approach

For each range update, iterate through and update each element. $O(n)$ per update, $O(n*k)$ total. Difference array reduces each update to $O(1)$.

⚡ Optimized Solution

Events at start (+color) and end (-color). Sweep through sorted events.

🔗 Python Code

```

# LeetCode 1943: Describe the Painting
# Pattern: Difference Array / Range Updates

from collections import defaultdict
events=defaultdict(int)
for l,r,c in segments:
    events[l]+=c; events[r]-=c
res=[]; cur=0; prev=None
for pos in sorted(events):
    if cur>0 and prev is not None: res.append([prev,pos,cur])
    cur+=events[pos]; prev=pos
return res

```

🔗 Dry Run

segments=[[1,4,5],[1,4,7],[4,7,1],[4,7,2]] → [[1,4,12],[4,7,3]]

🔗 Complexity Analysis

$O(n \log n)$ time.

🔗 Interview Tips

Explain: 'Instead of updating each element, I mark boundaries and reconstruct with prefix sum.'

Common follow-up: 2D difference arrays.

Problem 6: LC 2381 — Shifting Letters II

LeetCode #2381 · leetcode.com/problems/

🔗 Problem Understanding

Apply range shift operations to a string. Each op [l,r,d] shifts chars in [l..r] forward (d=1) or backward (d=0).

🔗 Pattern Recognition

This is a DIFFERENCE ARRAY problem. Key signals: range update operations, add value to all elements in range. $\text{diff}[l] += \text{val}$, $\text{diff}[r+1] -= \text{val}$, then prefix sum.

🔗 Brute Force Approach

For each range update, iterate through and update each element. $O(n)$ per update, $O(n*k)$ total. Difference array reduces each update to $O(1)$.

⚡ Optimized Solution

Difference array with +1/-1 shifts. Prefix sum to get net shift at each index.

🔗 Python Code

```
# LeetCode 2381: Shifting Letters II
# Pattern: Difference Array / Range Updates

diff=[0]*(len(s)+1)
for l,r,d in shifts:
    v=1 if d else -1
    diff[l]+=v; diff[r+1]-=v
cur=0; res=[]
for i,c in enumerate(s):
    cur+=diff[i]
    res.append(chr((ord(c)-ord('a')+cur)%26+ord('a'))))
return ''.join(res)
```

🔗 Dry Run

s='abc', shifts=[[0,1,0],[1,2,1]] → 'zcd'

🔗 Complexity Analysis

$O(n+m)$ time.

🔗 Interview Tips

Explain: 'Instead of updating each element, I mark boundaries and reconstruct with prefix sum.'
Common follow-up: 2D difference arrays.

Pattern 8 Summary: Difference Array / Range Updates

Core intuition: mark range boundaries, then reconstruct with prefix sum. Recognition tricks: bulk range update operations. More powerful than updating each element.

Pattern 9: Monotonic Stack

Problems: LC 739, LC 496, LC 503, LC 84, LC 85, LC 901

Problem 1: LC 739 — Daily Temperatures

LeetCode #739 · leetcode.com/problems/

🔖 Problem Understanding

For each day, find how many days until a warmer temperature.

🔖 Pattern Recognition

This is a MONOTONIC STACK problem. Key signals: next greater/smaller element, largest rectangle. Stack maintains elements in sorted order.

🔖 Brute Force Approach

For each element, scan right/left to find next greater/smaller. $O(n^2)$ time. Stack gives $O(n)$ amortized.

🔖 Optimized Solution

Stack stores indices of unresolved days. When we find a warmer day, pop and compute difference.

🔖 Python Code

```
# LeetCode 739: Daily Temperatures
# Pattern: Monotonic Stack

stack=[]; res=[0]*len(temperatures)
for i,t in enumerate(temperatures):
    while stack and temperatures[stack[-1]]<t:
        j=stack.pop(); res[j]=i-j
    stack.append(i)
return res
```

🔖 Dry Run

temperatures=[73,74,75,71,69,72,76,73] → [1,1,4,2,1,1,0,0]

🔖 Complexity Analysis

$O(n)$ time, $O(n)$ space.

🔖 Interview Tips

Trace through the stack clearly in your dry run. Explain why the stack maintains sorted order. Common follow-up: 'What if array is circular?' (LC 503).

Problem 2: LC 496 — Next Greater Element I

LeetCode #496 · leetcode.com/problems/

🔖 Problem Understanding

For each element in nums1 (subset of nums2), find next greater element in nums2.

🔍 Pattern Recognition

This is a MONOTONIC STACK problem. Key signals: next greater/smaller element, largest rectangle. Stack maintains elements in sorted order.

🔍 Brute Force Approach

For each element, scan right/left to find next greater/smaller. $O(n^2)$ time. Stack gives $O(n)$ amortized.

⚡ Optimized Solution

Build next-greater map for nums2 using monotonic stack. Lookup for nums1.

🔍 Python Code

```
# LeetCode 496: Next Greater Element I
# Pattern: Monotonic Stack

stack=[]; nge={}
for n in nums2:
    while stack and stack[-1]<n:
        nge[stack.pop()]=n
    stack.append(n)
for n in stack: nge[n]=-1
return [nge[n] for n in nums1]
```

🔍 Dry Run

nums1=[4,1,2], nums2=[1,3,4,2] → [-1,3,-1]

🔍 Complexity Analysis

$O(n)$ time, $O(n)$ space.

🔍 Interview Tips

Trace through the stack clearly in your dry run. Explain why the stack maintains sorted order. Common follow-up: 'What if array is circular?' (LC 503).

Problem 3: LC 503 — Next Greater Element II (Circular Array)

LeetCode #503 · leetcode.com/problems/

🔍 Problem Understanding

Find next greater element in a circular array.

🔍 Pattern Recognition

This is a MONOTONIC STACK problem. Key signals: next greater/smaller element, largest rectangle. Stack maintains elements in sorted order.

🔍 Brute Force Approach

For each element, scan right/left to find next greater/smaller. $O(n^2)$ time. Stack gives $O(n)$ amortized.

⚡ Optimized Solution

Traverse array twice (simulate circular). Monotonic stack stores indices.

Python Code

```
# LeetCode 503: Next Greater Element II (Circular Array)
# Pattern: Monotonic Stack

n=len(nums); res=[-1]*n; stack=[]
for i in range(2*n):
    while stack and nums[stack[-1]]<nums[i%n]:
        res[stack.pop()]=nums[i%n]
    if i<n: stack.append(i)
return res
```

Dry Run

nums=[1,2,1] → [2,-1,2]

Complexity Analysis

O(n) time, O(n) space.

Interview Tips

Trace through the stack clearly in your dry run. Explain why the stack maintains sorted order. Common follow-up: 'What if array is circular?' (LC 503).

Problem 4: LC 84 — Largest Rectangle in Histogram

LeetCode #84 · [leetcode.com/problems/](https://leetcode.com/problems/largest-rectangle-in-histogram/)

Problem Understanding

Find the largest rectangle area in a histogram.

Pattern Recognition

This is a MONOTONIC STACK problem. Key signals: next greater/smaller element, largest rectangle. Stack maintains elements in sorted order.

Brute Force Approach

For each element, scan right/left to find next greater/smaller. O(n²) time. Stack gives O(n) amortized.

⚡ Optimized Solution

Monotonic increasing stack. When we find a shorter bar, pop taller bars and compute rectangles.

Python Code

```
# LeetCode 84: Largest Rectangle in Histogram
# Pattern: Monotonic Stack

stack=[]; max_area=0; heights=heights+[0]
for i,h in enumerate(heights):
    start=i
    while stack and stack[-1][1]>h:
        idx,ht=stack.pop()
        max_area=max(max_area,ht*(i-idx))
    stack.append((start,h))
```

```
        start=idx
        stack.append((start,h))
    return max_area
```

🔍 Dry Run

heights=[2,1,5,6,2,3] → 10

🔍 Complexity Analysis

O(n) time, O(n) space.

🔍 Interview Tips

Trace through the stack clearly in your dry run. Explain why the stack maintains sorted order. Common follow-up: 'What if array is circular?' (LC 503).

Problem 5: LC 85 — Maximal Rectangle

LeetCode #85 · leetcode.com/problems/

🔍 Problem Understanding

Find largest rectangle in binary matrix of '0's and '1's.

🔍 Pattern Recognition

This is a MONOTONIC STACK problem. Key signals: next greater/smaller element, largest rectangle. Stack maintains elements in sorted order.

🔍 Brute Force Approach

For each element, scan right/left to find next greater/smaller. O(n²) time. Stack gives O(n) amortized.

⚡ Optimized Solution

Convert each row into histogram heights. Apply LC 84 on each row.

🔍 Python Code

```
# LeetCode 85: Maximal Rectangle
# Pattern: Monotonic Stack

if not matrix: return 0
n=len(matrix[0]); heights=[0]*n; res=0
for row in matrix:
    for j in range(n):
        heights[j]=heights[j]+1 if row[j]=='1' else 0
    # largest rectangle in histogram
    stack=[]; h2=heights+[0]
    for i,h in enumerate(h2):
        start=i
        while stack and stack[-1][1]>h:
            idx,ht=stack.pop()
            res=max(res,ht*(i-idx)); start=idx
        stack.append((start,h))
    return res
```

🔍 Dry Run

matrix=[['1','0','1','0','0'],['1','0','1','1','1'],['1','1','1','1','1'],['1','0','0','1','0']] → 6

🔗 Complexity Analysis

$O(m \cdot n)$ time, $O(n)$ space.

🔗 Interview Tips

Trace through the stack clearly in your dry run. Explain why the stack maintains sorted order. Common follow-up: 'What if array is circular?' (LC 503).

Problem 6: LC 901 — Online Stock Span

LeetCode #901 · leetcode.com/problems/

🔗 Problem Understanding

For each stock price, find how many consecutive days (including today) the price was $\leq$ today.

🔗 Pattern Recognition

This is a MONOTONIC STACK problem. Key signals: next greater/smaller element, largest rectangle. Stack maintains elements in sorted order.

🔗 Brute Force Approach

For each element, scan right/left to find next greater/smaller. $O(n^2)$ time. Stack gives $O(n)$ amortized.

⚡ Optimized Solution

Stack stores (price, span). Merge spans of all smaller-or-equal prices.

🔗 Python Code

```
# LeetCode 901: Online Stock Span
# Pattern: Monotonic Stack

class StockSpanner:
    def __init__(self): self.stack=[]
    def next(self,price):
        span=1
        while self.stack and self.stack[-1][0]<=price:
            span+=self.stack.pop()[1]
        self.stack.append((price,span))
        return span
```

🔗 Dry Run

prices=[100,80,60,70,60,75,85] → [1,1,1,2,1,4,6]

🔗 Complexity Analysis

$O(1)$ amortized per call, $O(n)$ space.

🔗 Interview Tips

Trace through the stack clearly in your dry run. Explain why the stack maintains sorted order. Common follow-up: 'What if array is circular?' (LC 503).

Pattern 9 Summary: Monotonic Stack

Core intuition: maintain a stack where elements are in sorted order. Popping happens when we find a violating element. Recognition tricks: 'next greater/smaller element', 'span', 'histogram'. Always processes in $O(n)$.

Pattern 10: Monotonic Queue / Deque

Problems: LC 239, LC 862, LC 1425, LC 1438, LC 1499, LC 1696

Problem 1: LC 239 — Sliding Window Maximum

LeetCode #239 · leetcode.com/problems/

🔖 Problem Understanding

Find maximum in every window of size k.

🔖 Pattern Recognition

This is a MONOTONIC DEQUE problem. Key signals: sliding window max/min, optimize DP with window. Deque gives $O(1)$ access to window extremes.

🔖 Brute Force Approach

For each window, scan all k elements to find max/min. $O(n*k)$ time. Deque gives $O(n)$ total.

🔖 Optimized Solution

Deque stores indices in decreasing order of value. Front is always the current window maximum.

🔖 Python Code

```
# LeetCode 239: Sliding Window Maximum
# Pattern: Monotonic Queue / Deque

from collections import deque
dq=deque(); res=[]
for i,n in enumerate(nums):
    while dq and nums[dq[-1]]<=n: dq.pop()
    dq.append(i)
    if dq[0]==i-k: dq.popleft()
    if i>=k-1: res.append(nums[dq[0]])
return res
```

🔖 Dry Run

nums=[1,3,-1,-3,5,3,6,7], k=3 → [3,3,5,5,6,7]

🔖 Complexity Analysis

$O(n)$ time, $O(k)$ space.

🔖 Interview Tips

Explain: 'Deque front = current window max, we pop from back when smaller elements enter.' Edge cases: window size equals array size.

Problem 2: LC 862 — Shortest Subarray with Sum at Least K

LeetCode #862 · leetcode.com/problems/

🔍 Problem Understanding

Find minimum length subarray with sum $\geq k$ (can have negative numbers).

🔍 Pattern Recognition

This is a MONOTONIC DEQUE problem. Key signals: sliding window max/min, optimize DP with window. Deque gives $O(1)$ access to window extremes.

🔍 Brute Force Approach

For each window, scan all k elements to find max/min. $O(n*k)$ time. Deque gives $O(n)$ total.

⚡ Optimized Solution

Prefix sums + monotonic deque of increasing prefix sums. For each r , find leftmost l where $pre[r] - pre[l] \geq k$.

🔍 Python Code

```
# LeetCode 862: Shortest Subarray with Sum at Least K
# Pattern: Monotonic Queue / Deque

from collections import deque
pre=[0]*(len(nums)+1)
for i,n in enumerate(nums): pre[i+1]=pre[i]+n
dq=deque(); res=float('inf')
for r in range(len(pre)):
    while dq and pre[r]-pre[dq[0]]>=k:
        res=min(res,r-dq.popleft())
    while dq and pre[dq[-1]]>=pre[r]: dq.pop()
    dq.append(r)
return res if res!=float('inf') else -1
```

🔍 Dry Run

nums=[2,-1,2], k=3 $\rightarrow$ 3

🔍 Complexity Analysis

$O(n)$ time, $O(n)$ space.

🔍 Interview Tips

Explain: 'Deque front = current window max, we pop from back when smaller elements enter.' Edge cases: window size equals array size.

Problem 3: LC 1425 — Constrained Subsequence Sum

LeetCode #1425 · leetcode.com/problems/

🔍 Problem Understanding

Find max sum subsequence where adjacent chosen indices differ by at most k .

🔍 Pattern Recognition

This is a MONOTONIC DEQUE problem. Key signals: sliding window max/min, optimize DP with window. Deque gives $O(1)$ access to window extremes.

🔍 Brute Force Approach

For each window, scan all k elements to find max/min. $O(n*k)$ time. Deque gives $O(n)$ total.

⚡ Optimized Solution

DP with deque storing max dp values in window of size k .

🔍 Python Code

```
# LeetCode 1425: Constrained Subsequence Sum
# Pattern: Monotonic Queue / Deque

from collections import deque
dp=nums[:]; dq=deque([0]); res=dp[0]
for i in range(1,len(nums)):
    dp[i]=nums[i]+max(0,dp[dq[0]])
    res=max(res,dp[i])
    while dq and dp[dq[-1]]<=dp[i]: dq.pop()
    dq.append(i)
    if dq[0]==i-k: dq.popleft()
return res
```

🔍 Dry Run

nums=[10,2,-10,5,20], $k=2 \rightarrow 37$

🔍 Complexity Analysis

$O(n)$ time, $O(k)$ space.

🔍 Interview Tips

Explain: 'Deque front = current window max, we pop from back when smaller elements enter.' Edge cases: window size equals array size.

Problem 4: LC 1438 — Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit

LeetCode #1438 · leetcode.com/problems/

🔍 Problem Understanding

Find longest subarray where $\max - \min \leq \text{limit}$.

🔍 Pattern Recognition

This is a MONOTONIC DEQUE problem. Key signals: sliding window max/min, optimize DP with window. Deque gives $O(1)$ access to window extremes.

🔍 Brute Force Approach

For each window, scan all k elements to find max/min. $O(n*k)$ time. Deque gives $O(n)$ total.

⚡ Optimized Solution

Two deques tracking window max and min. Shrink window when $\max - \min > \text{limit}$.

🔍 Python Code

```
# LeetCode 1438: Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit
# Pattern: Monotonic Queue / Deque

from collections import deque
max_dq=deque(); min_dq=deque(); l=0; res=0
for r,n in enumerate(nums):
    while max_dq and nums[max_dq[-1]]<=n: max_dq.pop()
    while min_dq and nums[min_dq[-1]]>=n: min_dq.pop()
    max_dq.append(r); min_dq.append(r)
    while nums[max_dq[0]]-nums[min_dq[0]]>limit:
        l+=1
        if max_dq[0]<l: max_dq.popleft()
        if min_dq[0]<l: min_dq.popleft()
    res=max(res,r-l+1)
return res
```

🔗 Dry Run

nums=[8,2,4,7], limit=4 → 2

🔗 Complexity Analysis

O(n) time, O(n) space.

🔗 Interview Tips

Explain: 'Deque front = current window max, we pop from back when smaller elements enter.' Edge cases: window size equals array size.

Problem 5: LC 1499 — Max Value of Equation

LeetCode #1499 · leetcode.com/problems/

🔗 Problem Understanding

Maximize $y_i + y_j + |x_i - x_j|$ for pairs where $x_j - x_i \leq k$.

🔗 Pattern Recognition

This is a MONOTONIC DEQUE problem. Key signals: sliding window max/min, optimize DP with window. Deque gives O(1) access to window extremes.

🔗 Brute Force Approach

For each window, scan all k elements to find max/min. O(n*k) time. Deque gives O(n) total.

⚡ Optimized Solution

Rewrite as $(y_j + x_j) + (y_i - x_i)$ where $x_i \leq x_j \leq x_i + k$. Deque stores max $(y_i - x_i)$ in window.

🔗 Python Code

```
# LeetCode 1499: Max Value of Equation
# Pattern: Monotonic Queue / Deque

from collections import deque
dq=deque(); res=float('-inf')
for xj,yj in points:
```

```

while dq and xj-dq[0][1]>k: dq.popleft()
if dq: res=max(res,yj+xj+dq[0][0])
while dq and dq[-1][0]<=yj-xj: dq.pop()
dq.append((yj-xj,xj))
return res

```

🔗 Dry Run

points=[[1,3],[2,0],[5,10],[6,-10]], k=1 → 4

🔗 Complexity Analysis

O(n) time, O(k) space.

🔗 Interview Tips

Explain: 'Deque front = current window max, we pop from back when smaller elements enter.' Edge cases: window size equals array size.

Problem 6: LC 1696 — Jump Game VI

LeetCode #1696 · leetcode.com/problems/

🔗 Problem Understanding

Max score path where each jump is at most k steps. $dp[i] = nums[i] + \max(dp[i-k..i-1])$.

🔗 Pattern Recognition

This is a MONOTONIC DEQUE problem. Key signals: sliding window max/min, optimize DP with window. Deque gives O(1) access to window extremes.

🔗 Brute Force Approach

For each window, scan all k elements to find max/min. O(n*k) time. Deque gives O(n) total.

⚡ Optimized Solution

Deque stores indices of max dp values in window of k.

🔗 Python Code

```

# LeetCode 1696: Jump Game VI
# Pattern: Monotonic Queue / Deque

from collections import deque
dp=nums[:]; dq=deque([0])
for i in range(1,len(nums)):
    while dq and dq[0]<i-k: dq.popleft()
    dp[i]=nums[i]+dp[dq[0]]
    while dq and dp[dq[-1]]<=dp[i]: dq.pop()
    dq.append(i)
return dp[-1]

```

🔗 Dry Run

nums=[1,-1,-2,4,-7,3], k=2 → 7

🔗 Complexity Analysis

$O(n)$ time, $O(k)$ space.

Interview Tips

Explain: 'Deque front = current window max, we pop from back when smaller elements enter.' Edge cases: window size equals array size.

Pattern 10 Summary: Monotonic Queue / Deque

Core intuition: efficiently find max/min in sliding windows. Recognition tricks: sliding window extremes, optimize $O(n^2)$ DP with window. Key: front is always the answer, pop from back to maintain order.

Pattern 11: Heap / Top K

Problems: LC 215, LC 347, LC 692, LC 703, LC 973, LC 1046

Problem 1: LC 215 — Kth Largest Element in an Array

LeetCode #215 · leetcode.com/problems/

🔖 Problem Understanding

Find the kth largest element (not kth distinct).

🔖 Pattern Recognition

This is a HEAP problem. Key signals: top-K elements, dynamic ordering, median. Python heapq is a min-heap; negate values for max-heap.

🔖 Brute Force Approach

Sort the entire array to find top-k. $O(n \log n)$. Heap gives $O(n \log k)$ which is better when $k \ll n$.

🚀 Optimized Solution

Min-heap of size k. Push each element; if heap > k, pop smallest. Top is kth largest.

🔖 Python Code

```
# LeetCode 215: Kth Largest Element in an Array
# Pattern: Heap / Top K

import heapq
heap=[]
for n in nums:
    heapq.heappush(heap,n)
    if len(heap)>k: heapq.heappop(heap)
return heap[0]
```

🔖 Dry Run

nums=[3,2,1,5,6,4], k=2 → 5

🔖 Complexity Analysis

$O(n \log k)$ time, $O(k)$ space.

🔖 Interview Tips

Know Python heapq is min-heap. For max-heap, push -value. Explain time complexity: $O(\log k)$ per operation. Follow-up: 'Can you do $O(n)$ with quickselect?'

Problem 2: LC 347 — Top K Frequent Elements

LeetCode #347 · leetcode.com/problems/

🔖 Problem Understanding

Return k most frequent elements.

🔍 Pattern Recognition

This is a HEAP problem. Key signals: top-K elements, dynamic ordering, median. Python heapq is a min-heap; negate values for max-heap.

🔍 Brute Force Approach

Sort the entire array to find top-k. $O(n \log n)$. Heap gives $O(n \log k)$ which is better when $k \ll n$.

⚡ Optimized Solution

Count frequencies with Counter. Use min-heap of size k by frequency.

🔍 Python Code

```
# LeetCode 347: Top K Frequent Elements
# Pattern: Heap / Top K

from collections import Counter
import heapq
count=Counter(nums)
return heapq.nlargest(k, count.keys(), key=count.get)
```

🔍 Dry Run

nums=[1,1,1,2,2,3], k=2 → [1,2]

🔍 Complexity Analysis

$O(n \log k)$ time, $O(n)$ space.

🔍 Interview Tips

Know Python heapq is min-heap. For max-heap, push -value. Explain time complexity: $O(\log k)$ per operation. Follow-up: 'Can you do $O(n)$ with quickselect?'

Problem 3: LC 692 — Top K Frequent Words

LeetCode #692 · [leetcode.com/problems/](https://leetcode.com/problems/top-k-frequent-words/)

🔍 Problem Understanding

Return k most frequent words. Ties broken alphabetically.

🔍 Pattern Recognition

This is a HEAP problem. Key signals: top-K elements, dynamic ordering, median. Python heapq is a min-heap; negate values for max-heap.

🔍 Brute Force Approach

Sort the entire array to find top-k. $O(n \log n)$. Heap gives $O(n \log k)$ which is better when $k \ll n$.

⚡ Optimized Solution

Count with Counter. Sort by $(-freq, word)$ — negative freq so most frequent comes first.

🔖 Python Code

```
# LeetCode 692: Top K Frequent Words
# Pattern: Heap / Top K

from collections import Counter
import heapq
count=Counter(words)
heap=[(-freq,word) for word,freq in count.items()]
heapq.heapify(heap)
return [heapq.heappop(heap)[1] for _ in range(k)]
```

🔖 Dry Run

words=['i','love','leetcode','i','love','coding'], k=2 → ['i','love']

🔖 Complexity Analysis

$O(n \log k)$ time, $O(n)$ space.

🔖 Interview Tips

Know Python heapq is min-heap. For max-heap, push -value. Explain time complexity: $O(\log k)$ per operation. Follow-up: 'Can you do $O(n)$ with quickselect?'

Problem 4: LC 703 — Kth Largest Element in a Stream

LeetCode #703 · leetcode.com/problems/

🔖 Problem Understanding

Design a class that finds kth largest element dynamically as numbers are added.

🔖 Pattern Recognition

This is a HEAP problem. Key signals: top-K elements, dynamic ordering, median. Python heapq is a min-heap; negate values for max-heap.

🔖 Brute Force Approach

Sort the entire array to find top-k. $O(n \log n)$. Heap gives $O(n \log k)$ which is better when $k \ll n$.

⚡ Optimized Solution

Min-heap of size k. Top is always kth largest.

🔖 Python Code

```
# LeetCode 703: Kth Largest Element in a Stream
# Pattern: Heap / Top K

import heapq
class KthLargest:
    def __init__(self,k,nums):
        self.k=k; self.heap=nums
        heapq.heapify(self.heap)
        while len(self.heap)>k: heapq.heappop(self.heap)
    def add(self,val):
        heapq.heappush(self.heap,val)
```



```
if len(self.heap)>self.k: heapq.heappop(self.heap)
return self.heap[0]
```

🔗 Dry Run

k=3, nums=[4,5,8,2], add(3)→4, add(5)→5, add(10)→5, add(9)→8, add(4)→8

🔗 Complexity Analysis

$O(\log k)$ per add, $O(k)$ space.

🔗 Interview Tips

Know Python heapq is min-heap. For max-heap, push -value. Explain time complexity: $O(\log k)$ per operation. Follow-up: 'Can you do $O(n)$ with quickselect?'

Problem 5: LC 973 — K Closest Points to Origin

LeetCode #973 · [leetcode.com/problems/](https://leetcode.com/problems/k-closest-points-to-origin/)

🔗 Problem Understanding

Find k points closest to origin.

🔗 Pattern Recognition

This is a HEAP problem. Key signals: top-K elements, dynamic ordering, median. Python heapq is a min-heap; negate values for max-heap.

🔗 Brute Force Approach

Sort the entire array to find top-k. $O(n \log n)$. Heap gives $O(n \log k)$ which is better when $k \ll n$.

⚡ Optimized Solution

Max-heap of size k by distance. Keep only k closest.

🔗 Python Code

```
# LeetCode 973: K Closest Points to Origin
# Pattern: Heap / Top K

import heapq
heap=[]
for x,y in points:
    dist=x*x+y*y
    heapq.heappush(heap, (-dist, (x,y)))
    if len(heap)>k: heapq.heappop(heap)
return [p for _,p in heap]
```

🔗 Dry Run

points=[[1,3],[-2,2]], k=1 → [[-2,2]]

🔗 Complexity Analysis

$O(n \log k)$ time, $O(k)$ space.

🔗 Interview Tips

Know Python `heapq` is min-heap. For max-heap, push `-value`. Explain time complexity: $O(\log k)$ per operation. Follow-up: 'Can you do $O(n)$ with `quickselect`?'

Problem 6: LC 1046 — Last Stone Weight

LeetCode #1046 · leetcode.com/problems/

🔍 Problem Understanding

Repeatedly smash two heaviest stones. Return weight of last stone (or 0).

🔍 Pattern Recognition

This is a HEAP problem. Key signals: top-K elements, dynamic ordering, median. Python `heapq` is a min-heap; negate values for max-heap.

🔍 Brute Force Approach

Sort the entire array to find top-k. $O(n \log n)$. Heap gives $O(n \log k)$ which is better when $k \ll n$.

⚡ Optimized Solution

Max-heap (negate values for Python). Pop two, compute difference, push back if non-zero.

🔍 Python Code

```
# LeetCode 1046: Last Stone Weight
# Pattern: Heap / Top K

import heapq
heap=[-s for s in stones]
heapq.heapify(heap)
while len(heap)>1:
    a=heapq.heappop(heap); b=heapq.heappop(heap)
    if a!=b: heapq.heappush(heap,a-b) # a>=b so a-b<0 is correct
return -heap[0] if heap else 0
```

🔍 Dry Run

stones=[2,7,4,1,8,1] → 1

🔍 Complexity Analysis

$O(n \log n)$ time, $O(n)$ space.

🔍 Interview Tips

Know Python `heapq` is min-heap. For max-heap, push `-value`. Explain time complexity: $O(\log k)$ per operation. Follow-up: 'Can you do $O(n)$ with `quickselect`?'

Pattern 11 Summary: Heap / Top K

Core intuition: partial sort — keep only top-k elements. Min-heap of size k always has kth largest at top.

Recognition tricks: 'K largest/smallest/frequent', 'dynamic median'. Python `heapq` is always min-heap.

Pattern 12: Intervals

Problems: LC 56, LC 57, LC 252, LC 253, LC 435, LC 452

Problem 1: LC 56 — Merge Intervals

LeetCode #56 · [leetcode.com/problems/](https://leetcode.com/problems/merge-intervals/)

🔗 Problem Understanding

Merge all overlapping intervals.

🔗 Pattern Recognition

This is an INTERVALS problem. Key signals: overlapping ranges, meeting rooms, merge operations. Always sort by start (or end) time first.

🔗 Brute Force Approach

Check every pair of intervals for overlap. $O(n^2)$. Sorting reduces to $O(n \log n)$.

🚀 Optimized Solution

Sort by start. For each interval, if it overlaps with last merged, extend it; else add new.

🔗 Python Code

```
# LeetCode 56: Merge Intervals
# Pattern: Intervals

intervals.sort()
res=[intervals[0]]
for s,e in intervals[1:]:
    if s<=res[-1][1]: res[-1][1]=max(res[-1][1],e)
    else: res.append([s,e])
return res
```

🔗 Dry Run

intervals=[[1,3],[2,6],[8,10],[15,18]] → [[1,6],[8,10],[15,18]]

🔗 Complexity Analysis

$O(n \log n)$ time, $O(n)$ space.

🔗 Interview Tips

Always sort first. Draw out the intervals on paper during interview. Edge cases: single interval, fully overlapping intervals, touching intervals (e.g., [1,2],[2,3]).

Problem 2: LC 57 — Insert Interval

LeetCode #57 · [leetcode.com/problems/](https://leetcode.com/problems/insert-interval/)

🔗 Problem Understanding

Insert a new interval into sorted non-overlapping intervals and merge if needed.

🔍 Pattern Recognition

This is an INTERVALS problem. Key signals: overlapping ranges, meeting rooms, merge operations. Always sort by start (or end) time first.

🔍 Brute Force Approach

Check every pair of intervals for overlap. $O(n^2)$. Sorting reduces to $O(n \log n)$.

⚡ Optimized Solution

Add all intervals that end before new starts. Merge overlapping. Add remaining.

🔍 Python Code

```
# LeetCode 57: Insert Interval
# Pattern: Intervals

res=[]; i=0; n=len(intervals)
while i<n and intervals[i][1]<newInterval[0]:
    res.append(intervals[i]); i+=1
while i<n and intervals[i][0]<=newInterval[1]:
    newInterval[0]=min(newInterval[0],intervals[i][0])
    newInterval[1]=max(newInterval[1],intervals[i][1]); i+=1
res.append(newInterval)
while i<n: res.append(intervals[i]); i+=1
return res
```

🔍 Dry Run

intervals=[[1,3],[6,9]], newInterval=[2,5] → [[1,5],[6,9]]

🔍 Complexity Analysis

$O(n)$ time, $O(n)$ space.

🔍 Interview Tips

Always sort first. Draw out the intervals on paper during interview. Edge cases: single interval, fully overlapping intervals, touching intervals (e.g., [1,2],[2,3]).

Problem 3: LC 252 — Meeting Rooms

LeetCode #252 · [leetcode.com/problems/](https://leetcode.com/problems/meeting-rooms/)

🔍 Problem Understanding

Can a person attend all meetings? Return True if no two meetings overlap.

🔍 Pattern Recognition

This is an INTERVALS problem. Key signals: overlapping ranges, meeting rooms, merge operations. Always sort by start (or end) time first.

🔍 Brute Force Approach

Check every pair of intervals for overlap. $O(n^2)$. Sorting reduces to $O(n \log n)$.

⚡ Optimized Solution

Sort by start time. If any meeting starts before previous ends, return False.

🔗 Python Code

```
# LeetCode 252: Meeting Rooms
# Pattern: Intervals

intervals.sort()
for i in range(1, len(intervals)):
    if intervals[i][0] < intervals[i-1][1]: return False
return True
```

🔗 Dry Run

intervals=[[0,30],[5,10],[15,20]] → False

🔗 Complexity Analysis

$O(n \log n)$ time.

🔗 Interview Tips

Always sort first. Draw out the intervals on paper during interview. Edge cases: single interval, fully overlapping intervals, touching intervals (e.g., [1,2],[2,3]).

Problem 4: LC 253 — Meeting Rooms II

LeetCode #253 · [leetcode.com/problems/](https://leetcode.com/problems/meeting-rooms-ii/)

🔗 Problem Understanding

Find minimum number of conference rooms required.

🔗 Pattern Recognition

This is an INTERVALS problem. Key signals: overlapping ranges, meeting rooms, merge operations. Always sort by start (or end) time first.

🔗 Brute Force Approach

Check every pair of intervals for overlap. $O(n^2)$. Sorting reduces to $O(n \log n)$.

⚡ Optimized Solution

Sort starts and ends separately. Two-pointer: if next start < next end, need new room.

🔗 Python Code

```
# LeetCode 253: Meeting Rooms II
# Pattern: Intervals

starts=sorted(i[0] for i in intervals)
ends=sorted(i[1] for i in intervals)
rooms=e=s=0
for _ in intervals:
    if starts[s]<ends[e]: rooms+=1; s+=1
```

```
    else: e+=1; s+=1
return rooms
```

🔗 Dry Run

intervals=[[0,30],[5,10],[15,20]] → 2

🔗 Complexity Analysis

$O(n \log n)$ time, $O(n)$ space.

🔗 Interview Tips

Always sort first. Draw out the intervals on paper during interview. Edge cases: single interval, fully overlapping intervals, touching intervals (e.g., [1,2],[2,3]).

Problem 5: LC 435 — Non-overlapping Intervals

LeetCode #435 · leetcode.com/problems/

🔗 Problem Understanding

Find minimum number of intervals to remove to make rest non-overlapping.

🔗 Pattern Recognition

This is an INTERVALS problem. Key signals: overlapping ranges, meeting rooms, merge operations. Always sort by start (or end) time first.

🔗 Brute Force Approach

Check every pair of intervals for overlap. $O(n^2)$. Sorting reduces to $O(n \log n)$.

⚡ Optimized Solution

Greedy: sort by end time. Keep interval with earliest end. Count removed.

🔗 Python Code

```
# LeetCode 435: Non-overlapping Intervals
# Pattern: Intervals

intervals.sort(key=lambda x:x[1])
prev_end=float('-inf'); removed=0
for s,e in intervals:
    if s>=prev_end: prev_end=e
    else: removed+=1
return removed
```

🔗 Dry Run

intervals=[[1,2],[2,3],[3,4],[1,3]] → 1

🔗 Complexity Analysis

$O(n \log n)$ time.

🔗 Interview Tips

Always sort first. Draw out the intervals on paper during interview. Edge cases: single interval, fully overlapping intervals, touching intervals (e.g., [1,2],[2,3]).

Problem 6: LC 452 — Minimum Number of Arrows to Burst Balloons

LeetCode #452 · [leetcode.com/problems/](https://leetcode.com/problems/minimum-number-of-arrows-to-burst-balloons/)

🔍 Problem Understanding

Find minimum arrows to burst all balloons (intervals on x-axis).

🔍 Pattern Recognition

This is an INTERVALS problem. Key signals: overlapping ranges, meeting rooms, merge operations. Always sort by start (or end) time first.

🔍 Brute Force Approach

Check every pair of intervals for overlap. $O(n^2)$. Sorting reduces to $O(n \log n)$.

⚡ Optimized Solution

Sort by end. Shoot at end of first balloon. If next balloon starts after current arrow, shoot again.

🔍 Python Code

```
# LeetCode 452: Minimum Number of Arrows to Burst Balloons
# Pattern: Intervals

points.sort(key=lambda x:x[1])
arrows=1; arrow_x=points[0][1]
for s,e in points[1:]:
    if s>arrow_x: arrows+=1; arrow_x=e
return arrows
```

🔍 Dry Run

points=[[10,16],[2,8],[1,6],[7,12]] → 2

🔍 Complexity Analysis

$O(n \log n)$ time.

🔍 Interview Tips

Always sort first. Draw out the intervals on paper during interview. Edge cases: single interval, fully overlapping intervals, touching intervals (e.g., [1,2],[2,3]).

Pattern 12 Summary: Intervals

Core intuition: sort + single-pass merge/count. Recognition tricks: 'overlapping ranges', 'meeting rooms', 'schedule'. Key: sort by start for merging, sort by end for greedy scheduling.

Pattern 13: Greedy Scheduling / Sorting

Problems: LC 55, LC 45, LC 406, LC 621, LC 763, LC 134

Problem 1: LC 55 — Jump Game

LeetCode #55 · leetcode.com/problems/

🔍 Problem Understanding

Can you reach the last index? Each element is max jump length.

🔍 Pattern Recognition

This is a GREEDY problem. Key signals: local optimal = global optimal, scheduling, jump games. Prove greedy choice is safe before coding.

🔍 Brute Force Approach

Try all possible choices at each step (dynamic programming or recursion). Often exponential time.

⚡ Optimized Solution

Track the farthest position reachable. If current index > farthest, can't proceed.

🔍 Python Code

```
# LeetCode 55: Jump Game
# Pattern: Greedy Scheduling / Sorting

reach=0
for i,n in enumerate(nums):
    if i>reach: return False
    reach=max(reach,i+n)
return True
```

🔍 Dry Run

nums=[2,3,1,1,4] → True; nums=[3,2,1,0,4] → False

🔍 Complexity Analysis

O(n) time, O(1) space.

🔍 Interview Tips

Prove your greedy choice is safe: 'If I make any other choice here, I can only do equal or worse.' Edge cases: single element, all same, negative values.

Problem 2: LC 45 — Jump Game II

LeetCode #45 · leetcode.com/problems/

🔍 Problem Understanding

Minimum number of jumps to reach last index.

🔍 Pattern Recognition

This is a GREEDY problem. Key signals: local optimal = global optimal, scheduling, jump games. Prove greedy choice is safe before coding.

🔍 Brute Force Approach

Try all possible choices at each step (dynamic programming or recursion). Often exponential time.

⚡ Optimized Solution

BFS-like: track current window end and farthest reachable. Jump when window exhausted.

🔍 Python Code

```
# LeetCode 45: Jump Game II
# Pattern: Greedy Scheduling / Sorting

jumps=cur_end=farthest=0
for i in range(len(nums)-1):
    farthest=max(farthest,i+nums[i])
    if i==cur_end: jumps+=1; cur_end=farthest
return jumps
```

🔍 Dry Run

nums=[2,3,1,1,4] → 2

🔍 Complexity Analysis

O(n) time, O(1) space.

🔍 Interview Tips

Prove your greedy choice is safe: 'If I make any other choice here, I can only do equal or worse.' Edge cases: single element, all same, negative values.

Problem 3: LC 406 — Queue Reconstruction by Height

LeetCode #406 · leetcode.com/problems/

🔍 Problem Understanding

Reconstruct queue where people[i]=[h,k]: height h, exactly k taller people in front.

🔍 Pattern Recognition

This is a GREEDY problem. Key signals: local optimal = global optimal, scheduling, jump games. Prove greedy choice is safe before coding.

🔍 Brute Force Approach

Try all possible choices at each step (dynamic programming or recursion). Often exponential time.

⚡ Optimized Solution

Sort by height descending, then k ascending. Insert each person at index k.

Python Code

```
# LeetCode 406: Queue Reconstruction by Height
# Pattern: Greedy Scheduling / Sorting

people.sort(key=lambda x: (-x[0], x[1]))
res=[]
for p in people: res.insert(p[1], p)
return res
```

Dry Run

people=[[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]] → [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]

Complexity Analysis

$O(n^2)$ time due to insertions.

Interview Tips

Prove your greedy choice is safe: 'If I make any other choice here, I can only do equal or worse.' Edge cases: single element, all same, negative values.

Problem 4: LC 621 — Task Scheduler

LeetCode #621 · leetcode.com/problems/

Problem Understanding

Minimum intervals to execute tasks with n cooldown between same task.

Pattern Recognition

This is a GREEDY problem. Key signals: local optimal = global optimal, scheduling, jump games. Prove greedy choice is safe before coding.

Brute Force Approach

Try all possible choices at each step (dynamic programming or recursion). Often exponential time.

Optimized Solution

Most frequent task determines the frame. Formula: $\max(\text{len}(\text{tasks}), (\text{max_freq} - 1) * (n + 1) + \text{count_of_max_freq})$.

Python Code

```
# LeetCode 621: Task Scheduler
# Pattern: Greedy Scheduling / Sorting

from collections import Counter
count=Counter(tasks)
max_freq=max(count.values())
max_count=sum(1 for v in count.values() if v==max_freq)
return max(len(tasks), (max_freq-1)*(n+1)+max_count)
```

Dry Run

tasks=['A','A','A','B','B','B'], n=2 → 8

🔖 Complexity Analysis

$O(n)$ time, $O(1)$ space.

🔖 Interview Tips

Prove your greedy choice is safe: 'If I make any other choice here, I can only do equal or worse.' Edge cases: single element, all same, negative values.

Problem 5: LC 763 — Partition Labels

LeetCode #763 · [leetcode.com/problems/](https://leetcode.com/problems/partition-labels/)

🔖 Problem Understanding

Partition string into max parts so each letter appears in only one part.

🔖 Pattern Recognition

This is a GREEDY problem. Key signals: local optimal = global optimal, scheduling, jump games. Prove greedy choice is safe before coding.

🔖 Brute Force Approach

Try all possible choices at each step (dynamic programming or recursion). Often exponential time.

⚡ Optimized Solution

For each char, record last occurrence. Greedily extend current partition to include last occurrence of all chars seen.

🔖 Python Code

```
# LeetCode 763: Partition Labels
# Pattern: Greedy Scheduling / Sorting

last={c:i for i,c in enumerate(s)}
res=[]; start=end=0
for i,c in enumerate(s):
    end=max(end,last[c])
    if i==end: res.append(end-start+1); start=i+1
return res
```

🔖 Dry Run

`s='ababcbacadefegdehijhklij'` → [9,7,8]

🔖 Complexity Analysis

$O(n)$ time, $O(1)$ space.

🔖 Interview Tips

Prove your greedy choice is safe: 'If I make any other choice here, I can only do equal or worse.' Edge cases: single element, all same, negative values.

Problem 6: LC 134 — Gas Station

LeetCode #134 · [leetcode.com/problems/](https://leetcode.com/problems/gas-station/)

🔍 Problem Understanding

Can you complete a circular route? Find starting gas station index.

🔍 Pattern Recognition

This is a GREEDY problem. Key signals: local optimal = global optimal, scheduling, jump games. Prove greedy choice is safe before coding.

🔍 Brute Force Approach

Try all possible choices at each step (dynamic programming or recursion). Often exponential time.

⚡ Optimized Solution

If total gas $\geq$ total cost, solution exists. Start from candidate where running sum drops below 0.

🔍 Python Code

```
# LeetCode 134: Gas Station
# Pattern: Greedy Scheduling / Sorting

total=start=tank=0
for i in range(len(gas)):
    diff=gas[i]-cost[i]
    total+=diff; tank+=diff
    if tank<0: start=i+1; tank=0
return start if total>=0 else -1
```

🔍 Dry Run

gas=[1,2,3,4,5], cost=[3,4,5,1,2] → 3

🔍 Complexity Analysis

$O(n)$ time, $O(1)$ space.

🔍 Interview Tips

Prove your greedy choice is safe: 'If I make any other choice here, I can only do equal or worse.' Edge cases: single element, all same, negative values.

Pattern 13 Summary: Greedy Scheduling / Sorting

Core intuition: locally optimal choices lead to globally optimal solution. Recognition tricks: 'minimum jumps', 'maximum reach', scheduling. Always ask: why can't a different choice be better?

Pattern 14: Linked List Manipulation

Problems: LC 21, LC 23, LC 24, LC 25, LC 92, LC 138

Problem 1: LC 21 — Merge Two Sorted Lists

LeetCode #21 · leetcode.com/problems/

🔍 Problem Understanding

Merge two sorted linked lists into one sorted list.

🔍 Pattern Recognition

This is a LINKED LIST problem. Key signals: pointer manipulation, in-place operations. Always use a dummy head node to simplify edge cases.

🔍 Brute Force Approach

Use extra data structures (arrays, sets) to simplify pointer operations. Extra $O(n)$ space.

⚡ Optimized Solution

Dummy head. Compare heads of both lists; append smaller one. Attach remaining list at end.

🔍 Python Code

```
# LeetCode 21: Merge Two Sorted Lists
# Pattern: Linked List Manipulation

dummy=ListNode(0); cur=dummy
while l1 and l2:
    if l1.val<=l2.val: cur.next=l1; l1=l1.next
    else: cur.next=l2; l2=l2.next
    cur=cur.next
cur.next=l1 or l2
return dummy.next
```

🔍 Dry Run

$l1=[1,2,4], l2=[1,3,4] \rightarrow [1,1,2,3,4,4]$

🔍 Complexity Analysis

$O(n+m)$ time, $O(1)$ space.

🔍 Interview Tips

Always use a dummy head. Draw the pointer diagram before coding. Edge cases: empty list, single node, k = length of list.

Problem 2: LC 23 — Merge k Sorted Lists

LeetCode #23 · leetcode.com/problems/

🔗 Problem Understanding

Merge k sorted linked lists.

🔗 Pattern Recognition

This is a LINKED LIST problem. Key signals: pointer manipulation, in-place operations. Always use a dummy head node to simplify edge cases.

🔗 Brute Force Approach

Use extra data structures (arrays, sets) to simplify pointer operations. Extra $O(n)$ space.

⚡ Optimized Solution

Use a min-heap. Push first node of each list. Pop min, add to result, push its next.

🔗 Python Code

```
# LeetCode 23: Merge k Sorted Lists
# Pattern: Linked List Manipulation

import heapq
heap=[]
for i,node in enumerate(lists):
    if node: heapq.heappush(heap, (node.val,i,node))
dummy=cur=ListNode(0)
while heap:
    val,i,node=heapq.heappop(heap)
    cur.next=node; cur=cur.next
    if node.next: heapq.heappush(heap, (node.next.val,i,node.next))
return dummy.next
```

🔗 Dry Run

lists=[[1,4,5],[1,3,4],[2,6]] → [1,1,2,3,4,4,5,6]

🔗 Complexity Analysis

$O(N \log k)$ where N =total nodes.

🔗 Interview Tips

Always use a dummy head. Draw the pointer diagram before coding. Edge cases: empty list, single node, k = length of list.

Problem 3: LC 24 — Swap Nodes in Pairs

LeetCode #24 · leetcode.com/problems/

🔗 Problem Understanding

Swap every two adjacent nodes.

🔗 Pattern Recognition

This is a LINKED LIST problem. Key signals: pointer manipulation, in-place operations. Always use a dummy head node to simplify edge cases.

🔍 Brute Force Approach

Use extra data structures (arrays, sets) to simplify pointer operations. Extra $O(n)$ space.

⚡ Optimized Solution

Use dummy head. For each pair: change pointers to swap the two nodes.

🔍 Python Code

```
# LeetCode 24: Swap Nodes in Pairs
# Pattern: Linked List Manipulation

dummy=ListNode(0,head); prev=dummy
while prev.next and prev.next.next:
    a=prev.next; b=prev.next.next
    prev.next=b; a.next=b.next; b.next=a
    prev=a
return dummy.next
```

🔍 Dry Run

head=[1,2,3,4] → [2,1,4,3]

🔍 Complexity Analysis

$O(n)$ time, $O(1)$ space.

🔍 Interview Tips

Always use a dummy head. Draw the pointer diagram before coding. Edge cases: empty list, single node, $k = \text{length of list}$.

Problem 4: LC 25 — Reverse Nodes in k-Group

LeetCode #25 · [leetcode.com/problems/](https://leetcode.com/problems/reverse-nodes-in-k-group/)

🔍 Problem Understanding

Reverse nodes in groups of k . Leave remaining nodes as-is.

🔍 Pattern Recognition

This is a LINKED LIST problem. Key signals: pointer manipulation, in-place operations. Always use a dummy head node to simplify edge cases.

🔍 Brute Force Approach

Use extra data structures (arrays, sets) to simplify pointer operations. Extra $O(n)$ space.

⚡ Optimized Solution

Check if k nodes exist. Reverse k nodes. Recursively handle rest.

🔍 Python Code

```
# LeetCode 25: Reverse Nodes in k-Group
# Pattern: Linked List Manipulation

def reverseKGroup(head, k):
```



```

cur=head
for _ in range(k):
    if not cur: return head # fewer than k nodes, no reverse
    cur=cur.next
# reverse k nodes
prev=None; cur=head
for _ in range(k): cur.next,prev,cur=prev,cur,cur.next
head.next=reverseKGroup(cur,k)
return prev

```

🔗 Dry Run

head=[1,2,3,4,5], k=2 → [2,1,4,3,5]

🔗 Complexity Analysis

O(n) time, O(n/k) space recursion.

🔗 Interview Tips

Always use a dummy head. Draw the pointer diagram before coding. Edge cases: empty list, single node, k = length of list.

Problem 5: LC 92 — Reverse Linked List II

LeetCode #92 · [leetcode.com/problems/](https://leetcode.com/problems/reverse-linked-list-ii/)

🔗 Problem Understanding

Reverse nodes from position left to right.

🔗 Pattern Recognition

This is a LINKED LIST problem. Key signals: pointer manipulation, in-place operations. Always use a dummy head node to simplify edge cases.

🔗 Brute Force Approach

Use extra data structures (arrays, sets) to simplify pointer operations. Extra O(n) space.

⚡ Optimized Solution

Use dummy head. Walk to left-1. Reverse the sublist. Re-attach.

🔗 Python Code

```

# LeetCode 92: Reverse Linked List II
# Pattern: Linked List Manipulation

dummy=ListNode(0,head); prev=dummy
for _ in range(left-1): prev=prev.next
cur=prev.next
for _ in range(right-left):
    nxt=cur.next
    cur.next=nxt.next
    nxt.next=prev.next
    prev.next=nxt
return dummy.next

```

🔍 Dry Run

head=[1,2,3,4,5], left=2, right=4 → [1,4,3,2,5]

🔍 Complexity Analysis

O(n) time, O(1) space.

🔍 Interview Tips

Always use a dummy head. Draw the pointer diagram before coding. Edge cases: empty list, single node, k = length of list.

Problem 6: LC 138 — Copy List with Random Pointer

LeetCode #138 · leetcode.com/problems/

🔍 Problem Understanding

Deep copy a linked list where each node has next and random pointers.

🔍 Pattern Recognition

This is a LINKED LIST problem. Key signals: pointer manipulation, in-place operations. Always use a dummy head node to simplify edge cases.

🔍 Brute Force Approach

Use extra data structures (arrays, sets) to simplify pointer operations. Extra O(n) space.

⚡ Optimized Solution

Hash map old→new nodes. First pass: create all new nodes. Second pass: assign next and random.

🔍 Python Code

```
# LeetCode 138: Copy List with Random Pointer
# Pattern: Linked List Manipulation

import copy
if not head: return None
old_to_new={}
cur=head
while cur: old_to_new[cur]=Node(cur.val); cur=cur.next
cur=head
while cur:
    if cur.next: old_to_new[cur].next=old_to_new[cur.next]
    if cur.random: old_to_new[cur].random=old_to_new[cur.random]
    cur=cur.next
return old_to_new[head]
```

🔍 Dry Run

head=[[7,null],[13,0],[11,4],[10,2],[1,0]] → deep copy

🔍 Complexity Analysis

O(n) time, O(n) space.

🔗 Interview Tips

Always use a dummy head. Draw the pointer diagram before coding. Edge cases: empty list, single node, $k = \text{length of list}$.

Pattern 14 Summary: Linked List Manipulation

Core intuition: pointer manipulation without extra space. Key techniques: dummy head (avoids edge cases), fast/slow pointers, reverse in-place. Recognition tricks: any 'in-place' linked list operation.

Pattern 15: Tree DFS

Problems: LC 104, LC 226, LC 112, LC 113, LC 543, LC 124

Problem 1: LC 104 — Maximum Depth of Binary Tree

LeetCode #104 · leetcode.com/problems/

🔍 Problem Understanding

Find the maximum depth (number of nodes along longest root-to-leaf path).

🔍 Pattern Recognition

This is a TREE DFS problem. Key signals: path problems, depth, subtree properties. Think recursively: what do I need from children? What do I return to parent?

🔍 Brute Force Approach

No real brute force for trees; the recursive approach is standard. Iterative with explicit stack is the alternative.

⚡ Optimized Solution

Recursively: $\text{depth} = 1 + \max(\text{left_depth}, \text{right_depth})$. Base: null node = 0.

🔍 Python Code

```
# LeetCode 104: Maximum Depth of Binary Tree
# Pattern: Tree DFS

def maxDepth(root):
    if not root: return 0
    return 1+max(maxDepth(root.left), maxDepth(root.right))
```

🔍 Dry Run

root=[3,9,20,null,null,15,7] → 3

🔍 Complexity Analysis

O(n) time, O(h) space where h=height.

🔍 Interview Tips

For each recursive call, clearly state: 'This function returns X' and 'I need X from children to compute Y at current node.' Edge cases: null root, single node, skewed tree.

Problem 2: LC 226 — Invert Binary Tree

LeetCode #226 · leetcode.com/problems/

🔍 Problem Understanding

Invert (mirror) a binary tree.

🔍 Pattern Recognition

This is a TREE DFS problem. Key signals: path problems, depth, subtree properties. Think recursively: what do I need from children? What do I return to parent?

🔍 Brute Force Approach

No real brute force for trees; the recursive approach is standard. Iterative with explicit stack is the alternative.

⚡ Optimized Solution

Recursively swap left and right children at every node.

🔍 Python Code

```
# LeetCode 226: Invert Binary Tree
# Pattern: Tree DFS

def invertTree(root):
    if not root: return None
    root.left, root.right = invertTree(root.right), invertTree(root.left)
    return root
```

🔍 Dry Run

root=[4,2,7,1,3,6,9] → [4,7,2,9,6,3,1]

🔍 Complexity Analysis

O(n) time, O(h) space.

🔍 Interview Tips

For each recursive call, clearly state: 'This function returns X' and 'I need X from children to compute Y at current node.' Edge cases: null root, single node, skewed tree.

Problem 3: LC 112 — Path Sum

LeetCode #112 · leetcode.com/problems/

🔍 Problem Understanding

Check if there's a root-to-leaf path summing to targetSum.

🔍 Pattern Recognition

This is a TREE DFS problem. Key signals: path problems, depth, subtree properties. Think recursively: what do I need from children? What do I return to parent?

🔍 Brute Force Approach

No real brute force for trees; the recursive approach is standard. Iterative with explicit stack is the alternative.

⚡ Optimized Solution

Recursively subtract node value from target. At leaf, check if remaining==0.

🔗 Python Code

```
# LeetCode 112: Path Sum
# Pattern: Tree DFS

def hasPathSum(root, target):
    if not root: return False
    if not root.left and not root.right: return root.val==target
    return hasPathSum(root.left, target-root.val) or hasPathSum(root.right, target-
root.val)
```

🔗 Dry Run

root=[5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum=22 → True

🔗 Complexity Analysis

O(n) time, O(h) space.

🔗 Interview Tips

For each recursive call, clearly state: 'This function returns X' and 'I need X from children to compute Y at current node.' Edge cases: null root, single node, skewed tree.

Problem 4: LC 113 — Path Sum II

LeetCode #113 · [leetcode.com/problems/](https://leetcode.com/problems/path-sum-ii/)

🔗 Problem Understanding

Return all root-to-leaf paths that sum to targetSum.

🔗 Pattern Recognition

This is a TREE DFS problem. Key signals: path problems, depth, subtree properties. Think recursively: what do I need from children? What do I return to parent?

🔗 Brute Force Approach

No real brute force for trees; the recursive approach is standard. Iterative with explicit stack is the alternative.

⚡ Optimized Solution

DFS with backtracking. Add node to path; at leaf check sum; remove on backtrack.

🔗 Python Code

```
# LeetCode 113: Path Sum II
# Pattern: Tree DFS

def pathSum(root, target):
    res=[]
    def dfs(node, path, rem):
        if not node: return
        path.append(node.val)
        if not node.left and not node.right and rem==node.val:
            res.append(path[:])
        dfs(node.left, path, rem-node.val)
```

```

        dfs(node.right, path, rem-node.val)
        path.pop()
    dfs(root, [], target)
    return res

```

🔍 Dry Run

root=[5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum=22 → [[5,4,11,2],[5,8,4,5]]

🔍 Complexity Analysis

O(n) time, O(n) space.

🔍 Interview Tips

For each recursive call, clearly state: 'This function returns X' and 'I need X from children to compute Y at current node.' Edge cases: null root, single node, skewed tree.

Problem 5: LC 543 — Diameter of Binary Tree

LeetCode #543 · leetcode.com/problems/

🔍 Problem Understanding

Find the length of the longest path between any two nodes (not necessarily through root).

🔍 Pattern Recognition

This is a TREE DFS problem. Key signals: path problems, depth, subtree properties. Think recursively: what do I need from children? What do I return to parent?

🔍 Brute Force Approach

No real brute force for trees; the recursive approach is standard. Iterative with explicit stack is the alternative.

⚡ Optimized Solution

At each node, diameter through it = left_depth + right_depth. Track global max.

🔍 Python Code

```

# LeetCode 543: Diameter of Binary Tree
# Pattern: Tree DFS

res=[0]
def depth(node):
    if not node: return 0
    l,r=depth(node.left),depth(node.right)
    res[0]=max(res[0],l+r)
    return 1+max(l,r)
depth(root)
return res[0]

```

🔍 Dry Run

root=[1,2,3,4,5] → 3

🔍 Complexity Analysis

$O(n)$ time, $O(h)$ space.

🔖 Interview Tips

For each recursive call, clearly state: 'This function returns X' and 'I need X from children to compute Y at current node.' Edge cases: null root, single node, skewed tree.

Problem 6: LC 124 — Binary Tree Maximum Path Sum

LeetCode #124 · [leetcode.com/problems/](https://leetcode.com/problems/binary-tree-maximum-path-sum/)

🔖 Problem Understanding

Find maximum path sum (path can start and end anywhere).

🔖 Pattern Recognition

This is a TREE DFS problem. Key signals: path problems, depth, subtree properties. Think recursively: what do I need from children? What do I return to parent?

🔖 Brute Force Approach

No real brute force for trees; the recursive approach is standard. Iterative with explicit stack is the alternative.

⚡ Optimized Solution

At each node: $\text{max contribution} = \text{node.val} + \max(0, \text{left_contrib}, \text{right_contrib})$. Update global max with $\text{left} + \text{right} + \text{node}$.

🔖 Python Code

```
# LeetCode 124: Binary Tree Maximum Path Sum
# Pattern: Tree DFS

res=[float('-inf')]
def dfs(node):
    if not node: return 0
    l=max(0,dfs(node.left))
    r=max(0,dfs(node.right))
    res[0]=max(res[0],node.val+l+r)
    return node.val+max(l,r)
dfs(root)
return res[0]
```

🔖 Dry Run

$\text{root} = [-10, 9, 20, \text{null}, \text{null}, 15, 7] \rightarrow 42$ ($15 + 20 + 7$)

🔖 Complexity Analysis

$O(n)$ time, $O(h)$ space.

🔖 Interview Tips

For each recursive call, clearly state: 'This function returns X' and 'I need X from children to compute Y at current node.' Edge cases: null root, single node, skewed tree.

Pattern 15 Summary: Tree DFS

Core intuition: post-order DFS for subtree aggregation, pre-order for top-down passing. Key question: 'What does this recursive call return?' Recognition tricks: path problems, diameter, max sum.

Pattern 16: Tree BFS / Level Order

Problems: LC 102, LC 103, LC 199, LC 515, LC 637, LC 116

Problem 1: LC 102 — Binary Tree Level Order Traversal

LeetCode #102 · leetcode.com/problems/

🔍 Problem Understanding

Return node values level by level.

🔍 Pattern Recognition

This is a TREE BFS problem. Key signals: level-by-level processing, shortest path in tree. Use a queue and process all nodes at each level before next.

🔍 Brute Force Approach

DFS with tracking depth is the brute force alternative to BFS. BFS is natural for level-order problems.

🚀 Optimized Solution

BFS with a queue. For each level, process all nodes and enqueue their children.

🔍 Python Code

```
# LeetCode 102: Binary Tree Level Order Traversal
# Pattern: Tree BFS / Level Order

from collections import deque
if not root: return []
res=[]; q=deque([root])
while q:
    level=[]
    for _ in range(len(q)):
        node=q.popleft()
        level.append(node.val)
        if node.left: q.append(node.left)
        if node.right: q.append(node.right)
    res.append(level)
return res
```

🔍 Dry Run

root=[3,9,20,null,null,15,7] → [[3],[9,20],[15,7]]

🔍 Complexity Analysis

O(n) time, O(n) space.

🔍 Interview Tips

BFS template is always: queue → while queue → pop → process → push children. Edge cases: null root, single node, very wide tree.

Problem 2: LC 103 — Binary Tree Zigzag Level Order Traversal

LeetCode #103 · leetcode.com/problems/

🔍 Problem Understanding

Level order but alternate left-to-right and right-to-left.

🔍 Pattern Recognition

This is a TREE BFS problem. Key signals: level-by-level processing, shortest path in tree. Use a queue and process all nodes at each level before next.

🔍 Brute Force Approach

DFS with tracking depth is the brute force alternative to BFS. BFS is natural for level-order problems.

⚡ Optimized Solution

Standard BFS. For odd levels, reverse the level list before appending.

🔍 Python Code

```
# LeetCode 103: Binary Tree Zigzag Level Order Traversal
# Pattern: Tree BFS / Level Order

from collections import deque
if not root: return []
res=[]; q=deque([root]); left_to_right=True
while q:
    level=[]
    for _ in range(len(q)):
        node=q.popleft()
        level.append(node.val)
        if node.left: q.append(node.left)
        if node.right: q.append(node.right)
    res.append(level if left_to_right else level[::-1])
    left_to_right=not left_to_right
return res
```

🔍 Dry Run

root=[3,9,20,null,null,15,7] → [[3],[20,9],[15,7]]

🔍 Complexity Analysis

O(n) time, O(n) space.

🔍 Interview Tips

BFS template is always: queue → while queue → pop → process → push children. Edge cases: null root, single node, very wide tree.

Problem 3: LC 199 — Binary Tree Right Side View

LeetCode #199 · leetcode.com/problems/

🔍 Problem Understanding

Return values visible from the right side (rightmost node at each level).

🔍 Pattern Recognition

This is a TREE BFS problem. Key signals: level-by-level processing, shortest path in tree. Use a queue and process all nodes at each level before next.

🔍 Brute Force Approach

DFS with tracking depth is the brute force alternative to BFS. BFS is natural for level-order problems.

⚡ Optimized Solution

BFS. Last element of each level is visible from right.

🔍 Python Code

```
# LeetCode 199: Binary Tree Right Side View
# Pattern: Tree BFS / Level Order

from collections import deque
if not root: return []
res=[]; q=deque([root])
while q:
    for i in range(len(q)):
        node=q.popleft()
        if i==len(q): res.append(node.val) # last in level
        if node.left: q.append(node.left)
        if node.right: q.append(node.right)
    return res
# simpler: res.append(level[-1]) after building full level
```

🔍 Dry Run

root=[1,2,3,null,5,null,4] → [1,3,4]

🔍 Complexity Analysis

O(n) time.

🔍 Interview Tips

BFS template is always: queue → while queue → pop → process → push children. Edge cases: null root, single node, very wide tree.

Problem 4: LC 515 — Find Largest Value in Each Tree Row

LeetCode #515 · leetcode.com/problems/

🔍 Problem Understanding

Find max value in each level of the tree.

🔍 Pattern Recognition

This is a TREE BFS problem. Key signals: level-by-level processing, shortest path in tree. Use a queue and process all nodes at each level before next.

🔍 Brute Force Approach

DFS with tracking depth is the brute force alternative to BFS. BFS is natural for level-order problems.

⚡ Optimized Solution

BFS. Track max value while processing each level.

🔗 Python Code

```
# LeetCode 515: Find Largest Value in Each Tree Row
# Pattern: Tree BFS / Level Order

from collections import deque
if not root: return []
res=[]; q=deque([root])
while q:
    level_max=float('-inf')
    for _ in range(len(q)):
        node=q.popleft()
        level_max=max(level_max,node.val)
        if node.left: q.append(node.left)
        if node.right: q.append(node.right)
    res.append(level_max)
return res
```

🔗 Dry Run

root=[1,3,2,5,3,null,9] → [1,3,9]

🔗 Complexity Analysis

O(n) time.

🔗 Interview Tips

BFS template is always: queue → while queue → pop → process → push children. Edge cases: null root, single node, very wide tree.

Problem 5: LC 637 — Average of Levels in Binary Tree

LeetCode #637 · [leetcode.com/problems/](https://leetcode.com/problems/average-of-levels-in-binary-tree/)

🔗 Problem Understanding

Return average value of nodes at each level.

🔗 Pattern Recognition

This is a TREE BFS problem. Key signals: level-by-level processing, shortest path in tree. Use a queue and process all nodes at each level before next.

🔗 Brute Force Approach

DFS with tracking depth is the brute force alternative to BFS. BFS is natural for level-order problems.

⚡ Optimized Solution

BFS. Compute sum / count for each level.

🔗 Python Code

```
# LeetCode 637: Average of Levels in Binary Tree
# Pattern: Tree BFS / Level Order

from collections import deque
if not root: return []
res=[]; q=deque([root])
while q:
    total=0; count=len(q)
    for _ in range(count):
        node=q.popleft(); total+=node.val
        if node.left: q.append(node.left)
        if node.right: q.append(node.right)
    res.append(total/count)
return res
```

🔍 Dry Run

root=[3,9,20,null,null,15,7] → [3.0,14.5,11.0]

🔍 Complexity Analysis

O(n) time.

🔍 Interview Tips

BFS template is always: queue → while queue → pop → process → push children. Edge cases: null root, single node, very wide tree.

Problem 6: LC 116 — Populating Next Right Pointers in Each Node

LeetCode #116 · [leetcode.com/problems/](https://leetcode.com/problems/populating-next-right-pointers-in-each-node/)

🔍 Problem Understanding

Connect each node to its next right node on the same level. Perfect binary tree.

🔍 Pattern Recognition

This is a TREE BFS problem. Key signals: level-by-level processing, shortest path in tree. Use a queue and process all nodes at each level before next.

🔍 Brute Force Approach

DFS with tracking depth is the brute force alternative to BFS. BFS is natural for level-order problems.

⚡ Optimized Solution

BFS or use already-connected next pointers to link children level by level.

🔍 Python Code

```
# LeetCode 116: Populating Next Right Pointers in Each Node
# Pattern: Tree BFS / Level Order

if not root: return root
q=deque([root])
while q:
    for i in range(len(q)-1): q[i].next=q[i+1]
    for _ in range(len(q)):
```

```
node=q.popleft()
if node.left: q.append(node.left)
if node.right: q.append(node.right)
return root
```

🔗 Dry Run

root=[1,2,3,4,5,6,7] → each node points to right neighbor

🔗 Complexity Analysis

O(n) time, O(n) space.

🔗 Interview Tips

BFS template is always: queue → while queue → pop → process → push children. Edge cases: null root, single node, very wide tree.

Pattern 16 Summary: Tree BFS / Level Order

Core intuition: queue processes nodes level by level. Pattern: while queue → snapshot level size → process all in level → push children. Recognition tricks: anything level-related.

Pattern 17: BST Problems

Problems: LC 700, LC 98, LC 230, LC 235, LC 450, LC 99

Problem 1: LC 700 — Search in a Binary Search Tree

LeetCode #700 · leetcode.com/problems/

🔖 Problem Understanding

Find node with given val in BST. Return subtree rooted at that node.

🔖 Pattern Recognition

This is a BST problem. Key signals: sorted property, find kth element, validate ordering. BST in-order = sorted array.

🔖 Brute Force Approach

Convert BST to sorted array via in-order traversal, then answer query on array. Extra $O(n)$ space.

🚀 Optimized Solution

BST property: go left if $val < node.val$, right if $val > node.val$.

🔖 Python Code

```
# LeetCode 700: Search in a Binary Search Tree
# Pattern: BST Problems

def searchBST(root, val):
    if not root or root.val == val: return root
    if val < root.val: return searchBST(root.left, val)
    return searchBST(root.right, val)
```

🔖 Dry Run

root=[4,2,7,1,3], val=2 → [2,1,3]

🔖 Complexity Analysis

$O(h)$ time, $O(h)$ space.

🔖 Interview Tips

Leverage BST property! 'I don't need to visit every node.' Interviewers expect $O(h)$ solutions. Follow-up: 'What if the BST is unbalanced?'

Problem 2: LC 98 — Validate Binary Search Tree

LeetCode #98 · leetcode.com/problems/

🔖 Problem Understanding

Check if a binary tree is a valid BST.

🔍 Pattern Recognition

This is a BST problem. Key signals: sorted property, find kth element, validate ordering. BST in-order = sorted array.

🔍 Brute Force Approach

Convert BST to sorted array via in-order traversal, then answer query on array. Extra $O(n)$ space.

⚡ Optimized Solution

DFS with min/max bounds. Each node must be strictly within (min, max).

🔍 Python Code

```
# LeetCode 98: Validate Binary Search Tree
# Pattern: BST Problems

def isValidBST(root, min_val=float('-inf'), max_val=float('inf')):
    if not root: return True
    if not (min_val < root.val < max_val): return False
    return (isValidBST(root.left, min_val, root.val) and
            isValidBST(root.right, root.val, max_val))
```

🔍 Dry Run

root=[2,1,3] → True; root=[5,1,4,null,null,3,6] → False

🔍 Complexity Analysis

$O(n)$ time, $O(h)$ space.

🔍 Interview Tips

Leverage BST property! 'I don't need to visit every node.' Interviewers expect $O(h)$ solutions. Follow-up: 'What if the BST is unbalanced?'

Problem 3: LC 230 — Kth Smallest Element in a BST

LeetCode #230 · leetcode.com/problems/

🔍 Problem Understanding

Find kth smallest element in BST.

🔍 Pattern Recognition

This is a BST problem. Key signals: sorted property, find kth element, validate ordering. BST in-order = sorted array.

🔍 Brute Force Approach

Convert BST to sorted array via in-order traversal, then answer query on array. Extra $O(n)$ space.

⚡ Optimized Solution

In-order traversal of BST gives sorted sequence. Return kth element.

🔍 Python Code

```
# LeetCode 230: Kth Smallest Element in a BST
# Pattern: BST Problems

res=[0,k] # [value, count]
def inorder(node):
    if not node or res[1]==0: return
    inorder(node.left)
    res[1]-=1
    if res[1]==0: res[0]=node.val; return
    inorder(node.right)
inorder(root)
return res[0]
```

Dry Run

root=[3,1,4,null,2], k=1 → 1

Complexity Analysis

O(h+k) time, O(h) space.

Interview Tips

Leverage BST property! 'I don't need to visit every node.' Interviewers expect O(h) solutions. Follow-up: 'What if the BST is unbalanced?'

Problem 4: LC 235 — Lowest Common Ancestor of a Binary Search Tree

LeetCode #235 · leetcode.com/problems/

Problem Understanding

Find LCA of two nodes in a BST.

Pattern Recognition

This is a BST problem. Key signals: sorted property, find kth element, validate ordering. BST in-order = sorted array.

Brute Force Approach

Convert BST to sorted array via in-order traversal, then answer query on array. Extra O(n) space.

Optimized Solution

BST property: if both nodes < root, go left. If both > root, go right. Else root is LCA.

Python Code

```
# LeetCode 235: Lowest Common Ancestor of a Binary Search Tree
# Pattern: BST Problems

def lowestCommonAncestor(root,p,q):
    while root:
        if p.val<root.val and q.val<root.val: root=root.left
        elif p.val>root.val and q.val>root.val: root=root.right
        else: return root
```

Dry Run

root=[6,2,8,0,4,7,9,null,null,3,5], p=2, q=8 → 6

🔍 Complexity Analysis

O(h) time, O(1) space.

🔍 Interview Tips

Leverage BST property! 'I don't need to visit every node.' Interviewers expect O(h) solutions. Follow-up: 'What if the BST is unbalanced?'

Problem 5: LC 450 — Delete Node in a BST

LeetCode #450 · [leetcode.com/problems/](https://leetcode.com/problems/delete-node-in-a-bst/)

🔍 Problem Understanding

Delete a node with given key from BST.

🔍 Pattern Recognition

This is a BST problem. Key signals: sorted property, find kth element, validate ordering. BST in-order = sorted array.

🔍 Brute Force Approach

Convert BST to sorted array via in-order traversal, then answer query on array. Extra O(n) space.

⚡ Optimized Solution

Three cases: (1) node is leaf - delete. (2) one child - replace with child. (3) two children - replace with inorder successor.

🔍 Python Code

```
# LeetCode 450: Delete Node in a BST
# Pattern: BST Problems

def deleteNode(root, key):
    if not root: return None
    if key < root.val: root.left = deleteNode(root.left, key)
    elif key > root.val: root.right = deleteNode(root.right, key)
    else:
        if not root.left: return root.right
        if not root.right: return root.left
        # find inorder successor (min of right subtree)
        succ = root.right
        while succ.left: succ = succ.left
        root.val = succ.val
        root.right = deleteNode(root.right, succ.val)
    return root
```

🔍 Dry Run

root=[5,3,6,2,4,null,7], key=3 → [5,4,6,2,null,null,7]

🔍 Complexity Analysis

O(h) time, O(h) space.

🔖 Interview Tips

Leverage BST property! 'I don't need to visit every node.' Interviewers expect $O(h)$ solutions. Follow-up: 'What if the BST is unbalanced?'

Problem 6: LC 99 — Recover Binary Search Tree

LeetCode #99 · leetcode.com/problems/

🔖 Problem Understanding

Two nodes of BST are swapped by mistake. Recover the tree.

🔖 Pattern Recognition

This is a BST problem. Key signals: sorted property, find kth element, validate ordering. BST in-order = sorted array.

🔖 Brute Force Approach

Convert BST to sorted array via in-order traversal, then answer query on array. Extra $O(n)$ space.

⚡ Optimized Solution

In-order traversal finds two nodes that are out of order. Swap their values.

🔖 Python Code

```
# LeetCode 99: Recover Binary Search Tree
# Pattern: BST Problems

first=second=prev=None
def inorder(node):
    nonlocal first, second, prev
    if not node: return
    inorder(node.left)
    if prev and prev.val > node.val:
        if not first: first=prev
        second=node
    prev=node
    inorder(node.right)
inorder(root)
first.val, second.val=second.val, first.val
```

🔖 Dry Run

root=[1,3,null,null,2] → [3,1,null,null,2]

🔖 Complexity Analysis

$O(n)$ time, $O(h)$ space.

🔖 Interview Tips

Leverage BST property! 'I don't need to visit every node.' Interviewers expect $O(h)$ solutions. Follow-up: 'What if the BST is unbalanced?'

Pattern 17 Summary: BST Problems

*Core intuition: BST in-order = sorted array. Use BST property to go left/right without visiting entire tree.
Recognition tricks: 'kth smallest', 'validate BST', 'LCA in BST'.*

Pattern 18: Backtracking Basics

Problems: LC 46, LC 47, LC 77, LC 78, LC 90, LC 39

Problem 1: LC 46 — Permutations

LeetCode #46 · leetcode.com/problems/

🔍 Problem Understanding

Return all permutations of distinct integers.

🔍 Pattern Recognition

This is a BACKTRACKING problem. Key signals: all possible combinations/permutations/subsets. Try → recurse → undo. Draw the decision tree.

🔍 Brute Force Approach

Generate all possibilities without pruning. Pruning (backtracking) dramatically reduces the search space.

⚡ Optimized Solution

Build permutation by choosing each unused number. Backtrack after each choice.

🔍 Python Code

```
# LeetCode 46: Permutations
# Pattern: Backtracking Basics

def permute(nums):
    res=[]
    def bt(path,used):
        if len(path)==len(nums): res.append(path[:]); return
        for n in nums:
            if n not in used:
                path.append(n); used.add(n)
                bt(path,used)
                path.pop(); used.remove(n)
    bt([],set())
    return res
```

🔍 Dry Run

nums=[1,2,3] → [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

🔍 Complexity Analysis

$O(n * n!)$ time, $O(n)$ space.

🔍 Interview Tips

Draw the recursion tree. Explain pruning conditions. State time complexity as $O(\text{branching_factor}^{\text{depth}})$. Edge cases: empty input, single element, target = 0.

Problem 2: LC 47 — Permutations II

LeetCode #47 · leetcode.com/problems/

🔖 Problem Understanding

Return all unique permutations (array may have duplicates).

🔖 Pattern Recognition

This is a BACKTRACKING problem. Key signals: all possible combinations/permutations/subsets. Try → recurse → undo. Draw the decision tree.

🔖 Brute Force Approach

Generate all possibilities without pruning. Pruning (backtracking) dramatically reduces the search space.

🚀 Optimized Solution

Sort first. Skip duplicate elements at same recursion level using 'if i>0 and nums[i]==nums[i-1] and not used[i-1]'.

🔖 Python Code

```
# LeetCode 47: Permutations II
# Pattern: Backtracking Basics

nums.sort(); res=[]; used=[False]*len(nums)
def bt(path):
    if len(path)==len(nums): res.append(path[:]); return
    for i,n in enumerate(nums):
        if used[i]: continue
        if i>0 and nums[i]==nums[i-1] and not used[i-1]: continue
        used[i]=True; path.append(n)
        bt(path)
        path.pop(); used[i]=False
bt([])
return res
```

🔖 Dry Run

nums=[1,1,2] → [[1,1,2],[1,2,1],[2,1,1]]

🔖 Complexity Analysis

O(n * n!) time.

🔖 Interview Tips

Draw the recursion tree. Explain pruning conditions. State time complexity as O(branching_factor^depth). Edge cases: empty input, single element, target = 0.

Problem 3: LC 77 — Combinations

LeetCode #77 · leetcode.com/problems/

🔖 Problem Understanding

Return all combinations of k numbers from 1 to n.

🔍 Pattern Recognition

This is a BACKTRACKING problem. Key signals: all possible combinations/permutations/subsets. Try → recurse → undo. Draw the decision tree.

🔍 Brute Force Approach

Generate all possibilities without pruning. Pruning (backtracking) dramatically reduces the search space.

⚡ Optimized Solution

Choose numbers from start to n. Recurse with start+1 and k-1 remaining.

🔍 Python Code

```
# LeetCode 77: Combinations
# Pattern: Backtracking Basics

res=[]
def bt(start,path):
    if len(path)==k: res.append(path[:]); return
    for i in range(start,n+1):
        path.append(i)
        bt(i+1,path)
        path.pop()
bt(1,[])
return res
```

🔍 Dry Run

n=4, k=2 → [[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]

🔍 Complexity Analysis

$O(C(n,k)*k)$ time.

🔍 Interview Tips

Draw the recursion tree. Explain pruning conditions. State time complexity as $O(\text{branching_factor}^{\text{depth}})$. Edge cases: empty input, single element, target = 0.

Problem 4: LC 78 — Subsets

LeetCode #78 · [leetcode.com/problems/](https://leetcode.com/problems/subsets/)

🔍 Problem Understanding

Return all possible subsets (the power set).

🔍 Pattern Recognition

This is a BACKTRACKING problem. Key signals: all possible combinations/permutations/subsets. Try → recurse → undo. Draw the decision tree.

🔍 Brute Force Approach

Generate all possibilities without pruning. Pruning (backtracking) dramatically reduces the search space.

⚡ Optimized Solution

At each step, either include or skip current element. Add current path to result at every node.

📄 Python Code

```
# LeetCode 78: Subsets
# Pattern: Backtracking Basics

res=[]
def bt(start,path):
    for i in range(start,len(nums)):
        path.append(nums[i])
        res.append(path[:])
        bt(i+1,path)
        path.pop()
bt(0,[])
return res
```

📄 Dry Run

nums=[1,2,3] → [[],[1],[1,2],[1,2,3],[1,3],[2],[2,3],[3]]

📄 Complexity Analysis

$O(2^n * n)$ time.

📄 Interview Tips

Draw the recursion tree. Explain pruning conditions. State time complexity as $O(\text{branching_factor}^{\text{depth}})$. Edge cases: empty input, single element, target = 0.

Problem 5: LC 90 — Subsets II

LeetCode #90 · [leetcode.com/problems/](https://leetcode.com/problems/subsets-ii/)

📄 Problem Understanding

Return all unique subsets when array may have duplicates.

📄 Pattern Recognition

This is a BACKTRACKING problem. Key signals: all possible combinations/permutations/subsets. Try → recurse → undo. Draw the decision tree.

📄 Brute Force Approach

Generate all possibilities without pruning. Pruning (backtracking) dramatically reduces the search space.

⚡ Optimized Solution

Sort first. Skip duplicate elements at the same recursion level.

📄 Python Code

```
# LeetCode 90: Subsets II
# Pattern: Backtracking Basics

nums.sort(); res=[]
def bt(start,path):
    for i in range(start,len(nums)):
        if i>start and nums[i]==nums[i-1]: continue
        path.append(nums[i])
        res.append(path[:])
        bt(i+1,path)
        path.pop()
bt(0, [])
return res
```

🔗 Dry Run

nums=[1,2,2] → [[],[1],[1,2],[1,2,2],[2],[2,2]]

🔗 Complexity Analysis

$O(2^n * n)$ time.

🔗 Interview Tips

Draw the recursion tree. Explain pruning conditions. State time complexity as $O(\text{branching_factor}^{\text{depth}})$. Edge cases: empty input, single element, target = 0.

Problem 6: LC 39 — Combination Sum

LeetCode #39 · leetcode.com/problems/

🔗 Problem Understanding

Find all combinations summing to target. Each number can be used multiple times.

🔗 Pattern Recognition

This is a BACKTRACKING problem. Key signals: all possible combinations/permutations/subsets. Try → recurse → undo. Draw the decision tree.

🔗 Brute Force Approach

Generate all possibilities without pruning. Pruning (backtracking) dramatically reduces the search space.

⚡ Optimized Solution

For each number, include it (and recurse from same index) or skip it (move to next).

🔗 Python Code

```
# LeetCode 39: Combination Sum
# Pattern: Backtracking Basics

res=[]
def bt(start,path,rem):
    if rem==0: res.append(path[:]); return
    for i in range(start,len(candidates)):
        # Pruning: if current number is greater than remaining sum, skip it
        if candidates[i] > rem: continue
        path.append(candidates[i])
        bt(i,path,rem-candidates[i])
        path.pop()
```

```
        if candidates[i]>rem: break
        path.append(candidates[i])
        bt(i,path,rem-candidates[i])
        path.pop()
    candidates.sort()
    bt(0, [],target)
    return res
```

🔗 Dry Run

candidates=[2,3,6,7], target=7 → [[2,2,3],[7]]

🔗 Complexity Analysis

$O(n^{(T/\min)})$ time where T =target.

🔗 Interview Tips

Draw the recursion tree. Explain pruning conditions. State time complexity as $O(\text{branching_factor}^{\text{depth}})$. Edge cases: empty input, single element, target = 0.

Pattern 18 Summary: Backtracking Basics

Core intuition: decision tree — try, recurse, undo. Key template: base case → loop over choices → choose → recurse → unchoose. Recognition tricks: 'all combinations/permutations/subsets'.

Pattern 19: Backtracking with Constraints

Problems: LC 40, LC 17, LC 79, LC 131, LC 51, LC 52

Problem 1: LC 40 — Combination Sum II

LeetCode #40 · leetcode.com/problems/

🔍 Problem Understanding

Find unique combinations summing to target. Each number can only be used once.

🔍 Pattern Recognition

This is CONSTRAINED BACKTRACKING. Key signals: pruning needed, constraints eliminate branches. Sort inputs first to enable early pruning.

🔍 Brute Force Approach

No pruning: explore all paths even when constraints are already violated. Add pruning to make it efficient.

⚡ Optimized Solution

Sort. Skip duplicates at same level. Each element used at most once.

🔍 Python Code

```
# LeetCode 40: Combination Sum II
# Pattern: Backtracking with Constraints

candidates.sort(); res=[]
def bt(start, path, rem):
    if rem==0: res.append(path[:]); return
    for i in range(start, len(candidates)):
        if candidates[i]>rem: break
        if i>start and candidates[i]==candidates[i-1]: continue
        path.append(candidates[i])
        bt(i+1, path, rem-candidates[i])
        path.pop()
    bt(0, [], target)
return res
```

🔍 Dry Run

candidates=[10,1,2,7,6,1,5], target=8 → [[1,1,6],[1,2,5],[1,7],[2,6]]

🔍 Complexity Analysis

$O(2^n)$ time.

🔍 Interview Tips

Sort input first to enable pruning. Skip duplicates at same level (not within a branch). Edge cases: no valid solution, all elements the same.

Problem 2: LC 17 — Letter Combinations of a Phone Number

LeetCode #17 · [leetcode.com/problems/](https://leetcode.com/problems/letter-combinations-of-a-phone-number/)

🔍 Problem Understanding

Given digit string, return all possible letter combinations (phone keypad).

🔍 Pattern Recognition

This is CONSTRAINED BACKTRACKING. Key signals: pruning needed, constraints eliminate branches. Sort inputs first to enable early pruning.

🔍 Brute Force Approach

No pruning: explore all paths even when constraints are already violated. Add pruning to make it efficient.

⚡ Optimized Solution

Map digits to letters. Backtrack through each digit choosing one letter at a time.

🔍 Python Code

```
# LeetCode 17: Letter Combinations of a Phone Number
# Pattern: Backtracking with Constraints

if not digits: return []
phone={'2':'abc','3':'def','4':'ghi','5':'jkl','6':'mno','7':'pqrs','8':'tuv','9':'wxyz'}
res=[]
def bt(i,path):
    if i==len(digits): res.append(''.join(path)); return
    for c in phone[digits[i]]:
        path.append(c); bt(i+1,path); path.pop()
bt(0,[])
return res
```

🔍 Dry Run

digits='23' → ['ad','ae','af','bd','be','bf','cd','ce','cf']

🔍 Complexity Analysis

$O(4^n * n)$ time.

🔍 Interview Tips

Sort input first to enable pruning. Skip duplicates at same level (not within a branch). Edge cases: no valid solution, all elements the same.

Problem 3: LC 79 — Word Search

LeetCode #79 · [leetcode.com/problems/](https://leetcode.com/problems/word-search/)

🔍 Problem Understanding

Find if word exists in grid following adjacent cells (no reuse).

🔍 Pattern Recognition

This is CONSTRAINED BACKTRACKING. Key signals: pruning needed, constraints eliminate branches. Sort inputs first to enable early pruning.

🔍 Brute Force Approach

No pruning: explore all paths even when constraints are already violated. Add pruning to make it efficient.

⚡ Optimized Solution

DFS from each cell. Mark visited. Backtrack if current path fails.

🔍 Python Code

```
# LeetCode 79: Word Search
# Pattern: Backtracking with Constraints

def dfs(i,j,k):
    if k==len(word): return True
    if not(0<=i<len(board) and 0<=j<len(board[0])): return False
    if board[i][j]!=word[k]: return False
    tmp=board[i][j]; board[i][j]='#'
    found=any(dfs(i+di,j+dj,k+1) for di,dj in [(0,1),(0,-1),(1,0),(-1,0)])
    board[i][j]=tmp
    return found
return any(dfs(i,j,0) for i in range(len(board)) for j in range(len(board[0])))
```

🔍 Dry Run

board=[['A','B','C','E'],['S','F','C','S'],['A','D','E','E']], word='ABCCED' → True

🔍 Complexity Analysis

$O(m*n*4^L)$ time where L =word length.

🔍 Interview Tips

Sort input first to enable pruning. Skip duplicates at same level (not within a branch). Edge cases: no valid solution, all elements the same.

Problem 4: LC 131 — Palindrome Partitioning

LeetCode #131 · leetcode.com/problems/

🔍 Problem Understanding

Partition string such that every substring is a palindrome.

🔍 Pattern Recognition

This is CONSTRAINED BACKTRACKING. Key signals: pruning needed, constraints eliminate branches. Sort inputs first to enable early pruning.

🔍 Brute Force Approach

No pruning: explore all paths even when constraints are already violated. Add pruning to make it efficient.

⚡ Optimized Solution

Try every prefix that is a palindrome; recurse on remainder.

📄 Python Code

```
# LeetCode 131: Palindrome Partitioning
# Pattern: Backtracking with Constraints

res=[]
def bt(start,path):
    if start==len(s): res.append(path[:]); return
    for end in range(start+1,len(s)+1):
        sub=s[start:end]
        if sub==sub[::-1]:
            path.append(sub); bt(end,path); path.pop()
bt(0,[])
return res
```

📄 Dry Run

$s='aab' \rightarrow [['a','a','b'], ['aa','b']]$

📄 Complexity Analysis

$O(n * 2^n)$ time.

📄 Interview Tips

Sort input first to enable pruning. Skip duplicates at same level (not within a branch). Edge cases: no valid solution, all elements the same.

Problem 5: LC 51 — N-Queens

LeetCode #51 · [leetcode.com/problems/](https://leetcode.com/problems/n-queens/)

📄 Problem Understanding

Place n queens on $n \times n$ chessboard such that no two attack each other.

📄 Pattern Recognition

This is CONSTRAINED BACKTRACKING. Key signals: pruning needed, constraints eliminate branches. Sort inputs first to enable early pruning.

📄 Brute Force Approach

No pruning: explore all paths even when constraints are already violated. Add pruning to make it efficient.

⚡ Optimized Solution

Place one queen per row. Track occupied columns and diagonals. Backtrack if conflict.

📄 Python Code

```
# LeetCode 51: N-Queens
# Pattern: Backtracking with Constraints

res=[]; n_val=n
cols=set(); diag1=set(); diag2=set()
board=[['.']*n for _ in range(n)]
def bt(row):
    if row==n_val:
        res.append([''.join(r) for r in board]); return
    for col in range(n_val):
        if col in cols or (row-col) in diag1 or (row+col) in diag2: continue
        cols.add(col); diag1.add(row-col); diag2.add(row+col)
        board[row][col]='Q'
        bt(row+1)
        cols.remove(col); diag1.remove(row-col); diag2.remove(row+col)
        board[row][col]='.'
    bt(0)
return res
```

🔗 Dry Run

$n=4 \rightarrow [["Q..", "...Q", "Q...", "..Q."], ...]$

🔗 Complexity Analysis

$O(n!)$ time.

🔗 Interview Tips

Sort input first to enable pruning. Skip duplicates at same level (not within a branch). Edge cases: no valid solution, all elements the same.

Problem 6: LC 52 — N-Queens II

LeetCode #52 · [leetcode.com/problems/](https://leetcode.com/problems/n-queens-ii/)

🔗 Problem Understanding

Return the count of distinct N-Queens solutions.

🔗 Pattern Recognition

This is CONSTRAINED BACKTRACKING. Key signals: pruning needed, constraints eliminate branches. Sort inputs first to enable early pruning.

🔗 Brute Force Approach

No pruning: explore all paths even when constraints are already violated. Add pruning to make it efficient.

⚡ Optimized Solution

Same as N-Queens but count solutions instead of storing boards.

🔗 Python Code

```
# LeetCode 52: N-Queens II
# Pattern: Backtracking with Constraints
```



```

count=[0]
cols=set(); diag1=set(); diag2=set()
def bt(row):
    if row==n: count[0]+=1; return
    for col in range(n):
        if col in cols or (row-col) in diag1 or (row+col) in diag2: continue
        cols.add(col); diag1.add(row-col); diag2.add(row+col)
        bt(row+1)
        cols.remove(col); diag1.remove(row-col); diag2.remove(row+col)
    bt(0)
return count[0]

```

🔗 Dry Run

$n=4 \rightarrow 2$; $n=1 \rightarrow 1$

🔗 Complexity Analysis

$O(n!)$ time.

🔗 Interview Tips

Sort input first to enable pruning. Skip duplicates at same level (not within a branch). Edge cases: no valid solution, all elements the same.

Pattern 19 Summary: Backtracking with Constraints

Core intuition: prune branches early when constraints are violated. Sort first to enable early termination.
Recognition tricks: same as backtracking but with extra validity checks at each step.

Pattern 20: Graph BFS / DFS

Problems: LC 200, LC 695, LC 733, LC 994, LC 1091, LC 1254

Problem 1: LC 200 — Number of Islands

LeetCode #200 · leetcode.com/problems/

🔍 Problem Understanding

Count number of islands (groups of connected '1's) in a grid.

🔍 Pattern Recognition

This is a GRAPH BFS/DFS problem. Key signals: connected components, flood fill, shortest path in unweighted graph. BFS for shortest path, DFS for connectivity.

🔍 Brute Force Approach

For connectivity: check all pairs of nodes for paths. $O(V^2 * (V+E))$. BFS/DFS with visited set is already optimal.

⚡ Optimized Solution

DFS/BFS from each unvisited '1'. Mark all connected land as visited. Count starts.

🔍 Python Code

```
# LeetCode 200: Number of Islands
# Pattern: Graph BFS / DFS

def dfs(i,j):
    if not(0<=i<len(grid) and 0<=j<len(grid[0])) or grid[i][j]!='1': return
    grid[i][j]='0'
    for di,dj in [(0,1),(0,-1),(1,0),(-1,0)]: dfs(i+di,j+dj)
count=0
for i in range(len(grid)):
    for j in range(len(grid[0])):
        if grid[i][j]=='1': dfs(i,j); count+=1
return count
```

🔍 Dry Run

grid=[['1','1','0'], ['0','1','0'], ['0','0','1']] → 2

🔍 Complexity Analysis

$O(m*n)$ time, $O(m*n)$ space.

🔍 Interview Tips

State whether graph is directed/undirected. Track visited set. BFS for shortest path, DFS for connected components. Edge cases: disconnected graph, no nodes.

Problem 2: LC 695 — Max Area of Island

LeetCode #695 · leetcode.com/problems/

🔍 Problem Understanding

Find the maximum area among all islands.

🔍 Pattern Recognition

This is a GRAPH BFS/DFS problem. Key signals: connected components, flood fill, shortest path in unweighted graph. BFS for shortest path, DFS for connectivity.

🔍 Brute Force Approach

For connectivity: check all pairs of nodes for paths. $O(V^2 * (V+E))$. BFS/DFS with visited set is already optimal.

🚀 Optimized Solution

DFS from each '1'. Return size of island. Track maximum.

🔍 Python Code

```
# LeetCode 695: Max Area of Island
# Pattern: Graph BFS / DFS

def dfs(i,j):
    if not(0<=i<len(grid) and 0<=j<len(grid[0])) or grid[i][j]==0: return 0
    grid[i][j]=0
    return 1+sum(dfs(i+di,j+dj) for di,dj in [(0,1),(0,-1),(1,0),(-1,0)])
return max((dfs(i,j) for i in range(len(grid)) for j in
range(len(grid[0]))),default=0)
```

🔍 Dry Run

grid=[[0,0,1,0,0],[0,1,1,0,0],[0,1,0,0,0]] → 4

🔍 Complexity Analysis

$O(m*n)$ time.

🔍 Interview Tips

State whether graph is directed/undirected. Track visited set. BFS for shortest path, DFS for connected components. Edge cases: disconnected graph, no nodes.

Problem 3: LC 733 — Flood Fill

LeetCode #733 · leetcode.com/problems/

🔍 Problem Understanding

Starting from (sr,sc), replace connected same-colored pixels with color.

🔍 Pattern Recognition

This is a GRAPH BFS/DFS problem. Key signals: connected components, flood fill, shortest path in unweighted graph. BFS for shortest path, DFS for connectivity.

🔗 Brute Force Approach

For connectivity: check all pairs of nodes for paths. $O(V^2 * (V+E))$. BFS/DFS with visited set is already optimal.

⚡ Optimized Solution

DFS/BFS from start. Change color of matching pixels.

🔗 Python Code

```
# LeetCode 733: Flood Fill
# Pattern: Graph BFS / DFS

orig=image[sr][sc]
if orig==color: return image
def dfs(i,j):
    if not(0<=i<len(image) and 0<=j<len(image[0])) or image[i][j]!=orig: return
    image[i][j]=color
    for di,dj in [(0,1),(0,-1),(1,0),(-1,0)]: dfs(i+di,j+dj)
dfs(sr,sc)
return image
```

🔗 Dry Run

image=[[1,1,1],[1,1,0],[1,0,1]], sr=1,sc=1, color=2 → [[2,2,2],[2,2,0],[2,0,1]]

🔗 Complexity Analysis

$O(m*n)$ time.

🔗 Interview Tips

State whether graph is directed/undirected. Track visited set. BFS for shortest path, DFS for connected components. Edge cases: disconnected graph, no nodes.

Problem 4: LC 994 — Rotting Oranges

LeetCode #994 · [leetcode.com/problems/](https://leetcode.com/problems/rotting-oranges/)

🔗 Problem Understanding

Find minutes until all fresh oranges are rotten. -1 if impossible.

🔗 Pattern Recognition

This is a GRAPH BFS/DFS problem. Key signals: connected components, flood fill, shortest path in unweighted graph. BFS for shortest path, DFS for connectivity.

🔗 Brute Force Approach

For connectivity: check all pairs of nodes for paths. $O(V^2 * (V+E))$. BFS/DFS with visited set is already optimal.

⚡ Optimized Solution

Multi-source BFS from all rotten oranges simultaneously. Count steps.

🔗 Python Code

```
# LeetCode 994: Rotting Oranges
# Pattern: Graph BFS / DFS

from collections import deque
q=deque(); fresh=0
for i in range(len(grid)):
    for j in range(len(grid[0])):
        if grid[i][j]==2: q.append((i,j,0))
        elif grid[i][j]==1: fresh+=1
time=0
while q:
    i,j,t=q.popleft()
    for di,dj in [(0,1),(0,-1),(1,0),(-1,0)]:
        ni,nj=i+di,j+dj
        if 0<=ni<len(grid) and 0<=nj<len(grid[0]) and grid[ni][nj]==1:
            grid[ni][nj]=2; fresh-=1; time=t+1; q.append((ni,nj,t+1))
return time if fresh==0 else -1
```

🔍 Dry Run

grid=[[2,1,1],[1,1,0],[0,1,1]] → 4

🔍 Complexity Analysis

$O(m*n)$ time.

🔍 Interview Tips

State whether graph is directed/undirected. Track visited set. BFS for shortest path, DFS for connected components. Edge cases: disconnected graph, no nodes.

Problem 5: LC 1091 — Shortest Path in Binary Matrix

LeetCode #1091 · leetcode.com/problems/

🔍 Problem Understanding

Find shortest clear path from top-left to bottom-right in binary matrix.

🔍 Pattern Recognition

This is a GRAPH BFS/DFS problem. Key signals: connected components, flood fill, shortest path in unweighted graph. BFS for shortest path, DFS for connectivity.

🔍 Brute Force Approach

For connectivity: check all pairs of nodes for paths. $O(V^2 * (V+E))$. BFS/DFS with visited set is already optimal.

⚡ Optimized Solution

BFS level by level. 8-directional. Count steps.

🔍 Python Code

```
# LeetCode 1091: Shortest Path in Binary Matrix
# Pattern: Graph BFS / DFS
```

```

from collections import deque
if grid[0][0]==1 or grid[-1][-1]==1: return -1
n=len(grid); q=deque([(0,0,1)]); grid[0][0]=1
while q:
    i,j,d=q.popleft()
    if i==n-1 and j==n-1: return d
    for di in [-1,0,1]:
        for dj in [-1,0,1]:
            ni,nj=i+di,j+dj
            if 0<=ni<n and 0<=nj<n and grid[ni][nj]==0:
                grid[ni][nj]=1; q.append((ni,nj,d+1))
return -1

```

🔍 Dry Run

grid=[[0,1],[1,0]] → 2

🔍 Complexity Analysis

$O(n^2)$ time.

🔍 Interview Tips

State whether graph is directed/undirected. Track visited set. BFS for shortest path, DFS for connected components. Edge cases: disconnected graph, no nodes.

Problem 6: LC 1254 — Number of Closed Islands

LeetCode #1254 · leetcode.com/problems/

🔍 Problem Understanding

Count islands completely surrounded by water (not touching border).

🔍 Pattern Recognition

This is a GRAPH BFS/DFS problem. Key signals: connected components, flood fill, shortest path in unweighted graph. BFS for shortest path, DFS for connectivity.

🔍 Brute Force Approach

For connectivity: check all pairs of nodes for paths. $O(V^2 * (V+E))$. BFS/DFS with visited set is already optimal.

⚡ Optimized Solution

DFS from all border land to mark open islands. Then count remaining closed islands.

🔍 Python Code

```

# LeetCode 1254: Number of Closed Islands
# Pattern: Graph BFS / DFS

def dfs(i,j):
    if not(0<=i<len(grid) and 0<=j<len(grid[0])) or grid[i][j]!=0: return
    grid[i][j]=1
    for di,dj in [(0,1),(0,-1),(1,0),(-1,0)]: dfs(i+di,j+dj)
# mark border-connected land

```

```

for i in range(len(grid)):
    for j in range(len(grid[0])):
        if (i==0 or i==len(grid)-1 or j==0 or j==len(grid[0])-1): dfs(i,j)
count=0
for i in range(len(grid)):
    for j in range(len(grid[0])):
        if grid[i][j]==0: dfs(i,j); count+=1
return count

```

🔗 Dry Run

grid=[[1,1,1,1,1,1,0],[1,0,0,0,0,1,0],[1,0,1,0,1,1,0],[1,0,0,0,0,1,0,1],[1,1,1,1,1,1,0]] → 2

🔗 Complexity Analysis

$O(m*n)$ time.

🔗 Interview Tips

State whether graph is directed/undirected. Track visited set. BFS for shortest path, DFS for connected components. Edge cases: disconnected graph, no nodes.

Pattern 20 Summary: Graph BFS / DFS

Core intuition: BFS for shortest path (unweighted), DFS for connectivity/backtracking. Key: visited set prevents infinite loops. Recognition tricks: connected components, flood fill, multi-source spread.

Pattern 21: Topological Sort / DAG

Problems: LC 207, LC 210, LC 802, LC 1462, LC 1203, LC 2115

Problem 1: LC 207 — Course Schedule

LeetCode #207 · [leetcode.com/problems/](https://leetcode.com/problems/course-schedule/)

🔍 Problem Understanding

Can you finish all courses given prerequisites? Detect cycle in directed graph.

🔍 Pattern Recognition

This is a TOPOLOGICAL SORT problem. Key signals: dependency ordering, DAG, course prerequisites. Detect cycles; if cycle exists, no valid ordering.

🔍 Brute Force Approach

DFS to find all orderings and verify. Kahn's algorithm (BFS) is the clean $O(V+E)$ solution.

⚡ Optimized Solution

Build adjacency list. DFS cycle detection: 0=unvisited, 1=visiting, 2=done.

🔍 Python Code

```
# LeetCode 207: Course Schedule
# Pattern: Topological Sort / DAG

adj=[[] for _ in range(numCourses)]
for a,b in prerequisites: adj[b].append(a)
state=[0]*numCourses
def dfs(node):
    if state[node]==1: return False # cycle
    if state[node]==2: return True
    state[node]=1
    for nb in adj[node]:
        if not dfs(nb): return False
    state[node]=2; return True
return all(dfs(i) for i in range(numCourses))
```

🔍 Dry Run

numCourses=2, prerequisites=[[1,0]] → True

🔍 Complexity Analysis

$O(V+E)$ time, $O(V+E)$ space.

🔍 Interview Tips

Two approaches: Kahn's (BFS, in-degree) or DFS. Both $O(V+E)$. If cycle exists, output size < n. Edge cases: no edges, all independent nodes.

Problem 2: LC 210 — Course Schedule II

LeetCode #210 · [leetcode.com/problems/](https://leetcode.com/problems/course-schedule-ii/)

🔍 Problem Understanding

Return order to finish all courses, or [] if impossible.

🔍 Pattern Recognition

This is a TOPOLOGICAL SORT problem. Key signals: dependency ordering, DAG, course prerequisites. Detect cycles; if cycle exists, no valid ordering.

🔍 Brute Force Approach

DFS to find all orderings and verify. Kahn's algorithm (BFS) is the clean $O(V+E)$ solution.

⚡ Optimized Solution

Kahn's algorithm (BFS topo sort): in-degree array, start with 0 in-degree nodes.

🔍 Python Code

```
# LeetCode 210: Course Schedule II
# Pattern: Topological Sort / DAG

from collections import deque
adj=[[] for _ in range(numCourses)]
in_deg=[0]*numCourses
for a,b in prerequisites: adj[b].append(a); in_deg[a]+=1
q=deque(i for i in range(numCourses) if in_deg[i]==0)
order=[]
while q:
    node=q.popleft(); order.append(node)
    for nb in adj[node]:
        in_deg[nb]-=1
        if in_deg[nb]==0: q.append(nb)
return order if len(order)==numCourses else []
```

🔍 Dry Run

numCourses=4, prerequisites=[[1,0],[2,0],[3,1],[3,2]] → [0,1,2,3] or [0,2,1,3]

🔍 Complexity Analysis

$O(V+E)$ time.

🔍 Interview Tips

Two approaches: Kahn's (BFS, in-degree) or DFS. Both $O(V+E)$. If cycle exists, output size < n. Edge cases: no edges, all independent nodes.

Problem 3: LC 802 — Find Eventual Safe States

LeetCode #802 · [leetcode.com/problems/](https://leetcode.com/problems/find-eventual-safe-states/)

🔍 Problem Understanding

Find all nodes that eventually lead only to terminal nodes (no cycles).

🔍 Pattern Recognition

This is a TOPOLOGICAL SORT problem. Key signals: dependency ordering, DAG, course prerequisites. Detect cycles; if cycle exists, no valid ordering.

🔍 Brute Force Approach

DFS to find all orderings and verify. Kahn's algorithm (BFS) is the clean $O(V+E)$ solution.

⚡ Optimized Solution

Reverse graph. Nodes with no outgoing edges in reverse = terminal in original. BFS from terminals.

🔍 Python Code

```
# LeetCode 802: Find Eventual Safe States
# Pattern: Topological Sort / DAG

from collections import deque
n=len(graph)
rev=[[] for _ in range(n)]
out_deg=[0]*n
for u,vs in enumerate(graph):
    out_deg[u]=len(vs)
    for v in vs: rev[v].append(u)
q=deque(i for i in range(n) if out_deg[i]==0)
safe=set(q)
while q:
    node=q.popleft()
    for nb in rev[node]:
        out_deg[nb]-=1
        if out_deg[nb]==0: safe.add(nb); q.append(nb)
return sorted(safe)
```

🔍 Dry Run

graph=[[1,2],[2,3],[5],[0],[5],[],[]] → [2,4,5,6]

🔍 Complexity Analysis

$O(V+E)$ time.

🔍 Interview Tips

Two approaches: Kahn's (BFS, in-degree) or DFS. Both $O(V+E)$. If cycle exists, output size < n. Edge cases: no edges, all independent nodes.

Problem 4: LC 1462 — Course Schedule IV

LeetCode #1462 · leetcode.com/problems/

🔍 Problem Understanding

Answer queries: is course u a prerequisite of course v (direct or indirect)?

🔍 Pattern Recognition

This is a TOPOLOGICAL SORT problem. Key signals: dependency ordering, DAG, course prerequisites. Detect cycles; if cycle exists, no valid ordering.

🔍 Brute Force Approach

DFS to find all orderings and verify. Kahn's algorithm (BFS) is the clean $O(V+E)$ solution.

⚡ Optimized Solution

Topo sort. Propagate reachability. `reachable[u]` = all courses reachable from u.

🔍 Python Code

```
# LeetCode 1462: Course Schedule IV
# Pattern: Topological Sort / DAG

from collections import deque
adj=[[] for _ in range(numCourses)]
in_deg=[0]*numCourses
for a,b in prerequisites: adj[a].append(b); in_deg[b]+=1
reach=[set() for _ in range(numCourses)]
for a,b in prerequisites: reach[a].add(b)
q=deque(i for i in range(numCourses) if in_deg[i]==0)
while q:
    u=q.popleft()
    for v in adj[u]:
        reach[v]|=reach[u]; reach[v].add(u)
        in_deg[v]-=1
        if in_deg[v]==0: q.append(v)
return [u in reach[v] for u,v in queries]
```

🔍 Dry Run

`numCourses=3, prerequisites=[[1,2],[1,0],[2,0]], queries=[[1,0],[1,2]] → [True,True]`

🔍 Complexity Analysis

$O(V^2 + E)$ time.

🔍 Interview Tips

Two approaches: Kahn's (BFS, in-degree) or DFS. Both $O(V+E)$. If cycle exists, output size $< n$. Edge cases: no edges, all independent nodes.

Problem 5: LC 1203 — Sort Items by Groups Respecting Dependencies

LeetCode #1203 · leetcode.com/problems/

🔍 Problem Understanding

Sort items; each item belongs to a group. Respect both item and group dependencies.

🔍 Pattern Recognition

This is a TOPOLOGICAL SORT problem. Key signals: dependency ordering, DAG, course prerequisites. Detect cycles; if cycle exists, no valid ordering.

🔍 Brute Force Approach

DFS to find all orderings and verify. Kahn's algorithm (BFS) is the clean $O(V+E)$ solution.

⚡ Optimized Solution

Two-level topo sort: sort items within groups; sort groups by group dependencies.

🔗 Python Code

```
# LeetCode 1203: Sort Items by Groups Respecting Dependencies
# Pattern: Topological Sort / DAG

# Assign unique group IDs to ungrouped items
# Build item-graph and group-graph
# Topo sort groups, then items within each group
```

🔗 Dry Run

$n=8, m=2$, group=[-1,-1,1,0,0,1,0,-1], beforeItems=[[],[6],[5],[6],[3,6],[],[],[]] → [6,3,4,1,5,2,0,7]

🔗 Complexity Analysis

$O(V+E)$ time.

🔗 Interview Tips

Two approaches: Kahn's (BFS, in-degree) or DFS. Both $O(V+E)$. If cycle exists, output size $< n$. Edge cases: no edges, all independent nodes.

Problem 6: LC 2115 — Find All Possible Recipes from Given Supplies

LeetCode #2115 · leetcode.com/problems/

🔗 Problem Understanding

Given recipes, ingredients, and initial supplies, find all makeable recipes.

🔗 Pattern Recognition

This is a TOPOLOGICAL SORT problem. Key signals: dependency ordering, DAG, course prerequisites. Detect cycles; if cycle exists, no valid ordering.

🔗 Brute Force Approach

DFS to find all orderings and verify. Kahn's algorithm (BFS) is the clean $O(V+E)$ solution.

⚡ Optimized Solution

Treat recipes and supplies as graph nodes. Topo sort: recipe is ready when all ingredients available.

🔗 Python Code

```
# LeetCode 2115: Find All Possible Recipes from Given Supplies
# Pattern: Topological Sort / DAG

from collections import defaultdict, deque
adj=defaultdict(list)
in_deg={}
for i,r in enumerate(recipes):
    in_deg[r]=len(ingredients[i])
    for ing in ingredients[i]: adj[ing].append(r)
q=deque(supplies)
res=[]
while q:
```

```
item=q.popleft()
if item in in_deg:
    if item in set(recipes): res.append(item)
for nb in adj[item]:
    in_deg[nb]-=1
    if in_deg[nb]==0: q.append(nb)
return res
```

🔗 Dry Run

recipes=['bread'], ingredients=[['yeast','flour']], supplies=['yeast','flour','corn'] → ['bread']

🔗 Complexity Analysis

$O(V+E)$ time.

🔗 Interview Tips

Two approaches: Kahn's (BFS, in-degree) or DFS. Both $O(V+E)$. If cycle exists, output size $< n$. Edge cases: no edges, all independent nodes.

Pattern 21 Summary: Topological Sort / DAG

Core intuition: order nodes such that all edges go 'forward'. Cycle detection is built-in: if topo order length $< n$, there's a cycle. Two approaches: DFS or Kahn's (in-degree + BFS).

Pattern 22: Union Find / DSU

Problems: LC 547, LC 684, LC 1319, LC 990, LC 1579, LC 1202

Problem 1: LC 547 — Number of Provinces

LeetCode #547 · leetcode.com/problems/

🔍 Problem Understanding

Find number of connected components (provinces) in undirected graph given adjacency matrix.

🔍 Pattern Recognition

This is a UNION FIND (DSU) problem. Key signals: dynamic connectivity, merge components, cycle detection. Use path compression + union by rank for $O(\alpha(n))$.

🔍 Brute Force Approach

BFS/DFS for each connectivity query. $O(n)$ per query. DSU gives $O(\alpha(n))$ amortized.

🚀 Optimized Solution

Union-Find: for each edge, union the two nodes. Count distinct roots.

🔍 Python Code

```
# LeetCode 547: Number of Provinces
# Pattern: Union Find / DSU

parent=list(range(n))
def find(x):
    while parent[x]!=x: parent[x]=parent[parent[x]]; x=parent[x]
    return x
def union(a,b):
    parent[find(a)]=find(b)
for i in range(n):
    for j in range(i+1,n):
        if isConnected[i][j]: union(i,j)
return len(set(find(i) for i in range(n)))
```

🔍 Dry Run

isConnected=[[1,1,0],[1,1,0],[0,0,1]] → 2

🔍 Complexity Analysis

$O(n^2 * \alpha(n))$ time.

🔍 Interview Tips

Explain path compression and union by rank. Without them, worst case $O(n)$ per operation. With both, $O(\alpha(n)) \approx O(1)$. Edge cases: self-loops, no edges.

Problem 2: LC 684 — Redundant Connection

LeetCode #684 · leetcode.com/problems/

🔍 Problem Understanding

Find the edge that creates a cycle in an undirected tree.

🔍 Pattern Recognition

This is a UNION FIND (DSU) problem. Key signals: dynamic connectivity, merge components, cycle detection. Use path compression + union by rank for $O(\alpha(n))$.

🔍 Brute Force Approach

BFS/DFS for each connectivity query. $O(n)$ per query. DSU gives $O(\alpha(n))$ amortized.

⚡ Optimized Solution

Process edges one by one. If both nodes already in same component, this edge creates cycle.

🔍 Python Code

```
# LeetCode 684: Redundant Connection
# Pattern: Union Find / DSU

parent=list(range(n+1))
def find(x):
    while parent[x]!=x: parent[x]=parent[parent[x]]; x=parent[x]
    return x
for a,b in edges:
    ra,rb=find(a),find(b)
    if ra==rb: return [a,b]
    parent[ra]=rb
return []
```

🔍 Dry Run

edges=[[1,2],[1,3],[2,3]] → [2,3]

🔍 Complexity Analysis

$O(n * \alpha(n))$ time.

🔍 Interview Tips

Explain path compression and union by rank. Without them, worst case $O(n)$ per operation. With both, $O(\alpha(n)) \approx O(1)$. Edge cases: self-loops, no edges.

Problem 3: LC 1319 — Number of Operations to Make Network Connected

LeetCode #1319 · leetcode.com/problems/

🔍 Problem Understanding

Minimum cables needed to connect all n computers. Return -1 if impossible.

🔍 Pattern Recognition

This is a UNION FIND (DSU) problem. Key signals: dynamic connectivity, merge components, cycle detection. Use path compression + union by rank for $O(\alpha(n))$.

🔍 Brute Force Approach

BFS/DFS for each connectivity query. $O(n)$ per query. DSU gives $O(\alpha(n))$ amortized.

⚡ Optimized Solution

Count components and extra edges. Need (components-1) extra edges.

📄 Python Code

```
# LeetCode 1319: Number of Operations to Make Network Connected
# Pattern: Union Find / DSU

parent=list(range(n))
def find(x):
    while parent[x]!=x: parent[x]=parent[parent[x]]; x=parent[x]
    return x
extra=0
for a,b in connections:
    ra,rb=find(a),find(b)
    if ra==rb: extra+=1
    else: parent[ra]=rb
comps=len(set(find(i) for i in range(n)))
needed=comps-1
return needed if extra>=needed else -1
```

🔍 Dry Run

$n=4$, connections=[[0,1],[0,2],[1,2]] → 1

🔍 Complexity Analysis

$O(n * \alpha(n))$ time.

🔍 Interview Tips

Explain path compression and union by rank. Without them, worst case $O(n)$ per operation. With both, $O(\alpha(n)) \approx O(1)$. Edge cases: self-loops, no edges.

Problem 4: LC 990 — Satisfiability of Equality Equations

LeetCode #990 · leetcode.com/problems/

🔍 Problem Understanding

Given equations like 'a==b' and 'a!=b', check if they are consistent.

🔍 Pattern Recognition

This is a UNION FIND (DSU) problem. Key signals: dynamic connectivity, merge components, cycle detection. Use path compression + union by rank for $O(\alpha(n))$.

🔍 Brute Force Approach

BFS/DFS for each connectivity query. $O(n)$ per query. DSU gives $O(\alpha(n))$ amortized.

⚡ Optimized Solution

Process == first (union). Then check != for conflicts.

📄 Python Code

```
# LeetCode 990: Satisfiability of Equality Equations
# Pattern: Union Find / DSU

parent=list(range(26))
def find(x):
    while parent[x]!=x: parent[x]=parent[parent[x]]; x=parent[x]
    return x
for eq in equations:
    if eq[1]=='=':
        a,b=ord(eq[0])-97,ord(eq[3])-97
        parent[find(a)]=find(b)
for eq in equations:
    if eq[1]=='!':
        a,b=ord(eq[0])-97,ord(eq[3])-97
        if find(a)==find(b): return False
return True
```

📄 Dry Run

equations=['a==b','b!=a'] → False

📄 Complexity Analysis

$O(n * \alpha(26))$ time.

📄 Interview Tips

Explain path compression and union by rank. Without them, worst case $O(n)$ per operation. With both, $O(\alpha(n)) \approx O(1)$. Edge cases: self-loops, no edges.

Problem 5: LC 1579 — Remove Max Number of Edges to Keep Graph Fully Traversable

LeetCode #1579 · leetcode.com/problems/

📄 Problem Understanding

Alice uses type 1 and 3 edges; Bob uses type 2 and 3. Maximize removable edges.

📄 Pattern Recognition

This is a UNION FIND (DSU) problem. Key signals: dynamic connectivity, merge components, cycle detection. Use path compression + union by rank for $O(\alpha(n))$.

📄 Brute Force Approach

BFS/DFS for each connectivity query. $O(n)$ per query. DSU gives $O(\alpha(n))$ amortized.

⚡ Optimized Solution

Use two DSUs (Alice and Bob). Add type 3 edges first (shared). Then type 1 and 2. Count unnecessary edges.

📄 Python Code

```
# LeetCode 1579: Remove Max Number of Edges to Keep Graph Fully Traversable
# Pattern: Union Find / DSU

# Two DSU instances
# Add type 3 (shared) edges first, count redundant
# Then type 1 for Alice, type 2 for Bob
# Check both are fully connected
```

🔗 Dry Run

$n=4$, edges=[[3,1,2],[3,2,3],[1,1,3],[1,2,4],[1,1,2],[2,3,4]] → 2

🔗 Complexity Analysis

$O(E * \alpha(n))$ time.

🔗 Interview Tips

Explain path compression and union by rank. Without them, worst case $O(n)$ per operation. With both, $O(\alpha(n)) \approx O(1)$. Edge cases: self-loops, no edges.

Problem 6: LC 1202 — Smallest String With Swaps

LeetCode #1202 · leetcode.com/problems/

🔗 Problem Understanding

Can swap chars at given index pairs any number of times. Return lexicographically smallest string.

🔗 Pattern Recognition

This is a UNION FIND (DSU) problem. Key signals: dynamic connectivity, merge components, cycle detection. Use path compression + union by rank for $O(\alpha(n))$.

🔗 Brute Force Approach

BFS/DFS for each connectivity query. $O(n)$ per query. DSU gives $O(\alpha(n))$ amortized.

⚡ Optimized Solution

Union connected swap indices. Sort chars within each connected component. Place back.

🔗 Python Code

```
# LeetCode 1202: Smallest String With Swaps
# Pattern: Union Find / DSU

from collections import defaultdict
parent=list(range(len(s)))
def find(x):
    while parent[x]!=x: parent[x]=parent[parent[x]]; x=parent[x]
    return x
for a,b in pairs:
    parent[find(a)]=find(b)
groups=defaultdict(list)
for i in range(len(s)): groups[find(i)].append(i)
res=list(s)
for idxs in groups.values():
    chars=sorted(s[i] for i in idxs)
```

```
    for i,c in zip(sorted(idxs),chars): res[i]=c
    return ''.join(res)
```

🔍 Dry Run

s='dcab', pairs=[[0,3],[1,2]] → 'bacd'

🔍 Complexity Analysis

$O(n \log n + E * \alpha(n))$ time.

🔍 Interview Tips

Explain path compression and union by rank. Without them, worst case $O(n)$ per operation. With both, $O(\alpha(n)) \approx O(1)$. Edge cases: self-loops, no edges.

Pattern 22 Summary: Union Find / DSU

Core intuition: efficiently merge and query connected components. Path compression + union by rank gives $O(\alpha(n)) \approx O(1)$. Recognition tricks: 'connected components', 'cycle in undirected graph'.

Pattern 23: Shortest Path

Problems: LC 743, LC 787, LC 1514, LC 1631, LC 1334, LC 1976

Problem 1: LC 743 — Network Delay Time

LeetCode #743 · leetcode.com/problems/

🔖 Problem Understanding

Find time for signal to reach all nodes from k. Return -1 if impossible.

🔖 Pattern Recognition

This is a SHORTEST PATH problem. Dijkstra for non-negative weights, Bellman-Ford for negative weights, Floyd-Warshall for all-pairs.

🔖 Brute Force Approach

BFS for unweighted, Bellman-Ford for weighted. Dijkstra is optimal for non-negative weights.

🚀 Optimized Solution

Dijkstra from k. $\text{dist}[\text{node}]$ = minimum time to reach node.

🔖 Python Code

```
# LeetCode 743: Network Delay Time
# Pattern: Shortest Path

import heapq
from collections import defaultdict
adj=defaultdict(list)
for u,v,w in times: adj[u].append((v,w))
dist={k:0}
heap=[(0,k)]
while heap:
    d,u=heapq.heappop(heap)
    if d>dist.get(u,float('inf')): continue
    for v,w in adj[u]:
        if d+w<dist.get(v,float('inf')):
            dist[v]=d+w; heapq.heappush(heap,(d+w,v))
return max(dist.values()) if len(dist)==n else -1
```

🔖 Dry Run

times=[[2,1,1],[2,3,1],[3,4,1]], n=4, k=2 → 2

🔖 Complexity Analysis

$O((V+E) \log V)$ time.

🔖 Interview Tips

Dijkstra assumes non-negative weights. For negative weights, use Bellman-Ford. For all-pairs, Floyd-Warshall. State which algorithm and why.

Problem 2: LC 787 — Cheapest Flights Within K Stops

LeetCode #787 · [leetcode.com/problems/](https://leetcode.com/problems/cheapest-flights-within-k-stops/)

🔖 Problem Understanding

Find cheapest price from src to dst with at most k stops.

🔖 Pattern Recognition

This is a SHORTEST PATH problem. Dijkstra for non-negative weights, Bellman-Ford for negative weights, Floyd-Warshall for all-pairs.

🔖 Brute Force Approach

BFS for unweighted, Bellman-Ford for weighted. Dijkstra is optimal for non-negative weights.

⚡ Optimized Solution

Bellman-Ford relaxed k+1 times. Or modified Dijkstra with stops count.

🔖 Python Code

```
# LeetCode 787: Cheapest Flights Within K Stops
# Pattern: Shortest Path

import heapq
from collections import defaultdict
adj=defaultdict(list)
for u,v,p in flights: adj[u].append((v,p))
heap=[(0,src,0)] # (cost,node,stops)
dist={(src,0):0}
while heap:
    cost,u,stops=heapq.heappop(heap)
    if u==dst: return cost
    if stops>k: continue
    for v,p in adj[u]:
        nc=cost+p
        if nc<dist.get((v,stops+1),float('inf')):
            dist[(v,stops+1)]=nc
            heapq.heappush(heap,(nc,v,stops+1))
return -1
```

🔖 Dry Run

n=3, flights=[[0,1,100],[1,2,100],[0,2,500]], src=0, dst=2, k=1 → 200

🔖 Complexity Analysis

$O(E * k \log(V*k))$ time.

🔖 Interview Tips

Dijkstra assumes non-negative weights. For negative weights, use Bellman-Ford. For all-pairs, Floyd-Warshall. State which algorithm and why.

Problem 3: LC 1514 — Path with Maximum Probability

LeetCode #1514 · leetcode.com/problems/

🔍 Problem Understanding

Find path from start to end with maximum probability of success.

🔍 Pattern Recognition

This is a SHORTEST PATH problem. Dijkstra for non-negative weights, Bellman-Ford for negative weights, Floyd-Warshall for all-pairs.

🔍 Brute Force Approach

BFS for unweighted, Bellman-Ford for weighted. Dijkstra is optimal for non-negative weights.

⚡ Optimized Solution

Modified Dijkstra maximizing probability (use max-heap with negated probabilities).

🔍 Python Code

```
# LeetCode 1514: Path with Maximum Probability
# Pattern: Shortest Path

from collections import defaultdict
import heapq
adj=defaultdict(list)
for (u,v),p in zip(edges,succProb): adj[u].append((v,p)); adj[v].append((u,p))
prob=[0.0]*n; prob[start]=1.0
heap=[(-1.0,start)]
while heap:
    p,u=heapq.heappop(heap)
    p=-p
    if p<prob[u]: continue
    for v,ep in adj[u]:
        if p*ep>prob[v]: prob[v]=p*ep; heapq.heappush(heap,(-prob[v],v))
return prob[end]
```

🔍 Dry Run

n=3, edges=[[0,1],[1,2],[0,2]], succProb=[0.5,0.5,0.2], start=0, end=2 → 0.25

🔍 Complexity Analysis

$O((V+E) \log V)$ time.

🔍 Interview Tips

Dijkstra assumes non-negative weights. For negative weights, use Bellman-Ford. For all-pairs, Floyd-Warshall. State which algorithm and why.

Problem 4: LC 1631 — Path With Minimum Effort

LeetCode #1631 · leetcode.com/problems/

🔍 Problem Understanding

Find path where maximum absolute difference between adjacent cells is minimized.

🔖 Pattern Recognition

This is a SHORTEST PATH problem. Dijkstra for non-negative weights, Bellman-Ford for negative weights, Floyd-Warshall for all-pairs.

🔖 Brute Force Approach

BFS for unweighted, Bellman-Ford for weighted. Dijkstra is optimal for non-negative weights.

⚡ Optimized Solution

Dijkstra where edge weight = abs difference. Minimize maximum effort.

🔖 Python Code

```
# LeetCode 1631: Path With Minimum Effort
# Pattern: Shortest Path

import heapq
m,n=len(heights),len(heights[0])
dist=[[float('inf')]*n for _ in range(m)]
dist[0][0]=0; heap=[(0,0,0)]
while heap:
    eff,i,j=heapq.heappop(heap)
    if eff>dist[i][j]: continue
    if i==m-1 and j==n-1: return eff
    for di,dj in [(0,1),(0,-1),(1,0),(-1,0)]:
        ni,nj=i+di,j+dj
        if 0<=ni<m and 0<=nj<n:
            new_eff=max(eff,abs(heights[ni][nj]-heights[i][j]))
            if new_eff<dist[ni][nj]:
                dist[ni][nj]=new_eff; heapq.heappush(heap,(new_eff,ni,nj))
return dist[m-1][n-1]
```

🔖 Dry Run

heights=[[1,2,2],[3,8,2],[5,3,5]] → 2

🔖 Complexity Analysis

$O(m*n \log(m*n))$ time.

🔖 Interview Tips

Dijkstra assumes non-negative weights. For negative weights, use Bellman-Ford. For all-pairs, Floyd-Warshall. State which algorithm and why.

Problem 5: LC 1334 — Find the City With the Smallest Number of Neighbors at a Threshold Distance

LeetCode #1334 · leetcode.com/problems/

🔖 Problem Understanding

Find city with fewest reachable cities within distance threshold. Return highest index if tie.

🔍 Pattern Recognition

This is a SHORTEST PATH problem. Dijkstra for non-negative weights, Bellman-Ford for negative weights, Floyd-Warshall for all-pairs.

🔍 Brute Force Approach

BFS for unweighted, Bellman-Ford for weighted. Dijkstra is optimal for non-negative weights.

⚡ Optimized Solution

Floyd-Warshall for all-pairs shortest paths. Then count reachable cities for each.

🔍 Python Code

```
# LeetCode 1334: Find the City With the Smallest Number of Neighbors at a Threshold Distance
# Pattern: Shortest Path

INF=float('inf')
dist=[[INF]*n for _ in range(n)]
for i in range(n): dist[i][i]=0
for u,v,w in edges: dist[u][v]=dist[v][u]=w
for k in range(n):
    for i in range(n):
        for j in range(n):
            dist[i][j]=min(dist[i][j],dist[i][k]+dist[k][j])
best_city=-1; min_count=INF
for i in range(n):
    cnt=sum(1 for j in range(n) if i!=j and dist[i][j]<=distanceThreshold)
    if cnt<=min_count: min_count=cnt; best_city=i
return best_city
```

🔍 Dry Run

n=4, edges=[[0,1,3],[1,2,1],[1,3,4],[2,3,1]], distanceThreshold=4 → 3

🔍 Complexity Analysis

$O(n^3)$ time.

🔍 Interview Tips

Dijkstra assumes non-negative weights. For negative weights, use Bellman-Ford. For all-pairs, Floyd-Warshall. State which algorithm and why.

Problem 6: LC 1976 — Number of Ways to Arrive at Destination

LeetCode #1976 · leetcode.com/problems/

🔍 Problem Understanding

Count paths from 0 to n-1 with minimum time.

🔍 Pattern Recognition

This is a SHORTEST PATH problem. Dijkstra for non-negative weights, Bellman-Ford for negative weights, Floyd-Warshall for all-pairs.

🔗 Brute Force Approach

BFS for unweighted, Bellman-Ford for weighted. Dijkstra is optimal for non-negative weights.

⚡ Optimized Solution

Dijkstra. Track number of ways to reach each node with minimum distance.

🔗 Python Code

```
# LeetCode 1976: Number of Ways to Arrive at Destination
# Pattern: Shortest Path

import heapq
from collections import defaultdict
MOD=10**9+7
adj=defaultdict(list)
for u,v,t in roads: adj[u].append((v,t)); adj[v].append((u,t))
dist=[float('inf')]*n; dist[0]=0
ways=[0]*n; ways[0]=1
heap=[(0,0)]
while heap:
    d,u=heapq.heappop(heap)
    if d>dist[u]: continue
    for v,t in adj[u]:
        if d+t<dist[v]:
            dist[v]=d+t; ways[v]=ways[u]; heapq.heappush(heap, (dist[v],v))
        elif d+t==dist[v]: ways[v]=(ways[v]+ways[u])%MOD
return ways[n-1]
```

🔗 Dry Run

$n=7$, roads=[[0,6,7],[0,1,2],[1,2,3],[1,3,3],[6,3,3],[3,5,1],[6,5,2],[0,4,5],[4,6,2]] → 4

🔗 Complexity Analysis

$O((V+E) \log V)$ time.

🔗 Interview Tips

Dijkstra assumes non-negative weights. For negative weights, use Bellman-Ford. For all-pairs, Floyd-Warshall. State which algorithm and why.

Pattern 23 Summary: Shortest Path

Core intuition: Dijkstra = greedy BFS with priority queue. Key algorithms: Dijkstra (non-negative), Bellman-Ford (negative), Floyd-Warshall (all-pairs). Always ask: can edge weights be negative?

Pattern 24: MST / Graph Greedy

Problems: LC 1584, LC 1135, LC 778, LC 1102, LC 1168, LC 1489

Problem 1: LC 1584 — Min Cost to Connect All Points

LeetCode #1584 · leetcode.com/problems/

🔖 Problem Understanding

Find minimum cost to connect all points (Manhattan distance edges).

🔖 Pattern Recognition

This is a MINIMUM SPANNING TREE problem. Kruskal's for sparse graphs (sort edges + DSU), Prim's for dense graphs (heap-based).

🔖 Brute Force Approach

Try all spanning trees (exponential). Kruskal's and Prim's are greedy $O(E \log E)$ algorithms.

🔖 Optimized Solution

Prim's or Kruskal's MST. For dense graphs, Prim's is faster.

🔖 Python Code

```
# LeetCode 1584: Min Cost to Connect All Points
# Pattern: MST / Graph Greedy

import heapq
n=len(points); visited=[False]*n
heap=[(0,0)]; total=0; count=0
while count<n:
    cost,u=heapq.heappop(heap)
    if visited[u]: continue
    visited[u]=True; total+=cost; count+=1
    for v in range(n):
        if not visited[v]:
            d=abs(points[u][0]-points[v][0])+abs(points[u][1]-points[v][1])
            heapq.heappush(heap, (d,v))
return total
```

🔖 Dry Run

points=[[0,0],[2,2],[3,10],[5,2],[7,0]] → 20

🔖 Complexity Analysis

$O(n^2 \log n)$ time (Prim's).

🔖 Interview Tips

Kruskal's = sort edges + DSU. Prim's = heap + visited. Both $O(E \log E)$. Explain MST property: adding cheapest edge that doesn't create cycle.

Problem 2: LC 1135 — Connecting Cities With Minimum Cost

LeetCode #1135 · leetcode.com/problems/

🔍 Problem Understanding

Find MST cost to connect n cities. Return -1 if impossible.

🔍 Pattern Recognition

This is a MINIMUM SPANNING TREE problem. Kruskal's for sparse graphs (sort edges + DSU), Prim's for dense graphs (heap-based).

🔍 Brute Force Approach

Try all spanning trees (exponential). Kruskal's and Prim's are greedy $O(E \log E)$ algorithms.

⚡ Optimized Solution

Kruskal's: sort edges by cost, union cities greedily.

🔍 Python Code

```
# LeetCode 1135: Connecting Cities With Minimum Cost
# Pattern: MST / Graph Greedy

connections.sort(key=lambda x:x[2])
parent=list(range(n+1))
def find(x):
    while parent[x]!=x: parent[x]=parent[parent[x]]; x=parent[x]
    return x
total=count=0
for u,v,cost in connections:
    ru,rv=find(u),find(v)
    if ru!=rv: parent[ru]=rv; total+=cost; count+=1
    if count==n-1: return total
return -1
```

🔍 Dry Run

$n=3$, connections=[[1,2,5],[1,3,6],[2,3,1]] → 6

🔍 Complexity Analysis

$O(E \log E)$ time.

🔍 Interview Tips

Kruskal's = sort edges + DSU. Prim's = heap + visited. Both $O(E \log E)$. Explain MST property: adding cheapest edge that doesn't create cycle.

Problem 3: LC 778 — Swim in Rising Water

LeetCode #778 · leetcode.com/problems/

🔍 Problem Understanding

Find minimum time to swim from (0,0) to (n-1,n-1). Can move when water level $\geq$ cell elevation.

🔍 Pattern Recognition

This is a MINIMUM SPANNING TREE problem. Kruskal's for sparse graphs (sort edges + DSU), Prim's for dense graphs (heap-based).

🔍 Brute Force Approach

Try all spanning trees (exponential). Kruskal's and Prim's are greedy $O(E \log E)$ algorithms.

⚡ Optimized Solution

Dijkstra/binary search: minimize the maximum elevation along the path.

🔍 Python Code

```
# LeetCode 778: Swim in Rising Water
# Pattern: MST / Graph Greedy

import heapq
n=len(grid); dist=[[float('inf')]*n for _ in range(n)]
dist[0][0]=grid[0][0]; heap=[(grid[0][0],0,0)]
while heap:
    t,i,j=heapq.heappop(heap)
    if i==n-1 and j==n-1: return t
    for di,dj in [(0,1),(0,-1),(1,0),(-1,0)]:
        ni,nj=i+di,j+dj
        if 0<=ni<n and 0<=nj<n:
            nt=max(t,grid[ni][nj])
            if nt<dist[ni][nj]:
                dist[ni][nj]=nt; heapq.heappush(heap,(nt,ni,nj))
return dist[n-1][n-1]
```

🔍 Dry Run

grid=[[0,2],[1,3]] → 3

🔍 Complexity Analysis

$O(n^2 \log n)$ time.

🔍 Interview Tips

Kruskal's = sort edges + DSU. Prim's = heap + visited. Both $O(E \log E)$. Explain MST property: adding cheapest edge that doesn't create cycle.

Problem 4: LC 1102 — Path With Maximum Minimum Value

LeetCode #1102 · leetcode.com/problems/

🔍 Problem Understanding

Find path from (0,0) to (m-1,n-1) maximizing the minimum value along the path.

🔍 Pattern Recognition

This is a MINIMUM SPANNING TREE problem. Kruskal's for sparse graphs (sort edges + DSU), Prim's for dense graphs (heap-based).

🔍 Brute Force Approach

Try all spanning trees (exponential). Kruskal's and Prim's are greedy $O(E \log E)$ algorithms.

⚡ Optimized Solution

Dijkstra but tracking minimum value. Max-heap. At each step choose highest available cell.

🔗 Python Code

```
# LeetCode 1102: Path With Maximum Minimum Value
# Pattern: MST / Graph Greedy

import heapq
m,n=len(grid),len(grid[0])
heap=[(-grid[0][0],0,0)]; vis=[[False]*n for _ in range(m)]
while heap:
    v,i,j=heapq.heappop(heap)
    if vis[i][j]: continue
    vis[i][j]=True
    if i==m-1 and j==n-1: return -v
    for di,dj in [(0,1),(0,-1),(1,0),(-1,0)]:
        ni,nj=i+di,j+dj
        if 0<=ni<m and 0<=nj<n and not vis[ni][nj]:
            heapq.heappush(heap, (-min(-v,grid[ni][nj]),ni,nj))
return -1
```

🔗 Dry Run

grid=[[5,4,5],[1,2,6],[7,4,6]] → 4

🔗 Complexity Analysis

$O(m*n \log(m*n))$ time.

🔗 Interview Tips

Kruskal's = sort edges + DSU. Prim's = heap + visited. Both $O(E \log E)$. Explain MST property: adding cheapest edge that doesn't create cycle.

Problem 5: LC 1168 — Optimize Water Distribution in a Village

LeetCode #1168 · leetcode.com/problems/

🔗 Problem Understanding

Minimize cost to supply water to all houses (either build well or lay pipe).

🔗 Pattern Recognition

This is a MINIMUM SPANNING TREE problem. Kruskal's for sparse graphs (sort edges + DSU), Prim's for dense graphs (heap-based).

🔗 Brute Force Approach

Try all spanning trees (exponential). Kruskal's and Prim's are greedy $O(E \log E)$ algorithms.

⚡ Optimized Solution

Add virtual node 0. Connect to each house with well cost. Then Kruskal's MST.

🔗 Python Code

```
# LeetCode 1168: Optimize Water Distribution in a Village
# Pattern: MST / Graph Greedy

edges=[]
for i,cost in enumerate(wells,1): edges.append((cost,0,i))
for pipe in pipes: edges.append((pipe[2],pipe[0],pipe[1]))
edges.sort()
parent=list(range(n+1))
def find(x):
    while parent[x]!=x: parent[x]=parent[parent[x]]; x=parent[x]
    return x
total=0
for cost,u,v in edges:
    ru,rv=find(u),find(v)
    if ru!=rv: parent[ru]=rv; total+=cost
return total
```

🔗 Dry Run

$n=3$, wells=[1,2,2], pipes=[[1,2,1],[2,3,1]] → 3

🔗 Complexity Analysis

$O(E \log E)$ time.

🔗 Interview Tips

Kruskal's = sort edges + DSU. Prim's = heap + visited. Both $O(E \log E)$. Explain MST property: adding cheapest edge that doesn't create cycle.

Problem 6: LC 1489 — Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree

LeetCode #1489 · leetcode.com/problems/

🔗 Problem Understanding

Classify edges as critical (removing increases MST weight) or pseudo-critical (in some MST).

🔗 Pattern Recognition

This is a MINIMUM SPANNING TREE problem. Kruskal's for sparse graphs (sort edges + DSU), Prim's for dense graphs (heap-based).

🔗 Brute Force Approach

Try all spanning trees (exponential). Kruskal's and Prim's are greedy $O(E \log E)$ algorithms.

⚡ Optimized Solution

For each edge: (1) remove it, if MST weight increases or graph disconnects → critical. (2) force include it, if MST weight same → pseudo-critical.

🔗 Python Code

```
# LeetCode 1489: Find Critical and Pseudo-Critical Edges in Minimum Spanning Tree
# Pattern: MST / Graph Greedy

# Kruskal's with edge exclusion/inclusion tests
# Run n times with/without each edge
# Compare MST weights
```

🔗 Dry Run

$n=5$, edges=[[0,1,1],[1,2,1],[2,3,2],[0,3,2],[0,4,3],[3,4,3],[1,4,6]] → [[0,1],[2,3,4,5]]

🔗 Complexity Analysis

$O(E^2 * \alpha(V))$ time.

🔗 Interview Tips

Kruskal's = sort edges + DSU. Prim's = heap + visited. Both $O(E \log E)$. Explain MST property: adding cheapest edge that doesn't create cycle.

Pattern 24 Summary: MST / Graph Greedy

Core intuition: minimum cost to connect all nodes. Kruskal's = sort edges + DSU. Prim's = start from any node + min-heap. Both give same MST weight.

Pattern 25: Trie

Problems: LC 208, LC 211, LC 648, LC 677, LC 212, LC 1268

Problem 1: LC 208 — Implement Trie (Prefix Tree)

LeetCode #208 · leetcode.com/problems/

🔖 Problem Understanding

Implement insert, search (exact word), and startsWith (prefix check) operations.

🔖 Pattern Recognition

This is a TRIE problem. Key signals: prefix matching, autocomplete, word search. Each node stores a character; mark end of words.

🔖 Brute Force Approach

Store all strings in a list. For prefix search, iterate and check startswith(). $O(n \cdot L)$ per query vs $O(L)$ for Trie.

⚡ Optimized Solution

Each node has children dict and is_end flag.

🔖 Python Code

```
# LeetCode 208: Implement Trie (Prefix Tree)
# Pattern: Trie

class TrieNode:
    def __init__(self): self.children={}; self.is_end=False

class Trie:
    def __init__(self): self.root=TrieNode()
    def insert(self,word):
        node=self.root
        for c in word:
            if c not in node.children: node.children[c]=TrieNode()
            node=node.children[c]
        node.is_end=True
    def search(self,word):
        node=self.root
        for c in word:
            if c not in node.children: return False
            node=node.children[c]
        return node.is_end
    def startsWith(self,prefix):
        node=self.root
        for c in prefix:
            if c not in node.children: return False
            node=node.children[c]
        return True
```

🔖 Dry Run

insert('apple'); search('apple')→True; search('app')→False; startsWith('app')→True

🔖 Complexity Analysis

$O(L)$ per operation where L =word length.

🔖 Interview Tips

Trie is essentially a 26-ary tree for lowercase English letters. Space is $O(n*L)$. Trade-off vs hash map: prefix queries are $O(L)$ vs $O(1)$ for exact match.

Problem 2: LC 211 — Design Add and Search Words Data Structure

LeetCode #211 · leetcode.com/problems/

🔖 Problem Understanding

Support addWord and search (with '.' wildcard matching any char).

🔖 Pattern Recognition

This is a TRIE problem. Key signals: prefix matching, autocomplete, word search. Each node stores a character; mark end of words.

🔖 Brute Force Approach

Store all strings in a list. For prefix search, iterate and check startswith(). $O(n*L)$ per query vs $O(L)$ for Trie.

🚀 Optimized Solution

Standard Trie. For search with '.', try all children recursively.

🔖 Python Code

```
# LeetCode 211: Design Add and Search Words Data Structure
# Pattern: Trie

class WordDictionary:
    def __init__(self):
        self.root = {'is_end': False}
    def addWord(self, word):
        node = self.root
        for c in word:
            if c not in node: node[c] = {'is_end': False}
            node = node[c]
        node['is_end'] = True
    def search(self, word):
        def dfs(node, i):
            if i == len(word): return node['is_end']
            c = word[i]
            if c == '.':
                return any(dfs(node[k], i+1) for k in node if k != 'is_end')
            if c not in node: return False
            return dfs(node[c], i+1)
        return dfs(self.root, 0)
```

🔖 Dry Run

addWord('bad'), addWord('dad'), search('.ad')→True, search('b..')→True

🔍 Complexity Analysis

$O(L)$ add, $O(26^L)$ worst-case search.

🔍 Interview Tips

Trie is essentially a 26-ary tree for lowercase English letters. Space is $O(n \cdot L)$. Trade-off vs hash map: prefix queries are $O(L)$ vs $O(1)$ for exact match.

Problem 3: LC 648 — Replace Words

LeetCode #648 · [leetcode.com/problems/](https://leetcode.com/problems/replace-words/)

🔍 Problem Understanding

Replace words in sentence with their shortest root from dictionary.

🔍 Pattern Recognition

This is a TRIE problem. Key signals: prefix matching, autocomplete, word search. Each node stores a character; mark end of words.

🔍 Brute Force Approach

Store all strings in a list. For prefix search, iterate and check startswith(). $O(n \cdot L)$ per query vs $O(L)$ for Trie.

⚡ Optimized Solution

Build trie from roots. For each word in sentence, find shortest matching prefix in trie.

🔍 Python Code

```
# LeetCode 648: Replace Words
# Pattern: Trie

class TrieNode:
    def __init__(self): self.children={}; self.is_end=False
root_node=TrieNode()
for root in dictionary:
    node=root_node
    for c in root:
        if c not in node.children: node.children[c]=TrieNode()
        node=node.children[c]
    node.is_end=True
def find_root(word):
    node=root_node
    for i,c in enumerate(word):
        if c not in node.children: break
        node=node.children[c]
        if node.is_end: return word[:i+1]
    return word
return ' '.join(find_root(w) for w in sentence.split())
```

🔍 Dry Run

dictionary=['cat','bat','rat'], sentence='the cattle was rattled by the battery' → 'the cat was rat by the bat'

🔍 Complexity Analysis

$O(\text{sum of root lengths} + \text{sentence length})$ time.

🔍 Interview Tips

Trie is essentially a 26-ary tree for lowercase English letters. Space is $O(n*L)$. Trade-off vs hash map: prefix queries are $O(L)$ vs $O(1)$ for exact match.

Problem 4: LC 677 — Map Sum Pairs

LeetCode #677 · [leetcode.com/problems/](https://leetcode.com/problems/map-sum-pairs/)

🔍 Problem Understanding

Insert key-value pairs and compute sum of values for keys with given prefix.

🔍 Pattern Recognition

This is a TRIE problem. Key signals: prefix matching, autocomplete, word search. Each node stores a character; mark end of words.

🔍 Brute Force Approach

Store all strings in a list. For prefix search, iterate and check `startswith()`. $O(n*L)$ per query vs $O(L)$ for Trie.

⚡ Optimized Solution

Trie where each node stores the sum of all values in its subtree.

🔍 Python Code

```
# LeetCode 677: Map Sum Pairs
# Pattern: Trie

class MapSum:
    def __init__(self): self.trie={}; self.vals={}
    def insert(self, key, val):
        diff=val-self.vals.get(key,0)
        self.vals[key]=val
        node=self.trie
        for c in key:
            if c not in node: node[c]={'_sum':0}
            node=node[c]; node['_sum']+=diff
    def sum(self, prefix):
        node=self.trie
        for c in prefix:
            if c not in node: return 0
            node=node[c]
        return node['_sum']
```

🔍 Dry Run

`insert('apple',3); sum('ap')→3; insert('app',2); sum('ap')→5`

🔖 Complexity Analysis

$O(L)$ per operation.

🔖 Interview Tips

Trie is essentially a 26-ary tree for lowercase English letters. Space is $O(n*L)$. Trade-off vs hash map: prefix queries are $O(L)$ vs $O(1)$ for exact match.

Problem 5: LC 212 — Word Search II

LeetCode #212 · leetcode.com/problems/

🔖 Problem Understanding

Find all words from a given list that exist in a grid.

🔖 Pattern Recognition

This is a TRIE problem. Key signals: prefix matching, autocomplete, word search. Each node stores a character; mark end of words.

🔖 Brute Force Approach

Store all strings in a list. For prefix search, iterate and check `startswith()`. $O(n*L)$ per query vs $O(L)$ for Trie.

⚡ Optimized Solution

Build trie from word list. DFS on grid, traverse trie simultaneously. Prune early.

🔖 Python Code

```
# LeetCode 212: Word Search II
# Pattern: Trie

class TrieNode:
    def __init__(self): self.children={}; self.word=None
root=TrieNode()
for word in words:
    node=root
    for c in word:
        if c not in node.children: node.children[c]=TrieNode()
        node=node.children[c]
    node.word=word
res=[]
def dfs(i,j,node):
    if node.word: res.append(node.word); node.word=None
    if not(0<=i<len(board) and 0<=j<len(board[0])): return
    c=board[i][j]
    if c not in node.children: return
    board[i][j]='#'; child=node.children[c]
    for di,dj in [(0,1),(0,-1),(1,0),(-1,0)]: dfs(i+di,j+dj,child)
    board[i][j]=c
for i in range(len(board)):
    for j in range(len(board[0])): dfs(i,j,root)
return res
```

🔍 Dry Run

board=[['o','a','a','n'],['e','t','a','e'],['i','h','k','r'],['i','f','l','v']], words=['oath','pea','eat','rain'] → ['oath','eat']

🔍 Complexity Analysis

$O(m \cdot n \cdot 4^L)$ time.

🔍 Interview Tips

Trie is essentially a 26-ary tree for lowercase English letters. Space is $O(n \cdot L)$. Trade-off vs hash map: prefix queries are $O(L)$ vs $O(1)$ for exact match.

Problem 6: LC 1268 — Search Suggestions System

LeetCode #1268 · leetcode.com/problems/

🔍 Problem Understanding

For each prefix of searchWord, return top 3 lexicographically smallest matching products.

🔍 Pattern Recognition

This is a TRIE problem. Key signals: prefix matching, autocomplete, word search. Each node stores a character; mark end of words.

🔍 Brute Force Approach

Store all strings in a list. For prefix search, iterate and check startswith(). $O(n \cdot L)$ per query vs $O(L)$ for Trie.

⚡ Optimized Solution

Sort products. For each prefix, binary search for range of matching products.

🔍 Python Code

```
# LeetCode 1268: Search Suggestions System
# Pattern: Trie

import bisect
products.sort()
res=[]
prefix=''
for c in searchWord:
    prefix+=c
    i=bisect.bisect_left(products,prefix)
    res.append([p for p in products[i:i+3] if p.startswith(prefix)])
return res
```

🔍 Dry Run

products=['mobile','mouse','moneypot','monitor','mousepad'], searchWord='mouse' →
[['mobile','moneypot','monitor'], ['mobile','moneypot','monitor'], ['mouse','mousepad'], ['mouse','mousepad'], ['mouse','mousepad']]

🔍 Complexity Analysis

$O(n \log n + L * \log n)$ time.

Interview Tips

Trie is essentially a 26-ary tree for lowercase English letters. Space is $O(n * L)$. Trade-off vs hash map: prefix queries are $O(L)$ vs $O(1)$ for exact match.

Pattern 25 Summary: Trie

Core intuition: tree of characters enabling $O(L)$ prefix operations. Each path from root to leaf is a word.

Recognition tricks: 'prefix search', 'autocomplete', 'word search II'. Space is the trade-off.

Pattern 26: Bit Manipulation

Problems: LC 136, LC 137, LC 191, LC 338, LC 268, LC 190

Problem 1: LC 136 — Single Number

LeetCode #136 · leetcode.com/problems/

🔍 Problem Understanding

Every element appears twice except one. Find it.

🔍 Pattern Recognition

This is a BIT MANIPULATION problem. Key signals: XOR tricks, counting bits, powers of 2. Know: $n \& (n-1)$ removes lowest set bit, $a \oplus a = 0$, $a \oplus 0 = a$.

🔍 Brute Force Approach

Use division/modulo or string representation. Bitwise operations are $O(1)$ and use hardware instructions directly.

⚡ Optimized Solution

XOR all numbers. Pairs cancel out ($a \oplus a = 0$). Single remains.

🔍 Python Code

```
# LeetCode 136: Single Number
# Pattern: Bit Manipulation

res=0
for n in nums: res^=n
return res
```

🔍 Dry Run

nums=[4,1,2,1,2] → 4

🔍 Complexity Analysis

$O(n)$ time, $O(1)$ space.

🔍 Interview Tips

Be comfortable with: XOR (cancel pairs), AND with $(n-1)$ (remove lowest set bit), shift operators. State whether integer is signed or unsigned.

Problem 2: LC 137 — Single Number II

LeetCode #137 · leetcode.com/problems/

🔍 Problem Understanding

Every element appears three times except one. Find it.

🔍 Pattern Recognition

This is a BIT MANIPULATION problem. Key signals: XOR tricks, counting bits, powers of 2. Know: $n \& (n-1)$ removes lowest set bit, $a \wedge a = 0$, $a \wedge 0 = a$.

🔍 Brute Force Approach

Use division/modulo or string representation. Bitwise operations are $O(1)$ and use hardware instructions directly.

⚡ Optimized Solution

Count bits modulo 3 for each bit position. Remaining bits form the answer.

🔍 Python Code

```
# LeetCode 137: Single Number II
# Pattern: Bit Manipulation

res=0
for i in range(32):
    bit_sum=sum((n>>i)&1 for n in nums)%3
    if bit_sum: res|=(bit_sum<<i)
# handle negative numbers for 32-bit
if res>=2**31: res-=2**32
return res
```

🔍 Dry Run

nums=[2,2,3,2] → 3

🔍 Complexity Analysis

$O(32n) = O(n)$ time, $O(1)$ space.

🔍 Interview Tips

Be comfortable with: XOR (cancel pairs), AND with $(n-1)$ (remove lowest set bit), shift operators. State whether integer is signed or unsigned.

Problem 3: LC 191 — Number of 1 Bits

LeetCode #191 · leetcode.com/problems/

🔍 Problem Understanding

Count number of 1 bits (Hamming weight) in a 32-bit unsigned integer.

🔍 Pattern Recognition

This is a BIT MANIPULATION problem. Key signals: XOR tricks, counting bits, powers of 2. Know: $n \& (n-1)$ removes lowest set bit, $a \wedge a = 0$, $a \wedge 0 = a$.

🔍 Brute Force Approach

Use division/modulo or string representation. Bitwise operations are $O(1)$ and use hardware instructions directly.

🔍 Dry Run

$n=5 \rightarrow [0,1,1,2,1,2]$

🔍 Complexity Analysis

$O(n)$ time, $O(n)$ space.

🔍 Interview Tips

Be comfortable with: XOR (cancel pairs), AND with $(n-1)$ (remove lowest set bit), shift operators. State whether integer is signed or unsigned.

Problem 5: LC 268 — Missing Number

LeetCode #268 · [leetcode.com/problems/](https://leetcode.com/problems/missing-number/)

🔍 Problem Understanding

Find missing number in array containing n distinct numbers in range $[0, n]$.

🔍 Pattern Recognition

This is a BIT MANIPULATION problem. Key signals: XOR tricks, counting bits, powers of 2. Know: $n \& (n-1)$ removes lowest set bit, $a \wedge a = 0$, $a \wedge 0 = a$.

🔍 Brute Force Approach

Use division/modulo or string representation. Bitwise operations are $O(1)$ and use hardware instructions directly.

⚡ Optimized Solution

XOR all indices and values. Missing number remains (all others cancel).

🔍 Python Code

```
# LeetCode 268: Missing Number
# Pattern: Bit Manipulation

res = len(nums)
for i, n in enumerate(nums): res ^= i ^ n
return res
```

🔍 Dry Run

$\text{nums}=[3,0,1] \rightarrow 2$

🔍 Complexity Analysis

$O(n)$ time, $O(1)$ space.

🔍 Interview Tips

Be comfortable with: XOR (cancel pairs), AND with $(n-1)$ (remove lowest set bit), shift operators. State whether integer is signed or unsigned.

Problem 6: LC 190 — Reverse Bits

LeetCode #190 · leetcode.com/problems/

🔍 Problem Understanding

Reverse bits of a 32-bit unsigned integer.

🔍 Pattern Recognition

This is a BIT MANIPULATION problem. Key signals: XOR tricks, counting bits, powers of 2. Know: $n \& (n-1)$ removes lowest set bit, $a^a=0$, $a^0=a$.

🔍 Brute Force Approach

Use division/modulo or string representation. Bitwise operations are $O(1)$ and use hardware instructions directly.

⚡ Optimized Solution

Shift result left and take LSB of n . Repeat 32 times.

🔍 Python Code

```
# LeetCode 190: Reverse Bits
# Pattern: Bit Manipulation

res=0
for _ in range(32):
    res=(res<<1) | (n&1)
    n>>=1
return res
```

🔍 Dry Run

$n=00000010100101000001111010011100 \rightarrow$ reversed bits value

🔍 Complexity Analysis

$O(32) = O(1)$ time.

🔍 Interview Tips

Be comfortable with: XOR (cancel pairs), AND with $(n-1)$ (remove lowest set bit), shift operators. State whether integer is signed or unsigned.

Pattern 26 Summary: Bit Manipulation

Core intuitions: XOR cancels pairs, $n \& (n-1)$ removes lowest bit, right shift = divide by 2. Recognition tricks: 'single number', 'count bits', 'power of 2'. Use 32-bit integer carefully in Python.

Pattern 27: 1D DP Basics

Problems: LC 70, LC 198, LC 213, LC 322, LC 279, LC 300

Problem 1: LC 70 — Climbing Stairs

LeetCode #70 · leetcode.com/problems/

🔖 Problem Understanding

How many ways to climb n stairs taking 1 or 2 steps at a time?

🔖 Pattern Recognition

This is 1D DYNAMIC PROGRAMMING. Key signals: optimal substructure, overlapping subproblems. Define $dp[i]$, write recurrence, identify base cases.

🔖 Brute Force Approach

Recursion without memoization: recomputes same subproblems exponentially. DP caches results.

🔖 Optimized Solution

$dp[i] = dp[i-1] + dp[i-2]$. It's Fibonacci!

🔖 Python Code

```
# LeetCode 70: Climbing Stairs
# Pattern: 1D DP Basics

a,b=1,1
for _ in range(n-1): a,b=b,a+b
return b
```

🔖 Dry Run

$n=3 \rightarrow 3$ (1+1+1, 1+2, 2+1)

🔖 Complexity Analysis

$O(n)$ time, $O(1)$ space.

🔖 Interview Tips

Define dp array meaning precisely. State recurrence, then base case. Consider space optimization (rolling array). Edge cases: $n=0$, $n=1$.

Problem 2: LC 198 — House Robber

LeetCode #198 · leetcode.com/problems/

🔖 Problem Understanding

Rob houses in a line without robbing adjacent houses. Maximize money.

🔍 Pattern Recognition

This is 1D DYNAMIC PROGRAMMING. Key signals: optimal substructure, overlapping subproblems. Define $dp[i]$, write recurrence, identify base cases.

🔍 Brute Force Approach

Recursion without memoization: recomputes same subproblems exponentially. DP caches results.

⚡ Optimized Solution

$dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$. Either skip current or rob it.

🔍 Python Code

```
# LeetCode 198: House Robber
# Pattern: 1D DP Basics

prev2=prev1=0
for n in nums: prev2,prev1=prev1,max(prev1,prev2+n)
return prev1
```

🔍 Dry Run

$\text{nums}=[2,7,9,3,1] \rightarrow 12$ (rob 2,9,1)

🔍 Complexity Analysis

$O(n)$ time, $O(1)$ space.

🔍 Interview Tips

Define dp array meaning precisely. State recurrence, then base case. Consider space optimization (rolling array). Edge cases: $n=0$, $n=1$.

Problem 3: LC 213 — House Robber II

LeetCode #213 · leetcode.com/problems/

🔍 Problem Understanding

Houses arranged in a circle. First and last are adjacent.

🔍 Pattern Recognition

This is 1D DYNAMIC PROGRAMMING. Key signals: optimal substructure, overlapping subproblems. Define $dp[i]$, write recurrence, identify base cases.

🔍 Brute Force Approach

Recursion without memoization: recomputes same subproblems exponentially. DP caches results.

⚡ Optimized Solution

Run House Robber twice: once on $\text{nums}[0..n-2]$, once on $\text{nums}[1..n-1]$. Return max.

🔍 Python Code

```
# LeetCode 213: House Robber II
# Pattern: 1D DP Basics
```

```
def rob(arr):
    p2=p1=0
    for n in arr: p2,p1=p1,max(p1,p2+n)
    return p1
return max(nums[0], rob(nums[:-1]), rob(nums[1:]))
```

🔗 Dry Run

nums=[2,3,2] → 3; nums=[1,2,3,1] → 4

🔗 Complexity Analysis

O(n) time, O(1) space.

🔗 Interview Tips

Define dp array meaning precisely. State recurrence, then base case. Consider space optimization (rolling array). Edge cases: n=0, n=1.

Problem 4: LC 322 — Coin Change

LeetCode #322 · leetcode.com/problems/

🔗 Problem Understanding

Find minimum coins to make up amount. Return -1 if impossible.

🔗 Pattern Recognition

This is 1D DYNAMIC PROGRAMMING. Key signals: optimal substructure, overlapping subproblems. Define dp[i], write recurrence, identify base cases.

🔗 Brute Force Approach

Recursion without memoization: recomputes same subproblems exponentially. DP caches results.

⚡ Optimized Solution

dp[i] = min coins to make amount i. For each coin, dp[i] = min(dp[i], dp[i-coin]+1).

🔗 Python Code

```
# LeetCode 322: Coin Change
# Pattern: 1D DP Basics

dp=[float('inf')]*(amount+1); dp[0]=0
for coin in coins:
    for i in range(coin,amount+1):
        dp[i]=min(dp[i],dp[i-coin]+1)
return dp[amount] if dp[amount]!=float('inf') else -1
```

🔗 Dry Run

coins=[1,5,11], amount=15 → 3 (5+5+5). Note: greedy fails here. DP is correct.

🔗 Complexity Analysis

O(amount * n) time, O(amount) space.

🔖 Interview Tips

Define dp array meaning precisely. State recurrence, then base case. Consider space optimization (rolling array). Edge cases: $n=0$, $n=1$.

Problem 5: LC 279 — Perfect Squares

LeetCode #279 · [leetcode.com/problems/](https://leetcode.com/problems/perfect-squares/)

🔖 Problem Understanding

Find minimum number of perfect squares that sum to n .

🔖 Pattern Recognition

This is 1D DYNAMIC PROGRAMMING. Key signals: optimal substructure, overlapping subproblems. Define $dp[i]$, write recurrence, identify base cases.

🔖 Brute Force Approach

Recursion without memoization: recomputes same subproblems exponentially. DP caches results.

⚡ Optimized Solution

Same as coin change where coins are perfect squares.

🔖 Python Code

```
# LeetCode 279: Perfect Squares
# Pattern: 1D DP Basics

dp=[float('inf')]*(n+1); dp[0]=0
for i in range(1,n+1):
    j=1
    while j*j<=i: dp[i]=min(dp[i],dp[i-j*j]+1); j+=1
return dp[n]
```

🔖 Dry Run

$n=12 \rightarrow 3$ (4+4+4); $n=13 \rightarrow 2$ (4+9)

🔖 Complexity Analysis

$O(n * \sqrt{n})$ time, $O(n)$ space.

🔖 Interview Tips

Define dp array meaning precisely. State recurrence, then base case. Consider space optimization (rolling array). Edge cases: $n=0$, $n=1$.

Problem 6: LC 300 — Longest Increasing Subsequence

LeetCode #300 · [leetcode.com/problems/](https://leetcode.com/problems/longest-increasing-subsequence/)

🔖 Problem Understanding

Find length of longest strictly increasing subsequence.

🔍 Pattern Recognition

This is 1D DYNAMIC PROGRAMMING. Key signals: optimal substructure, overlapping subproblems. Define $dp[i]$, write recurrence, identify base cases.

🔍 Brute Force Approach

Recursion without memoization: recomputes same subproblems exponentially. DP caches results.

⚡ Optimized Solution

$dp[i]$ = LIS ending at index i . For each $j < i$ where $nums[j] < nums[i]$: $dp[i] = \max(dp[i], dp[j] + 1)$.

🔍 Python Code

```
# LeetCode 300: Longest Increasing Subsequence
# Pattern: 1D DP Basics

dp = [1] * len(nums)
for i in range(1, len(nums)):
    for j in range(i):
        if nums[j] < nums[i]: dp[i] = max(dp[i], dp[j] + 1)
return max(dp)

# O(n log n) with binary search:
import bisect
tails = []
for n in nums:
    i = bisect.bisect_left(tails, n)
    if i == len(tails): tails.append(n)
    else: tails[i] = n
return len(tails)
```

🔍 Dry Run

$nums = [10, 9, 2, 5, 3, 7, 101, 18] \rightarrow 4$ (2,3,7,101 or 2,5,7,101)

🔍 Complexity Analysis

$O(n^2)$ DP, $O(n \log n)$ binary search approach.

🔍 Interview Tips

Define dp array meaning precisely. State recurrence, then base case. Consider space optimization (rolling array). Edge cases: $n=0$, $n=1$.

Pattern 27 Summary: 1D DP Basics

Core intuition: $dp[i]$ = answer for problem of size i . Build up from base cases. Key steps: define state, write recurrence, identify base cases, optimize space if possible.

Pattern 28: Knapsack / Subset DP

Problems: LC 416, LC 494, LC 518, LC 474, LC 1049, LC 879

Problem 1: LC 416 — Partition Equal Subset Sum

LeetCode #416 · [leetcode.com/problems/](https://leetcode.com/problems/partition-equal-subset-sum/)

🔍 Problem Understanding

Can array be partitioned into two equal-sum subsets?

🔍 Pattern Recognition

This is a KNAPSACK DP problem. Key signals: select items with constraint to maximize/count. 0/1 Knapsack: each item once. Unbounded: unlimited use.

🔍 Brute Force Approach

Try all 2^n subsets (0/1 knapsack). DP reduces to $O(n * \text{capacity})$.

⚡ Optimized Solution

Find if subset sums to total/2. 0/1 knapsack. $dp[j]$ = can we achieve sum j?

🔍 Python Code

```
# LeetCode 416: Partition Equal Subset Sum
# Pattern: Knapsack / Subset DP

total = sum(nums)
if total % 2: return False
target = total // 2
dp = {0}
for n in nums:
    dp |= {s + n for s in dp if s + n <= target}
return target in dp
```

🔍 Dry Run

$\text{nums} = [1, 5, 11, 5] \rightarrow \text{True} (1+5+5=11)$

🔍 Complexity Analysis

$O(n * \text{target})$ time, $O(\text{target})$ space.

🔍 Interview Tips

0/1 knapsack: iterate capacity in reverse to avoid reusing items. Unbounded: iterate forward. 2D DP can often be optimized to 1D.

Problem 2: LC 494 — Target Sum

LeetCode #494 · [leetcode.com/problems/](https://leetcode.com/problems/target-sum/)

🔗 Problem Understanding

Assign + or - to each number. Count ways to achieve target sum.

🔗 Pattern Recognition

This is a KNAPSACK DP problem. Key signals: select items with constraint to maximize/count. 0/1 Knapsack: each item once. Unbounded: unlimited use.

🔗 Brute Force Approach

Try all 2^n subsets (0/1 knapsack). DP reduces to $O(n * \text{capacity})$.

⚡ Optimized Solution

DP with hash map counting ways to reach each sum.

🔗 Python Code

```
# LeetCode 494: Target Sum
# Pattern: Knapsack / Subset DP

from collections import defaultdict
dp=defaultdict(int); dp[0]=1
for n in nums:
    new_dp=defaultdict(int)
    for s,cnt in dp.items():
        new_dp[s+n]+=cnt; new_dp[s-n]+=cnt
    dp=new_dp
return dp[target]
```

🔗 Dry Run

nums=[1,1,1,1,1], target=3 → 5

🔗 Complexity Analysis

$O(n * \text{sum})$ time, $O(\text{sum})$ space.

🔗 Interview Tips

0/1 knapsack: iterate capacity in reverse to avoid reusing items. Unbounded: iterate forward. 2D DP can often be optimized to 1D.

Problem 3: LC 518 — Coin Change II

LeetCode #518 · leetcode.com/problems/

🔗 Problem Understanding

Count number of ways to make up amount using given coins (unlimited use).

🔗 Pattern Recognition

This is a KNAPSACK DP problem. Key signals: select items with constraint to maximize/count. 0/1 Knapsack: each item once. Unbounded: unlimited use.

🔗 Brute Force Approach

Try all 2^n subsets (0/1 knapsack). DP reduces to $O(n * \text{capacity})$.

⚡ Optimized Solution

Unbounded knapsack. $dp[i]$ = ways to make amount i .

🔗 Python Code

```
# LeetCode 518: Coin Change II
# Pattern: Knapsack / Subset DP

dp=[0]*(amount+1); dp[0]=1
for coin in coins:
    for i in range(coin,amount+1):
        dp[i]+=dp[i-coin]
return dp[amount]
```

🔗 Dry Run

amount=5, coins=[1,2,5] → 4 (5, 2+2+1, 2+1+1+1, 1+1+1+1+1)

🔗 Complexity Analysis

$O(\text{amount} * n)$ time, $O(\text{amount})$ space.

🔗 Interview Tips

0/1 knapsack: iterate capacity in reverse to avoid reusing items. Unbounded: iterate forward. 2D DP can often be optimized to 1D.

Problem 4: LC 474 — Ones and Zeroes

LeetCode #474 · [leetcode.com/problems/](https://leetcode.com/problems/ones-and-zeroes/)

🔗 Problem Understanding

Find max subset of strings satisfying at most m zeros and n ones.

🔗 Pattern Recognition

This is a KNAPSACK DP problem. Key signals: select items with constraint to maximize/count. 0/1 Knapsack: each item once. Unbounded: unlimited use.

🔗 Brute Force Approach

Try all 2^n subsets (0/1 knapsack). DP reduces to $O(n * \text{capacity})$.

⚡ Optimized Solution

2D 0/1 knapsack where constraints are (zeros, ones) instead of weight.

🔗 Python Code

```
# LeetCode 474: Ones and Zeroes
# Pattern: Knapsack / Subset DP

dp=[[0]*(n+1) for _ in range(m+1)]
for s in strs:
    zeros=s.count('0'); ones=s.count('1')
    for i in range(m,zeros-1,-1):
        for j in range(n,ones-1,-1):
```

```
dp[i][j]=max(dp[i][j],dp[i-zeros][j-ones]+1)
return dp[m][n]
```

🔗 Dry Run

strs=['10','0001','111001','1','0'], m=5, n=3 → 4

🔗 Complexity Analysis

$O(\text{len}(\text{strs}) * m * n)$ time.

🔗 Interview Tips

0/1 knapsack: iterate capacity in reverse to avoid reusing items. Unbounded: iterate forward. 2D DP can often be optimized to 1D.

Problem 5: LC 1049 — Last Stone Weight II

LeetCode #1049 · leetcode.com/problems/

🔗 Problem Understanding

Split stones into two groups. Minimize difference of their sums.

🔗 Pattern Recognition

This is a KNAPSACK DP problem. Key signals: select items with constraint to maximize/count. 0/1 Knapsack: each item once. Unbounded: unlimited use.

🔗 Brute Force Approach

Try all 2^n subsets (0/1 knapsack). DP reduces to $O(n * \text{capacity})$.

⚡ Optimized Solution

Find largest subset sum $\leq \text{total}/2$. Equivalent to minimizing $|S1 - S2|$.

🔗 Python Code

```
# LeetCode 1049: Last Stone Weight II
# Pattern: Knapsack / Subset DP

total=sum(stones)
target=total//2
dp=[False]*(target+1); dp[0]=True
for s in stones:
    for j in range(target,s-1,-1):
        dp[j]=dp[j] or dp[j-s]
best=max(j for j in range(target+1) if dp[j])
return total-2*best
```

🔗 Dry Run

stones=[2,7,4,1,8,1] → 1

🔗 Complexity Analysis

$O(n * \text{total})$ time, $O(\text{total})$ space.

🔗 Interview Tips

0/1 knapsack: iterate capacity in reverse to avoid reusing items. Unbounded: iterate forward. 2D DP can often be optimized to 1D.

Problem 6: LC 879 — Profitable Schemes

LeetCode #879 · [leetcode.com/problems/](https://leetcode.com/problems/profitable-schemes/)

🔍 Problem Understanding

Count schemes where profit $\geq$ minProfit using at most n members.

🔍 Pattern Recognition

This is a KNAPSACK DP problem. Key signals: select items with constraint to maximize/count. 0/1 Knapsack: each item once. Unbounded: unlimited use.

🔍 Brute Force Approach

Try all 2^n subsets (0/1 knapsack). DP reduces to $O(n * \text{capacity})$.

⚡ Optimized Solution

3D DP: $\text{dp}[\text{members}][\text{profit}] = \text{count}$. Treat profit as bounded dimension.

🔍 Python Code

```
# LeetCode 879: Profitable Schemes
# Pattern: Knapsack / Subset DP

MOD=10**9+7
dp=[[0]*(minProfit+1) for _ in range(n+1)]
dp[0][0]=1
for g,p in zip(group,profit):
    for j in range(n,g-1,-1):
        for k in range(minProfit,-1,-1):
            new_k=min(k+p,minProfit)
            dp[j][new_k]=(dp[j][new_k]+dp[j-g][k])%MOD
return sum(dp[j][minProfit] for j in range(n+1))%MOD
```

🔍 Dry Run

$n=5, \text{minProfit}=3, \text{group}=[2,2], \text{profit}=[2,3] \rightarrow 2$

🔍 Complexity Analysis

$O(\text{len} * n * \text{minProfit})$ time.

🔍 Interview Tips

0/1 knapsack: iterate capacity in reverse to avoid reusing items. Unbounded: iterate forward. 2D DP can often be optimized to 1D.

Pattern 28 Summary: Knapsack / Subset DP

Core intuition: $dp[capacity]$ = best answer using items seen so far. 0/1: reverse iteration. Unbounded: forward iteration. Key reduction: many subset problems $\rightarrow$ knapsack.

Pattern 29: Grid DP

Problems: LC 62, LC 63, LC 64, LC 221, LC 931, LC 120

Problem 1: LC 62 — Unique Paths

LeetCode #62 · [leetcode.com/problems/](https://leetcode.com/problems/unique-paths/)

🔖 Problem Understanding

Count paths from top-left to bottom-right of $m \times n$ grid (only right/down moves).

🔖 Pattern Recognition

This is GRID DP. Key signals: paths in 2D grid, rectangle problems. $dp[i][j]$ depends on $dp[i-1][j]$ and $dp[i][j-1]$ (or diagonals).

🔖 Brute Force Approach

Recursion without memoization: exponential paths. DP fills table in $O(m \cdot n)$.

🔖 Optimized Solution

$dp[i][j] = dp[i-1][j] + dp[i][j-1]$. Can optimize to 1D.

🔖 Python Code

```
# LeetCode 62: Unique Paths
# Pattern: Grid DP

dp = [1] * n
for _ in range(m-1):
    for j in range(1, n): dp[j] += dp[j-1]
return dp[-1]
```

🔖 Dry Run

$m=3, n=7 \rightarrow 28$

🔖 Complexity Analysis

$O(m \cdot n)$ time, $O(n)$ space.

🔖 Interview Tips

Draw the dp table and fill it manually for small example. State which direction we iterate. Edge cases: 1x1 grid, obstacles.

Problem 2: LC 63 — Unique Paths II

LeetCode #63 · [leetcode.com/problems/](https://leetcode.com/problems/unique-paths-ii/)

🔖 Problem Understanding

Same as Unique Paths but grid has obstacles.

🔍 Pattern Recognition

This is GRID DP. Key signals: paths in 2D grid, rectangle problems. $dp[i][j]$ depends on $dp[i-1][j]$ and $dp[i][j-1]$ (or diagonals).

🔍 Brute Force Approach

Recursion without memoization: exponential paths. DP fills table in $O(m*n)$.

⚡ Optimized Solution

If obstacle, $dp[i][j]=0$. Else $dp[i][j]=dp[i-1][j]+dp[i][j-1]$.

🔍 Python Code

```
# LeetCode 63: Unique Paths II
# Pattern: Grid DP

m,n=len(obstacleGrid),len(obstacleGrid[0])
dp=[0]*n; dp[0]=1
for i in range(m):
    for j in range(n):
        if obstacleGrid[i][j]: dp[j]=0
        elif j>0: dp[j]+=dp[j-1]
return dp[-1]
```

🔍 Dry Run

$obstacleGrid=[[0,0,0],[0,1,0],[0,0,0]] \rightarrow 2$

🔍 Complexity Analysis

$O(m*n)$ time, $O(n)$ space.

🔍 Interview Tips

Draw the dp table and fill it manually for small example. State which direction we iterate. Edge cases: 1x1 grid, obstacles.

Problem 3: LC 64 — Minimum Path Sum

LeetCode #64 · leetcode.com/problems/

🔍 Problem Understanding

Find path from top-left to bottom-right with minimum sum.

🔍 Pattern Recognition

This is GRID DP. Key signals: paths in 2D grid, rectangle problems. $dp[i][j]$ depends on $dp[i-1][j]$ and $dp[i][j-1]$ (or diagonals).

🔍 Brute Force Approach

Recursion without memoization: exponential paths. DP fills table in $O(m*n)$.

⚡ Optimized Solution

$dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$.

🔗 Python Code

```
# LeetCode 64: Minimum Path Sum
# Pattern: Grid DP

m,n=len(grid),len(grid[0])
for i in range(m):
    for j in range(n):
        if i==0 and j==0: continue
        elif i==0: grid[i][j]+=grid[i][j-1]
        elif j==0: grid[i][j]+=grid[i-1][j]
        else: grid[i][j]+=min(grid[i-1][j],grid[i][j-1])
return grid[-1][-1]
```

🔗 Dry Run

grid=[[1,3,1],[1,5,1],[4,2,1]] → 7 (1→3→1→1→1)

🔗 Complexity Analysis

$O(m*n)$ time, $O(1)$ space (in-place).

🔗 Interview Tips

Draw the dp table and fill it manually for small example. State which direction we iterate. Edge cases: 1x1 grid, obstacles.

Problem 4: LC 221 — Maximal Square

LeetCode #221 · leetcode.com/problems/

🔗 Problem Understanding

Find largest square containing only '1's in binary matrix.

🔗 Pattern Recognition

This is GRID DP. Key signals: paths in 2D grid, rectangle problems. $dp[i][j]$ depends on $dp[i-1][j]$ and $dp[i][j-1]$ (or diagonals).

🔗 Brute Force Approach

Recursion without memoization: exponential paths. DP fills table in $O(m*n)$.

⚡ Optimized Solution

$dp[i][j]$ = side length of largest square with bottom-right at (i,j). $dp[i][j] = \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1$ if $matrix[i][j]=='1'$.

🔗 Python Code

```
# LeetCode 221: Maximal Square
# Pattern: Grid DP

m,n=len(matrix),len(matrix[0])
dp=[[0]*(n+1) for _ in range(m+1)]
max_side=0
for i in range(1,m+1):
    for j in range(1,n+1):
```

```

        if matrix[i-1][j-1]=='1':
            dp[i][j]=min(dp[i-1][j],dp[i][j-1],dp[i-1][j-1])+1
            max_side=max(max_side,dp[i][j])
    return max_side*max_side

```

🔍 Dry Run

matrix=[['1','0','1','0','0'],['1','0','1','1','1'],['1','1','1','1','1'],['1','0','0','1','0']] → 4

🔍 Complexity Analysis

$O(m*n)$ time, $O(m*n)$ space.

🔍 Interview Tips

Draw the dp table and fill it manually for small example. State which direction we iterate. Edge cases: 1x1 grid, obstacles.

Problem 5: LC 931 — Minimum Falling Path Sum

LeetCode #931 · leetcode.com/problems/

🔍 Problem Understanding

Find minimum sum path from first row to last row (can move to adjacent diagonal).

🔍 Pattern Recognition

This is GRID DP. Key signals: paths in 2D grid, rectangle problems. $dp[i][j]$ depends on $dp[i-1][j]$ and $dp[i][j-1]$ (or diagonals).

🔍 Brute Force Approach

Recursion without memoization: exponential paths. DP fills table in $O(m*n)$.

⚡ Optimized Solution

$dp[i][j] = \min(dp[i-1][j-1], dp[i-1][j], dp[i-1][j+1]) + matrix[i][j]$.

🔍 Python Code

```

# LeetCode 931: Minimum Falling Path Sum
# Pattern: Grid DP

n=len(matrix)
for i in range(1,n):
    for j in range(n):
        best=matrix[i-1][j]
        if j>0: best=min(best,matrix[i-1][j-1])
        if j<n-1: best=min(best,matrix[i-1][j+1])
        matrix[i][j]+=best
return min(matrix[-1])

```

🔍 Dry Run

matrix=[[2,1,3],[6,5,4],[7,8,9]] → 13 (1+5+7)

🔍 Complexity Analysis

$O(n^2)$ time, $O(1)$ space.

🔖 Interview Tips

Draw the dp table and fill it manually for small example. State which direction we iterate. Edge cases: 1x1 grid, obstacles.

Problem 6: LC 120 — Triangle

LeetCode #120 · [leetcode.com/problems/](https://leetcode.com/problems/triangle/)

🔖 Problem Understanding

Find minimum path sum from top to bottom of triangle.

🔖 Pattern Recognition

This is GRID DP. Key signals: paths in 2D grid, rectangle problems. $dp[i][j]$ depends on $dp[i-1][j]$ and $dp[i][j-1]$ (or diagonals).

🔖 Brute Force Approach

Recursion without memoization: exponential paths. DP fills table in $O(m*n)$.

⚡ Optimized Solution

Bottom-up DP. Start from second-to-last row. $dp[j] = triangle[i][j] + \min(dp[j], dp[j+1])$.

🔖 Python Code

```
# LeetCode 120: Triangle
# Pattern: Grid DP

dp=triangle[-1][:]
for i in range(len(triangle)-2,-1,-1):
    for j in range(len(triangle[i])):
        dp[j]=triangle[i][j]+min(dp[j],dp[j+1])
return dp[0]
```

🔖 Dry Run

triangle=[[2],[3,4],[6,5,7],[4,1,8,3]] → 11 (2+3+5+1)

🔖 Complexity Analysis

$O(n^2)$ time, $O(n)$ space.

🔖 Interview Tips

Draw the dp table and fill it manually for small example. State which direction we iterate. Edge cases: 1x1 grid, obstacles.

Pattern 29 Summary: Grid DP

Core intuition: $dp[i][j]$ = answer for top-left $i \times j$ subgrid. Transitions come from up/left/diagonal. Key optimization: often 1D array (rolling) suffices.

Pattern 30: String DP / Sequence DP

Problems: LC 1143, LC 72, LC 115, LC 583, LC 97, LC 1312

Problem 1: LC 1143 — Longest Common Subsequence

LeetCode #1143 · leetcode.com/problems/

🔖 Problem Understanding

Find length of longest subsequence present in both strings.

🔖 Pattern Recognition

This is STRING DP. Key signals: edit distance, LCS, palindrome. Usually $dp[i][j]$ represents state of two string prefixes.

🔖 Brute Force Approach

Recursion without memoization: exponential subproblems. DP with 2D table gives $O(m*n)$.

🚀 Optimized Solution

$dp[i][j]$ = LCS of $s1[:i]$ and $s2[:j]$. If chars match: $dp[i-1][j-1]+1$. Else $\max(dp[i-1][j], dp[i][j-1])$.

🔖 Python Code

```
# LeetCode 1143: Longest Common Subsequence
# Pattern: String DP / Sequence DP

m, n = len(text1), len(text2)
dp = [[0] * (n+1) for _ in range(m+1)]
for i in range(1, m+1):
    for j in range(1, n+1):
        if text1[i-1] == text2[j-1]: dp[i][j] = dp[i-1][j-1] + 1
        else: dp[i][j] = max(dp[i-1][j], dp[i][j-1])
return dp[m][n]
```

🔖 Dry Run

$text1 = 'abcde', text2 = 'ace' \rightarrow 3 ('ace')$

🔖 Complexity Analysis

$O(m*n)$ time, $O(m*n)$ space.

🔖 Interview Tips

Draw the 2D dp table. Fill row by row. LCS is the foundation: many string DP problems reduce to LCS or a variant.

Problem 2: LC 72 — Edit Distance

LeetCode #72 · leetcode.com/problems/

🔗 Problem Understanding

Find minimum edit operations (insert, delete, replace) to transform word1 to word2.

🔗 Pattern Recognition

This is STRING DP. Key signals: edit distance, LCS, palindrome. Usually $dp[i][j]$ represents state of two string prefixes.

🔗 Brute Force Approach

Recursion without memoization: exponential subproblems. DP with 2D table gives $O(m*n)$.

⚡ Optimized Solution

$dp[i][j]$ = edit distance between $word1[:i]$ and $word2[:j]$. If same: $dp[i-1][j-1]$. Else $1 + \min(\text{insert, delete, replace})$.

🔗 Python Code

```
# LeetCode 72: Edit Distance
# Pattern: String DP / Sequence DP

m,n=len(word1),len(word2)
dp=list(range(n+1))
for i in range(1,m+1):
    prev=dp[0]; dp[0]=i
    for j in range(1,n+1):
        temp=dp[j]
        if word1[i-1]==word2[j-1]: dp[j]=prev
        else: dp[j]=1+min(prev,dp[j],dp[j-1])
        prev=temp
    return dp[n]
```

🔗 Dry Run

$word1='horse', word2='ros' \rightarrow 3$

🔗 Complexity Analysis

$O(m*n)$ time, $O(n)$ space.

🔗 Interview Tips

Draw the 2D dp table. Fill row by row. LCS is the foundation: many string DP problems reduce to LCS or a variant.

Problem 3: LC 115 — Distinct Subsequences

LeetCode #115 · leetcode.com/problems/

🔗 Problem Understanding

Count distinct ways to form t as a subsequence of s .

🔗 Pattern Recognition

This is STRING DP. Key signals: edit distance, LCS, palindrome. Usually $dp[i][j]$ represents state of two string prefixes.

🔍 Brute Force Approach

Recursion without memoization: exponential subproblems. DP with 2D table gives $O(m*n)$.

⚡ Optimized Solution

$dp[i][j]$ = ways to form $t[:j]$ from $s[:i]$. If chars match: $dp[i-1][j-1] + dp[i-1][j]$. Else $dp[i-1][j]$.

🔍 Python Code

```
# LeetCode 115: Distinct Subsequences
# Pattern: String DP / Sequence DP

m,n=len(s),len(t)
dp=[0]*(n+1); dp[0]=1
for i in range(m):
    for j in range(n-1,-1,-1): # reverse to avoid using s[i] twice
        if s[i]==t[j]: dp[j+1]+=dp[j]
return dp[n]
```

🔍 Dry Run

$s='rabbbit', t='rabbit' \rightarrow 3$

🔍 Complexity Analysis

$O(m*n)$ time, $O(n)$ space.

🔍 Interview Tips

Draw the 2D dp table. Fill row by row. LCS is the foundation: many string DP problems reduce to LCS or a variant.

Problem 4: LC 583 — Delete Operation for Two Strings

LeetCode #583 · [leetcode.com/problems/](https://leetcode.com/problems/delete-operation-for-two-strings/)

🔍 Problem Understanding

Find minimum deletions to make both strings equal.

🔍 Pattern Recognition

This is STRING DP. Key signals: edit distance, LCS, palindrome. Usually $dp[i][j]$ represents state of two string prefixes.

🔍 Brute Force Approach

Recursion without memoization: exponential subproblems. DP with 2D table gives $O(m*n)$.

⚡ Optimized Solution

Answer = $(m - \text{LCS}) + (n - \text{LCS}) = m + n - 2 * \text{LCS}$.

🔍 Python Code

```
# LeetCode 583: Delete Operation for Two Strings
# Pattern: String DP / Sequence DP

m,n=len(word1),len(word2)
```

```

dp=[[0]*(n+1) for _ in range(m+1)]
for i in range(1,m+1):
    for j in range(1,n+1):
        if word1[i-1]==word2[j-1]: dp[i][j]=dp[i-1][j-1]+1
        else: dp[i][j]=max(dp[i-1][j],dp[i][j-1])
lcs=dp[m][n]
return m+n-2*lcs

```

🔗 Dry Run

word1='sea', word2='eat' → 2 (delete 's' and 't')

🔗 Complexity Analysis

$O(m*n)$ time, $O(m*n)$ space.

🔗 Interview Tips

Draw the 2D dp table. Fill row by row. LCS is the foundation: many string DP problems reduce to LCS or a variant.

Problem 5: LC 97 — Interleaving String

LeetCode #97 · leetcode.com/problems/

🔗 Problem Understanding

Check if s3 is formed by interleaving s1 and s2.

🔗 Pattern Recognition

This is STRING DP. Key signals: edit distance, LCS, palindrome. Usually $dp[i][j]$ represents state of two string prefixes.

🔗 Brute Force Approach

Recursion without memoization: exponential subproblems. DP with 2D table gives $O(m*n)$.

⚡ Optimized Solution

$dp[i][j]$ = can $s3[:i+j]$ be formed from $s1[:i]$ and $s2[:j]$.

🔗 Python Code

```

# LeetCode 97: Interleaving String
# Pattern: String DP / Sequence DP

m,n=len(s1),len(s2)
if m+n!=len(s3): return False
dp=[[False]*(n+1) for _ in range(m+1)]
dp[0][0]=True
for i in range(m+1):
    for j in range(n+1):
        if i>0: dp[i][j]=dp[i][j] or (dp[i-1][j] and s1[i-1]==s3[i+j-1])
        if j>0: dp[i][j]=dp[i][j] or (dp[i][j-1] and s2[j-1]==s3[i+j-1])
return dp[m][n]

```

🔗 Dry Run

s1='aabcc', s2='dbbca', s3='aadbcbcbcac' → True

🔖 Complexity Analysis

$O(m \cdot n)$ time, $O(m \cdot n)$ space.

🔖 Interview Tips

Draw the 2D dp table. Fill row by row. LCS is the foundation: many string DP problems reduce to LCS or a variant.

Problem 6: LC 1312 — Minimum Insertion Steps to Make a String Palindrome

LeetCode #1312 · leetcode.com/problems/

🔖 Problem Understanding

Find minimum characters to insert to make string a palindrome.

🔖 Pattern Recognition

This is STRING DP. Key signals: edit distance, LCS, palindrome. Usually $dp[i][j]$ represents state of two string prefixes.

🔖 Brute Force Approach

Recursion without memoization: exponential subproblems. DP with 2D table gives $O(m \cdot n)$.

⚡ Optimized Solution

Answer = n - LPS (Longest Palindromic Subsequence). LPS = $LCS(s, reversed(s))$.

🔖 Python Code

```
# LeetCode 1312: Minimum Insertion Steps to Make a String Palindrome
# Pattern: String DP / Sequence DP

n=len(s)
dp=[[0]*(n+1) for _ in range(n+1)]
rev=s[::-1]
for i in range(1,n+1):
    for j in range(1,n+1):
        if s[i-1]==rev[j-1]: dp[i][j]=dp[i-1][j-1]+1
        else: dp[i][j]=max(dp[i-1][j], dp[i][j-1])
lps=dp[n][n]
return n-lps
```

🔖 Dry Run

s='zzazz' → 0 (already palindrome); s='mbadm' → 2

🔖 Complexity Analysis

$O(n^2)$ time, $O(n^2)$ space.

🔖 Interview Tips

Draw the 2D dp table. Fill row by row. LCS is the foundation: many string DP problems reduce to LCS or a variant.

Pattern 30 Summary: String DP / Sequence DP

Core intuition: $dp[i][j]$ = answer for pair of prefixes. Diagonal (match), horizontal (skip s_2), vertical (skip s_1). Foundation: LCS $\rightarrow$ LPS, Edit Distance, Distinct Subsequences all build on it.

Quick Reference: Pattern Recognition Cheatsheet

Longest/shortest subarray/substring with condition → **Sliding Window**
Pair/triplet sum in sorted array → **Two Pointers**
Linked list cycle, middle, palindrome → **Fast/Slow Pointers**
Sorted array + $O(\log n)$ → **Binary Search**
Minimize max / maximize min → **Binary Search on Answer**
Count frequency, find complement → **Hashing**
Range sum query, count subarrays with sum = k → **Prefix Sum**
Bulk range update operations → **Difference Array**
Next greater/smaller element, histogram → **Monotonic Stack**
Sliding window max/min, optimize DP → **Monotonic Deque**
Top-K elements, dynamic ordering → **Heap**
Overlapping intervals, meeting rooms → **Intervals**
Minimum jumps, maximize reach, scheduling → **Greedy**
In-place linked list operations → **Linked List**
Path sum, diameter, invert tree → **Tree DFS**
Level order, right side view → **Tree BFS**
K-th smallest, validate, LCA in BST → **BST**
All permutations/combinations/subsets → **Backtracking**
Word search, N-Queens, palindrome partition → **Constrained Backtracking**
Connected components, flood fill → **Graph BFS/DFS**
Course scheduling, dependency order → **Topological Sort**
Dynamic connectivity, merge components → **Union Find**
Weighted shortest path → **Dijkstra/Shortest Path**
Minimum cost to connect all nodes → **MST**
Prefix search, autocomplete → **Trie**
XOR tricks, count bits, missing number → **Bit Manipulation**
Count ways to climb stairs, rob houses → **1D DP**
Select items with capacity constraint → **Knapsack DP**
Paths in 2D grid, maximal square → **Grid DP**
LCS, edit distance, palindrome → **String DP**